

Plataforma WEB Reservas de Hotel - StayNet

Gustavo Soares de Barros da Silva - a51848

Rafael da Costa Saraiva - a54946

Trabalho realizado sob a orientação de

Nuno Romeu

José Exposto

Licenciatura em Engenharia Informática

2025-2026

Plataforma WEB Reservas de Hotel - StayNet

Relatório da UC de Projeto
Licenciatura em Engenharia Informática
Escola Superior de Tecnologia e Gestão

Gustavo Silva – a51848

Rafael Saraiva – 54946

2025-2026

A Escola Superior de Tecnologia e Gestão não se responsabiliza pelas opiniões expressas neste relatório.

Declaro que o trabalho descrito neste relatório é da minha autoria e é da minha vontade que o mesmo seja submetido a avaliação.

Gustavo Silva – a51848

Rafael Saraiva – a54946

Dedicatória

Dedico este projeto a todos que, de forma direta contribuíram para a concretização deste projeto e para o meu crescimento pessoal e acadêmico.

A todos os professores do curso pois sem eles os conhecimentos desenvolvidos para a realização do mesmo foram determinantes.

E dedico também há minha família pois sem o apoio deles seria mais difícil.
(Gustavo Silva)

Este projeto é dedicado principalmente à minha família e aos amigos mais próximos, pelo apoio, incentivo e compreensão ao longo do meu percurso acadêmico. Sem a motivação e paciência, a realização deste projeto não teria sido possível. (Rafael Saraiva)

Agradecimentos

Ao chegar à fase final da Licenciatura em Engenharia Informática, considero importante deixar uma nota de agradecimento a todos os que, de alguma forma, contribuíram para que este projeto fosse possível e para que este percurso chegasse a bom porto.

Em primeiro lugar, agradeço aos meus orientadores, professor José Exposto e professor Nuno Romeu, pela orientação, disponibilidade e acompanhamento ao longo de todo o desenvolvimento. As suas recomendações, correções e perspetiva crítica foram decisivas para melhorar o trabalho e para ultrapassar os desafios que foram surgindo.

Quero também agradecer ao meu colega de projeto, Rafael, pelo compromisso e pela dedicação demonstrados ao longo deste processo. A cooperação, a troca de ideias e a ajuda constante fizeram a diferença, tanto na evolução do projeto como na forma como enfrentei os momentos mais exigentes.

Por último, deixo um agradecimento muito especial à minha família, em particular aos meus pais, pelo apoio incondicional, pela paciência e pela motivação permanente. Nos períodos de maior cansaço e pressão, foram o meu maior suporte e a principal razão para continuar até ao fim. (Gustavo Silva)

A realização deste projeto, marcou uma fase muito importante no meu percurso académico, pois foi aqui, onde tivemos a oportunidade de aplicar de forma prática, tudo aquilo que aprendemos ao longo da licenciatura, transformando conhecimento em experiência real.

Dito isto, gostaria de agradecer especialmente aos professores, Nuno Romeu Sequeira e José Exposto, pela orientação constante e pelos conselhos transmitidos, que foram imprescindíveis para a conclusão deste projeto.

Quero também dar um agradecimento especial aos amigos e colegas mais próximos que, direta ou indiretamente, ajudaram com sugestões e apoio ao longo deste projeto.

Agradeço também especialmente ao meu parceiro de projeto, Gustavo Silva, pela dedicação e empenho neste projeto, sem ele nada disto seria possível.

Por fim, um agradecimento especial, à minha família, pois, sem eles nada disto teria sido possível de alcançar. (Rafael Saraiva)

Resumo

O tema da Unidade Curricular de Projeto, determinante para a conclusão da licenciatura em Engenharia Informática, consiste no desenvolvimento de uma plataforma digital orientada a reservas de hotéis, com foco na simplicidade de utilização e na eficiência dos processos. O objetivo é disponibilizar uma solução única que sirva tanto quem procura reservar como quem precisa de gerir diariamente os alojamentos.

A plataforma foi desenvolvida para três perfis principais: utilizadores finais, gestores de hotéis e administradores da própria plataforma. Do lado do utilizador final, a StayNet permite procurar hotéis, consultar disponibilidade de quartos e efetuar reservas, incluindo sistema de autenticação para gestão de conta e consulta das reservas. Do lado dos gestores de hotéis, existe uma área de administração que suporta a gestão do alojamento e dos quartos, o acompanhamento de reservas e hóspedes, e o registo de operações como check-in e check-out, de forma centralizada e controlada.

A escolha deste tema resulta da observação do mercado, onde é comum encontrar soluções muito fortes para o cliente final, mas menos completas no que toca à gestão interna do hotel dentro da mesma plataforma. Assim, a StayNet procura aproximar estes dois lados, experiência do cliente e operação do hotel, num único sistema, promovendo maior consistência da informação e melhor controlo do processo.

Desta forma, o projeto materializa uma proposta prática e alinhada com necessidades atuais e realistas, aplicando conceitos de gestão de utilizadores, controlo de acessos, organização de dados e suporte a processos de negócio. Ao mesmo tempo, estabelece uma base sólida para evolução futura, com potencial para incluir integrações e funcionalidades adicionais à medida que os requisitos do contexto real se tornem mais exigentes.

Palavras-chave: Plataforma Web, Reservas de Hotel, Check-in e Check-out, Pagamentos.

Abstract

The subject of the Project Unit, crucial for completing the Computer Engineering degree, consists of developing a digital platform for hotel reservations, focusing on ease of use and process efficiency. The goal is to provide a single solution that caters to both those seeking to book and those needing to manage accommodation daily.

The platform was developed for three main profiles: end users, hotel managers, and platform administrators. For end users, StayNet allows searching for hotels, checking room availability, and making reservations, including an authentication system for account management and reservation tracking. For hotel managers, there is an administration area that supports the management of accommodations and rooms, tracking reservations and guests, and recording operations such as check-in and check-out, in a centralized and controlled manner.

The choice of this subject is a result of observing the market, where it is common to find very strong solutions for the end customer, but less complete when it comes to internal hotel management within the same platform. In this way, StayNet seeks to bring these two aspects, customer experience and hotel operations, closer together in a single system, promoting greater consistency of information and better process control.

In this way, the project materializes a practical proposal aligned with current and realistic needs, applying concepts of user management, access control, data organization, and business process support. At the same time, it establishes a solid foundation for future evolution, with the potential to include integrations and additional functionalities as the requirements of the real-world context become more demanding.

Keywords: Web platform, Hotel Reservations, Check-in and Check-out, Payments.

Conteúdo

Dedicatória	ix
Agradecimentos	x
Resumo	xii
Abstract.....	xiii
Conteúdo	xiv
Lista De Figuras	xviii
Lista de Código	xx
Siglas	xxii
Capítulo 1 Introdução	1
1.1 Enquadramento	2
1.2 Objetivos	3
1.3 Contribuições do trabalho	4
1.4 Estrutura do documento	5
Capítulo 2 Contexto e Tecnologias/Ferramentas	7
2.1 Análise do Problema	7
2.2 Estado de Arte	8
2.3 Gestão de Tecnologias/Linguagens	9
2.3.1 Justificação da Stack utilizada.....	10
2.4 Linguagens	11
2.4.1 TypeScript.....	11
2.4.2 SQL.....	11
2.4.3 HTML	11

2.4.4	CSS	12
2.5	Tecnologias	12
2.5.1	Next.js	12
2.5.2	Node.js	13
2.5.3	Neon Database	13
2.5.4	Cloudinary	13
2.5.5	Drizzle ORM	14
2.5.6	Docker	14
2.6	Bibliotecas de Software	15
2.6.1	VSCode (Visual Studio Code)	15
2.6.2	Git-Lab	15
2.7	Dependências	15
2.7.1	Dependências de Autenticação	15
2.7.2	Dependências de Estilos	15
2.7.3	Dependências de Pagamento	16
2.7.4	Dependências Adicionais	16
2.8	Conclusões e Reflexões	16
Capítulo 3	Abordagem/Análise/Modelação	18
3.1	Modelação	19
3.2	Figuras	20
3.2	Descrição da Arquitetura	21
3.3	Modelo de dados	23
3.4	Mockups	24
3.5	Metodologia e Organização de Recursos	31
Capítulo 4	Desenvolvimento/Implementação	33
4.1	Implementação do Frontend (Next.js)	34
4.1.1	Layout da Landing Page	34
4.1.2	Landing Page	35

4.1.3 Layout de Autenticação	38
4.1.4 Página de Login	39
4.1.5 Página de Registo	39
4.1.6 Página de Recuperar password	40
4.1.7 Dashboard de utilizador	41
4.1.8 Dashboard de Parceiro	52
4.1.9 Dashboard de Administrador	55
4.2 Implementação do Backend	58
4.2.1 Estrutura Geral da Plataforma	59
4.3 Criação da Base de Dados	61
4.4 Ligação á Base de Dados	65
4.5 Autenticação de Utilizadores e Gestão de Sessões	65
4.6 Rotas e API	69
4.7 Armazenamento Temporário	70
4.8 Envio de Emails Automático	72
4.9 Docker	74
4.10 Validação e Segurança	75
Capítulo 5 Testes/Avaliação/Discussão	78
5.1 Apresentação de Resultados	78
5.1.1 Resultados Página Inicial	79
5.1.2 Login	80
5.1.3 Registar	83
5.1.4 Área de utilizador	84
5.1.5 Área de Parceiro	89
5.1.6 Área de Administrador	94
Capítulo 6 Conclusões	97
Bibliografia	99

Apêndice A.....	1
Proposta Original de Projeto	2
Apêndice B. Conteúdos Adicionais	1
B.1 Componentes	1
B.1.1 Componentes de Autenticação	1
B.1.2 Componentes da Dashboard de Utilizador	7
B.1.3 Componentes da Dashboard de Parceiro	27
B.1.4 Componentes da Dashboard de Administrador	39
B.1.5 Componentes da Navbar e Footer	42
B.1.6 Componentes da visualização de Mapas.....	47

Lista De Figuras

Figura 1 - Diagrama de Casos de Uso.....	20
Figura 2 - Arquitetura da StayNet.....	21
Figura 3 - Modelo ER.....	24
Figura 4 - Página de Registo	26
Figura 5 - Página de Login.....	26
Figura 6 - Página Inicial StayNet	27
Figura 7 - Página Explorar Hotéis.....	28
Figura 8 - Página de Detalhes do Hotel.....	28
Figura 9 - Página de Favoritos	29
Figura 10 - Página de Reservas	29
Figura 11 - Página de Notificações	29
Figura 12 - Área do Parceiro	30
Figura 13 - Área do Administrador	31
Figura 14 - Estrutura da plataforma	61
Figura 15 - Resultados Página Inicial	74
Figura 16 - Página de Login com credenciais corretas	75
Figura 17 - Página de Login com credenciais incorretas	76
Figura 18 - Recuperar palavra-passe.....	76
Figura 19 - Email de Recuperação	77
Figura 20 - Conta criada com sucesso.....	78
Figura 21 - Email de verificação de conta.....	78
Figura 22 - Página de início com pesquisa.....	79
Figura 23 - Página de detalhes dos hotéis	80
Figura 24 Carrinho de Reservas	81
Figura 25 – Processo de Pagamento.....	81
Figura 26 - As minhas reservas	82
Figura 27 - Favoritos	82
Figura 28 - Candidatura a parceiro.....	83
Figura 29 - Página Quem Somos.....	83
Figura 30 - Ver conta	84
Figura 31 - Navbar do Parceiro	85

Figura 32 - Painel de Parceiro	85
Figura 33 - Adicionar Hotel	86
Figura 34 - Os meus hoteis.....	87
Figura 35 - Reservas dos meus Hoteis	87
Figura 36 - Lista de Hospedes.....	88
Figura 37 - Check-ins e Check-outs	88
Figura 38 - Painel de Admin	89
Figura 39 - Gestão de Parceiros	89
Figura 40 - Lista de Hotéis anunciados	90

Lista de Código

Código 1 - Layout da Landing Page.....	35
Código 2 - Landing Page.....	38
Código 3 - Layout de Autenticação.....	39
Código 4 - Página de Login.....	39
Código 5 - Página de Registo	40
Código 6 - Página de Recuperar Password	41
Código 7 - Layout Dashboard Utilizador	43
Código 8 - Página Inicial.....	45
Código 9 - Página de detalhes do Hotel	47
Código 10 - Página Carrinho.....	48
Código 11 Página Minhas Reservas.....	49
Código 12 - Página Favoritos.....	49
Código 13 - Página Torne-se Parceiro	49
Código 14 - Página Quem nós somos	50
Código 15 - Página a minha conta	51
Código 16 - Página Inicial Parceiro	53
Código 17 - Página Adicionar Hotel.....	53
Código 18 - Página Hotéis	54
Código 19 - Página Reservas	54
Código 20 - Página Hospedes	55
Código 21 - Página check-in e check-out.....	55
Código - 22 Página Inicial do Dashboard do Administrador	56
Código - 23 Página Parceiros	57
Código - 24 Página Hotéis	57
Código 25 - Código do Schema	65
Código 26 - Conexão Drizzle e Neon Data Base	65
Código 27 - Autenticação.....	68
Código 28 - Proxy	69
Código 29 - Stripe	70
Código 30 - Redis.....	72

Código 31 - Alterar e recuperar password	73
Código 32 - Email de verificação.....	73
Código 33 - Docker compose.....	75
Código 34 – Validações de formulários	77

Siglas

ESTiG Escola Superior de Tecnologia e Gestão.

API Application Programming Interface

HTML HyperText Markup Language

CSS Cascading Style Sheets

SQL Structured Query Language

VSCode Visual Studio Code

IA Inteligência Artificial

ER Entidade e Relacionamento

API Application Programming Interface

JS JavaScript

TSX TypeScript + JSX

ORM Object-Relational Mapping

Capítulo 1

Introdução

O setor do turismo e da hotelaria tem mudado muito nos últimos anos, impulsionado pela digitalização. Atualmente, é comum que grande parte das reservas seja feita online, onde o cliente consegue pesquisar alojamentos, comparar preços, ver disponibilidade e tratar do pagamento de forma simples.

Ainda assim, muitas dessas plataformas estão desenhadas sobretudo para quem reserva, deixando de lado as necessidades de quem gere o hotel no dia a dia. Na prática, isto pode obrigar as unidades hoteleiras a usar ferramentas separadas para controlar quartos, reservas e processos internos, o que aumenta trabalho manual e a necessidade de investir em sistemas próprios de gestão de reservas, que por vezes são dispendiosos. Por isso, torna-se importante apostar em soluções que aproximem estes dois lados, a experiência do cliente e a gestão interna, garantindo um controlo mais centralizado.

Dito isto, surge o tema da StayNet que irá preencher esta lacuna no mercado, ao unir num único sistema a experiência do utilizador final com a administração interna.

A StayNet foi criada com o sentido de ser uma plataforma integrada no campo de gestão hoteleira que facilita os utilizadores a realizar reservas com segurança, e é ampla para pessoas com qualquer familiaridade tecnológica pois é muito fácil de navegar na plataforma e de realizar qualquer processo.

Dado que a plataforma realiza tratamento de dados pessoais, foi dada uma atenção especial em relação à segurança, utilizando autenticação segura, criptografia de dados sensíveis, proteção dos processos de pagamento e controlos de acesso.

Este tipo de plataforma tem várias vantagens como já referidas, mas também agrega algumas desvantagens:

1. Dependência de tecnologia e conectividade.
 - É necessário o uso de internet para utilizar a plataforma.
 - Problemas de rede podem afetar o funcionamento.
2. Concorrência com plataformas já consolidadas internacionalmente.
 - Concorrência como o Booking, Airbnb e Trivago os quais já têm muita presença no mercado hoteleiro [1] [2] [3].
3. Riscos de falhas técnicas.
 - Erros no software podem afetar reservas e a própria gestão dos hotéis.
 - Requer monitorização constante e planos de recuperação.

Pensando num futuro próximo e na evolução da StayNet, foi usada uma arquitetura e tecnologias que permitem uma fácil evolução e escalabilidade, facilmente acrescentado novas funcionalidades e integrações adaptando-se às necessidades do mercado futuro.

1.1 Enquadramento

Na finalização da Licenciatura em Engenharia Informática no Instituto Politécnico de Bragança na Escola Superior de Tecnologia e Gestão (ESTiG), o Projeto Final de Curso é a etapa em que tudo o que foi aprendido ao longo do curso é aplicado. Por isso, a escolha do tema procurou juntar três aspetos: utilidade no mundo real, desafio técnico e a oportunidade de aplicar, de forma prática, os conhecimentos adquiridos.

Foi nesse contexto que surgiu a StayNet, uma plataforma web pensada para aproximar dois lados que muitas vezes funcionam separados: a experiência de quem reserva e a gestão diária de quem administra o hotel. O presente projeto enquadra-se no tema de Plataformas WEB (ver Apêndice A), uma vez que o objetivo passa pela criação de uma solução acessível via navegador, capaz de juntar reservas e a operação hoteleira de forma simples, organizada e com segurança.

1.2 Objetivos

O objetivo deste projeto é desenvolver a plataforma digital StayNet, tornando a gestão de reservas hoteleiras mais simples e organizada. Para isso, a solução pretende reunir num único sistema o processo de reserva e o acompanhamento da estadia (check-in e check-out), reduzindo trabalho manual, evitando erros de registo e melhorando a eficiência no dia a dia.

Objetivos específicos:

- Definir perfis de utilizador (cliente, gestor do hotel e administrador), garantindo que cada um acede apenas às funcionalidades adequadas às suas responsabilidades.
- Permitir ao cliente pesquisar hotéis, consultar disponibilidade e efetuar reservas de forma clara e intuitiva.
- Disponibilizar ao hotel uma área de gestão que facilite o controlo de unidades, quartos e reservas de modo centralizado.
- Suportar o registo e acompanhamento de check-ins e check-outs, mantendo o estado dos quartos atualizado e coerente com a operação.
- Assegurar o cumprimento das principais regras de negócio (disponibilidade, estados da reserva/estadia e prevenção de conflitos), mantendo a informação consistente ao longo do processo.
- Verificar o correto funcionamento do sistema através de testes e cenários representativos, cobrindo o fluxo completo desde a reserva até ao check-out.

1.3 Contribuições do trabalho

No presente Projeto Final foi desenvolvida a plataforma web StayNet, com o objetivo de integrar, num único sistema, a experiência do utilizador final e a gestão operacional de unidades hoteleiras. A solução centraliza processos de reserva e de acompanhamento da estadia, reduzindo tarefas manuais, minimizando erros de registo e reforçando a consistência da informação. As principais contribuições do trabalho incluem:

- **Plataforma Web integrada e funcional:**

Desenvolvimento de uma plataforma acessível via navegador, orientada para a gestão de reservas e para a operação hoteleira, garantindo uma experiência de utilização clara e intuitiva para diferentes perfis.

- **Fluxo completo de reservas e estadias:**

Implementação do ciclo principal de funcionamento do sistema, desde a pesquisa/consulta e criação de reservas até ao pagamento das mesmas, assegurando o acompanhamento das operações ao longo do tempo.

- **Estrutura com perfis de acesso diferenciados:**

Foi criada uma arquitetura com diferentes tipos de utilizadores (cliente, gestor do hotel e administrador), garantindo controlo de acessos, melhor organização das funcionalidades e maior segurança na utilização da plataforma.

- **Gestão centralizada para unidades hoteleiras:**

Criação de uma área de administração para os hotéis, permitindo gerir hotéis, quartos, reservas e o estado de ocupação.

- **Regras de negócio e consistência da informação:**

Definição e aplicação de regras básicas para gerir a disponibilidade dos quartos, os estados das reservas e das estadias, evitando conflitos e garantindo que a informação apresentada é correta.

- **Estrutura pensada para futuras melhorias:**

Estruturação da plataforma de forma a permitir evoluções futuras.

Estas contribuições demonstra um esforço centrado na criação de uma solução prática e alinhada com necessidades reais do setor hoteleiro, combinando aspetos técnicos (arquitetura, dados, segurança) com uma preocupação clara pela experiência do utilizador e pela eficiência das operações.

1.4 Estrutura do documento

Para garantir uma leitura clara e um acompanhamento lógico de todas as fases do desenvolvimento, o relatório encontra-se organizado da seguinte forma:

- **Capítulo 1 – Introdução:** Apresenta-se o enquadramento do projeto, a motivação, os objetivos e as principais contribuições, contextualizando a necessidade de uma plataforma que una reservas e gestão operacional em contexto hoteleiro.
- **Capítulo 2 – Contexto e Tecnologias/Ferramentas:** Descreve-se o domínio do problema e apresentam-se as tecnologias e ferramentas utilizadas no desenvolvimento da StayNet, justificando as opções tomadas.
- **Capítulo 3 – Análise e Modelação:** Apresenta-se o levantamento e análise de requisitos, bem como a modelação do sistema, incluindo casos de uso, regras de negócio e modelos/diagramas que suportam a solução.

- **Capítulo 4 – Arquitetura e Implementação:** Detalha-se a arquitetura da plataforma e a implementação das funcionalidades principais, incluindo a organização do projeto, backend/frontend e estrutura da base de dados.
- **Capítulo 5 – Testes e Avaliação:** Descrevem-se os testes realizados e os resultados obtidos, avaliando o funcionamento do sistema e o cumprimento dos objetivos definidos para os diferentes perfis de utilizador.
- **Capítulo 6 – Conclusões e Trabalho Futuro:** Apresentam-se as conclusões, as aprendizagens e limitações identificadas, bem como possíveis evoluções futuras da plataforma.
- **Apêndice A – Proposta do Projeto:** Inclui-se a proposta original submetida para aprovação do tema.
- **Apêndice B – Excertos de Código/Material de Suporte:** Reúnem-se excertos de código e outros elementos relevantes que complementam o desenvolvimento apresentado.

Capítulo 2

Contexto e Tecnologias/Ferramentas

2.1 Análise do Problema

Observámos que a maioria dos hotéis, dependem muito da internet, principalmente na interação com o cliente final. Atualmente todas as plataformas de reservas online mais conhecidas, permitem realizar reservas online de forma rápida e segura, tornando o processo do utilizador final muito simples. No entanto, estas plataformas apenas abrangem a experiência do utilizador final, deixando de parte as necessidades de gestão interna dos hotéis.

Assim sendo, muitos hotéis acabam por recorrer a várias ferramentas em simultâneo para gerir reservas, quartos, hóspedes, pagamentos e tarefas diárias. Esta utilização de várias ferramentas leva facilmente a problemas como duplicação de informação, maior probabilidade de erro e diminui a eficiência das operações. Para hotéis de menor dimensão, que muitas das vezes não dispõem de equipas dedicadas nem de grande poder financeiro, esta realidade torna-se ainda mais crítica.

A falta de uma solução no mercado, acaba por afetar a eficiência dos hotéis e pode ter impacto negativo na experiência do cliente, especialmente através de falhas na comunicação, erros na gestão de disponibilidades ou atrasos nos processos de reserva.

Nesta situação, identificámos como problema principal a falta de uma plataforma que una, as reservas realizadas pelo cliente e a gestão interna do hotel. Torna-se, assim, necessária uma solução que utilize apenas um sistema, que simplifique o trabalho diário dos hotéis.

2.2 Estado de Arte

Com a digitalização crescente do setor do turismo, tornou-se natural que grande parte das reservas de alojamento passe a ser feita online. Hoje, o utilizador espera conseguir procurar opções, ver disponibilidade e concluir a reserva com pagamento digital, de forma rápida e conveniente. Do lado dos hotéis, esta evolução também aumentou a necessidade de ferramentas que ajudem a manter a operação organizada e a informação sempre atualizada, desde a reserva até ao final da estadia.

Apesar de existirem várias plataformas no mercado, nem todas conseguem garantir, de forma simples e consistente, que a disponibilidade apresentada corresponde à realidade quando a reserva é finalizada. Um dos problemas mais comuns surge quando vários utilizadores tentam reservar o mesmo quarto para as mesmas datas, o que pode levar a conflitos e incoerências se o sistema não tiver validações robustas. Por isso, soluções mais fiáveis costumam reforçar o controlo de disponibilidade em momentos críticos do processo, especialmente quando a reserva está prestes a ser confirmada.

Neste contexto, a integração do pagamento no fluxo de reserva também tem um papel importante. A confirmação final da reserva deve acontecer apenas quando o pagamento é concluído com sucesso, garantindo que o estado da reserva e a disponibilidade permanecem alinhados. É precisamente por essa razão que faz sentido validar novamente a disponibilidade no momento do pagamento, reduzindo a probabilidade de reservas inválidas e evitando problemas na operação do hotel.

Tendo por base estas necessidades, a StayNet foi concebida para aproximar a experiência do cliente e a gestão interna do hotel no mesmo sistema. Para reforçar a consistência e a confiança no processo, a plataforma aplica verificações de disponibilidade em duas fases: uma ao nível do estado da reserva e outra no momento do pagamento, assegurando que a reserva só é finalizada quando continua válida.

2.3 Gestão de Tecnologias/Linguagens

As tecnologias foram escolhidas com o objetivo de responder às necessidades do projeto e pensando já na sua futura evolução, demos preferência a tecnologias atuais, e às mais utilizadas no mercado, adequadas ao desenvolvimento de aplicações web escaláveis.

Foi utilizado o Next.js como uma framework full stack, permitindo o desenvolvimento da interface do utilizador e da lógica de servidor na mesma aplicação, através da utilização de Server Actions e APIs [4].

Para o funcionamento do Next.js, foi também necessário o Node.js, que funcionou como ambiente de execução no servidor, permitindo que o Next.js processasse o código no backend e executasse a lógica de forma eficiente [5].

Para a armazenagem de dados, recorreu-se à base de dados, Neon Database, A interação com a base de dados é efetuada através do Drizzle ORM, facilitando a gestão da mesma. Também recorreremos ao uso de uma segunda base de dados, a Cloudinary, onde armazenámos exclusivamente todas as imagens da plataforma [6] [7] [8].

De forma a assegurar a acessibilidade em diferentes sistemas operativos, e a pensar numa execução e publicação da aplicação mais rápida, foi utilizado o Docker. Esta ferramenta permitiu isolar e gerir os vários componentes do sistema, como o componente Redis, que foi usado neste projeto como serviço de cache [9].

Em conjunto, esta stack tecnológica, permite desenvolver uma plataforma segura, escalável e funcional, preparada para evolução futura.

2.3.1 Justificação da Stack utilizada

Decidimos optar por estas tecnologias não só pelas suas características técnicas, mas também por serem amplamente utilizadas no desenvolvimento de aplicações web por grandes empresas.

Optámos pelo Next.js devido à sua simplicidade, elevado desempenho e tempos de carregamento muito reduzidos e da sua segurança podendo usar a renderização no lado do servidor sem expor dados sensíveis. Além disso, o Next.js possui uma grande quantidade de bibliotecas e pacotes, o que permite adicionar facilmente novas funcionalidades, todas com a sua documentação de como pode ser feito.

Quanto á escolha das bases de dados, decidimos optar pelo Neon Database e o Cloudinary, pelo facto de ambas serem soluções serveless(sem servidor local), ou seja, são base de dados hospedadas na cloud.

Decidimos escolher o Drizzle ORM, pela necessidade de simplificar a gestão da base de dados. Permitindo-nos consultar e escrever na base de dados com mais segurança e simplicidade. Esta escolha permitiu-nos facilitar o desenvolvimento e a manutenção da aplicação.

O uso do Docker também foi uma escolha importante, pois permite executar o projeto em qualquer sistema, e simplifica a configuração inicial, tanto em ambiente de desenvolvimento como em ambiente de produção.

Em conjunto, esta stack tecnológica Next.js, Neon Database, Drizzle ORM, Cloudinary e Docker, garantiu o desenvolvimento simples de uma plataforma segura e escalável.

2.4 Linguagens

No desenvolvimento desta plataforma usámos várias linguagens de programação, tais como:

- TypeScript
- SQL
- HTML
- CSS

2.4.1 TypeScript

O TypeScript foi utilizado em toda a aplicação, tanto no frontend como na lógica do backend através do Next.js. Na parte do frontend foi utilizado para criar os componentes e a interface que o utilizador visualiza, e na parte do backend foi usado para criar as Server Actions e as APIs. Decidimos utilizar o TypeScript em vez do JavaScript devido à sua tipagem que permite detetar os erros antes da execução do código [\[10\]](#).

2.4.2 SQL

No projeto o SQL foi utilizado na Neon database e na Cloudinary com a ajuda do Drizzle ORM. Esta linguagem foi o que permitiu criar, consultar, atualizar e eliminar dados da base de dados.

2.4.3 HTML

O HTML (HyperText Markup Language) é a linguagem de marcação com o propósito de estruturar páginas web. É utilizada principalmente para organizar texto, imagens, links e formulários numa página.

No projeto, o HTML foi utilizado para todas as páginas, adicionando a estrutura necessária para mostrar o conteúdo de forma consistente.

2.4.4 CSS

O CSS (Cascading Style Sheets) é a linguagem utilizada para declarar os estilos das páginas web, incluindo cores, fontes, espaçamentos e responsividade. No projeto não utilizamos CSS puro, pois recorreremos ao Tailwind CSS. Tailwind CSS é uma framework de CSS, que permite a utilização dos CSS diretamente nos componentes, assim conseguindo acelerar o processo de desenvolvimento das aplicações.

2.5 Tecnologias

2.5.1 Next.js

O Next.js é uma Framework full stack baseada em React, a qual utilizamos para desenvolver o frontend e parte do backend.

Esta framework foi utilizada para a criação das páginas web, aproveitando da sua vantagem de criação de componentes reutilizáveis, o que possibilita construir a plataforma de forma modular e organizada.

Após isso estes componentes podem ser apresentados de forma dinâmica, apresentados no lado do cliente ou no lado do servidor dependendo da sua função, o que torna o código muito mais seguro, organizado e com tempos de carregamento de páginas mais reduzidos em comparação com outras tecnologias.

O Next.js permite criar as funções de backend diretamente na aplicação, utilizando Server Actions e APIs, sem ser necessário usar um servidor separado, simplificando o desenvolvimento. Além disso, também permite criar rotas automáticas com base na sua estrutura de pastas, o que poupa tempo de desenvolvimento e mantém uma estrutura organizada e fácil de escalar futuramente.

Outra grande vantagem, é a diversidade de bibliotecas muitas das quais foram utilizadas neste projeto que nos permitiu integrar várias funcionalidades de forma rápida e eficiente, como a autenticação, integração de mapas interativos, entre outras.

2.5.2 Node.js

Neste projeto o Node.js foi essencial, pois o Next.js necessita de um ambiente de execução no servidor para executar as funções do backend, gerir Server Actions e APIs. Garantindo o bom funcionamento da plataforma.

O gestor de pacotes nativo do Node.js, também foi essencial para gerir as dependências da plataforma, pois permitiu instalar, atualizar e organizar todas as bibliotecas necessárias ao funcionamento da aplicação.

2.5.3 Neon Database

O Neon Database foi escolhido como solução de armazenamento principal para a nossa plataforma devido à sua estrutura serverless, ou seja, toda a base de dados é gerida na cloud, o que evitou a necessidade de configuração de servidores locais e adicionou uma camada extra de segurança aos dados sensíveis da plataforma.

Decidimos integrar o Drizzle-ORM para ajudar na gestão de todas as tabelas da aplicação, esta combinação permitiu simplificar a criação, consulta, atualização e eliminação de dados, enquanto mantém a segurança e organização dos dados.

2.5.4 Cloudinary

A Cloudinary é um serviço de armazenamento, como a Neon, esta também é serverless, ou seja, é gerida na cloud. Esta base de dados é especialmente desenvolvida para a gestão e entrega de conteúdos multimédia, garantindo carregamentos rápidos e compatibilidade com diversos dispositivos e tamanhos de ecrã.

Esta base de dados foi implementada no nosso projeto para prevenir a sobrecarga da base de dados principal, e para prevenir possíveis erros de compatibilidade de imagens. Esta integração também simplifica o redimensionamento, compressão e conversão de formatos, sem necessidade de intervenção manual ou armazenamento local.

O Drizzle ORM tal como na Neon Database, também foi utilizado para ajudar a gerir as tabelas das imagens e as suas devidas relações, permitindo que a ligação entre cada imagem e o respetivo hotel seja feita de forma segura.

2.5.5 Drizzle ORM

O Drizzle ORM é uma biblioteca que permite interagir com bases de dados SQL de forma simples e organizada, esta ferramenta permite a criação, gestão e acesso das tabelas, tornando o desenvolvimento mais rápido.

Nesta plataforma foi utilizado como camada de gestão entre a aplicação e a bases de dados Neon e Cloudinary, o que permitiu a criação de entidades tais como, utilizadores, hotéis, quartos, reservas, imagens, candidaturas a parceiros e os tokens de verificação, recorrendo a enums, chaves primárias, chaves estrangeiras e relações entre tabelas.

Adicionalmente, o Drizzle disponibiliza de um dashboard local onde podemos monitorizar todas as tabelas e os dados de cada uma.

2.5.6 Docker

O Docker é uma ferramenta open-source, ou seja, de código aberto, que permite criar ambientes virtuais divididos em contentores, assim podendo isolar cada serviço, o que adiciona uma camada extra de segurança e organização.

Nesta plataforma decidimos implementar o Docker para gerir todos os seus serviços, como o ambiente de execução da aplicação, o Redis e o serviço do Stripe. Além disso o que mais nos levou a integrar esta tecnologia, foi a compatibilidade com todos os sistemas operativos, permitindo replicar o projeto rapidamente em qualquer computador ou servidor, quer seja em ambiente de desenvolvimento ou futuramente em produção.

2.6 Bibliotecas de Software

2.6.1 VSCode (Visual Studio Code)

O Visual Studio Code (VS Code) é um editor de código da Microsoft, disponível para Windows, Linux, macOS e também no navegador.

Foi usado neste projeto por ser leve, fácil de configurar e por oferecer tudo o que era necessário para o desenvolvimento do projeto, como suporte ao Git e extensões que ajudam a programar de forma mais rápida e organizada.

2.6.2 Git-Lab

O GitLab é uma plataforma de gestão de repositórios Git, que permite guardar e versionar o código, mantendo histórico de alterações e facilitando o trabalho colaborativo.

No desenvolvimento da StayNet, foi utilizado o GitLab disponibilizado pela ESTiG (Escola Superior de Tecnologia e Gestão) como repositório central do projeto, para organizar o desenvolvimento, gerir versões e integrar práticas de colaboração como branches e merge requests, garantindo maior controlo e acompanhamento das mudanças ao longo do tempo.

2.7 Dependências

2.7.1 Dependências de Autenticação

- **Auth.js** - Permitiu gerir autenticação e sessões de forma segura;
- **Bcrypt.js** - Serviu para encriptar passwords para proteger os dados dos utilizadores.

2.7.2 Dependências de Estilos

- **Tailwind CSS** - Implementado para criar e adicionar estilos aos componentes;
- **Shadcn/ui** - Foi utilizado para adicionar componentes pré-construídos, e acelerar o desenvolvimento [11].

2.7.3 Dependências de Pagamento

- **Stripe** - Adicionado para tratar e processar os pagamentos [12].

2.7.4 Dependências Adicionais

- **Openstreetmap** – Permite interagir com mapas, e mostrar localizações [13].
- **Resend** – Adiciona a funcionalidade de enviar emails [14].
- **Redis** – Permite armazenar os dados temporários [15].

2.8 Conclusões e Reflexões

A análise do estado da arte e das tecnologias usadas em plataformas de reservas ajudou-nos a perceber o que já existe no setor hoteleiro e onde ainda há falhas, sobretudo na ligação entre o lado do cliente (reservar) e o lado do hotel (gerir a operação). Foi exatamente essa lacuna que orientou o desenvolvimento da StayNet, pensada para juntar num só sistema o processo de reserva e o acompanhamento da estadia, incluindo check-in e check-out, com informação mais organizada e fiável.

Ao longo do projeto, o levantamento de requisitos e a observação de soluções semelhantes foram essenciais para definir as funcionalidades e as regras de negócio, especialmente no que toca à disponibilidade e ao controlo do estado das reservas. Como o pagamento é um momento crítico, a plataforma foi desenvolvida para validar a disponibilidade em mais do que uma etapa, reduzindo o risco de inconsistências e garantindo que a reserva só é concluída quando faz sentido.

Em termos técnicos, a escolha do Next.js facilitou a criação de uma aplicação moderna e bem estruturada, e a utilização de base de dados principal Neon Database, com o Drizzle ORM ajudou a manter o acesso aos dados mais seguro e consistente. No final, o projeto cumpriu o objetivo principal: criar uma solução funcional que suporta o fluxo essencial de um hotel, e que fica com base preparada para crescer no futuro com

funcionalidades como integração de uma API para conectar com o sistema de gestão dos próprios hotéis, ou até mesmo adicionar um bot de inteligência artificial para ajudar na questão de suporte e apoio ao cliente, relatórios de gestão e melhorias na parte financeira e operacional.

Capítulo 3

Abordagem/Análise/Modelação

Atualmente, na hotelaria, quase tudo passa por sistemas digitais: o cliente quer reservar rápido e sem dúvidas, e o hotel precisa de ter a operação controlada sem andar a confirmar tudo manualmente. Quando isso não existe (ou está mal feito), aparecem problemas típicos: reservas duplicadas, disponibilidade errada, falhas na comunicação e muito trabalho extra para a receção.

Foi a pensar nisso que surgiu a StayNet. A ideia do projeto é ter uma plataforma simples e organizada que trate do essencial: permitir ao cliente ver quartos disponíveis, escolher datas (check-in e check-out) e fazer a reserva, enquanto do lado do hotel fica tudo registado e controlado, reservas, ocupação e gestão do sistema, de forma clara e consistente.

Para chegar a essa solução, a primeira etapa foi analisar e modelar o sistema: perceber quem vai usar (visitante, utilizador, administrador ou parceiro), o que cada um pode fazer, e como a informação deve ser guardada para não dar confusão. Esta base é importante porque é o que garante que, mais à frente, o sistema funcione como deve ser e pode crescer com novas funcionalidades sem alterar a estrutura.

3.1 Modelação

A modelação foi uma das fases mais importantes no desenvolvimento da StayNet, porque foi aqui que definimos com clareza como a plataforma ia funcionar antes de começar a implementar. Esta etapa ajudou a evitar confusões e retrabalho, sobretudo em partes críticas como reservas, confirmação de pedidos e pagamento integrado.

Numa primeira fase, foi feito o levantamento e a organização dos requisitos da plataforma, identificando o que cada perfil pode fazer no sistema: Visitante, Utilizador, Administrador e Parceiro. Estes requisitos serviram de base para estruturar as funcionalidades principais e garantir que as permissões e responsabilidades de cada tipo de utilizador faziam sentido no contexto da aplicação.

De seguida, foi elaborado o diagrama de casos de uso, no visual paradigm, para representar de forma simples as interações entre os utilizadores e a StayNet. Nesta representação ficaram definidos os processos essenciais, como consultar disponibilidade, efetuar reservas, realizar o pagamento e acompanhar/gerir reservas, bem como as ações do Parceiro, que é responsável pela gestão do hotel na plataforma, incluindo confirmar pedidos e tratar operações de check-in e check-out [16].

Por fim, foram desenvolvidos Mockups no Figma, permitindo validar a estrutura das páginas e os fluxos de navegação antes da implementação. O Figma é uma ferramenta colaborativa de design e prototipagem, o que ajudou a desenvolver a interface com os requisitos definidos e manter consistência ao longo do desenvolvimento [17].

3.2 Figuras

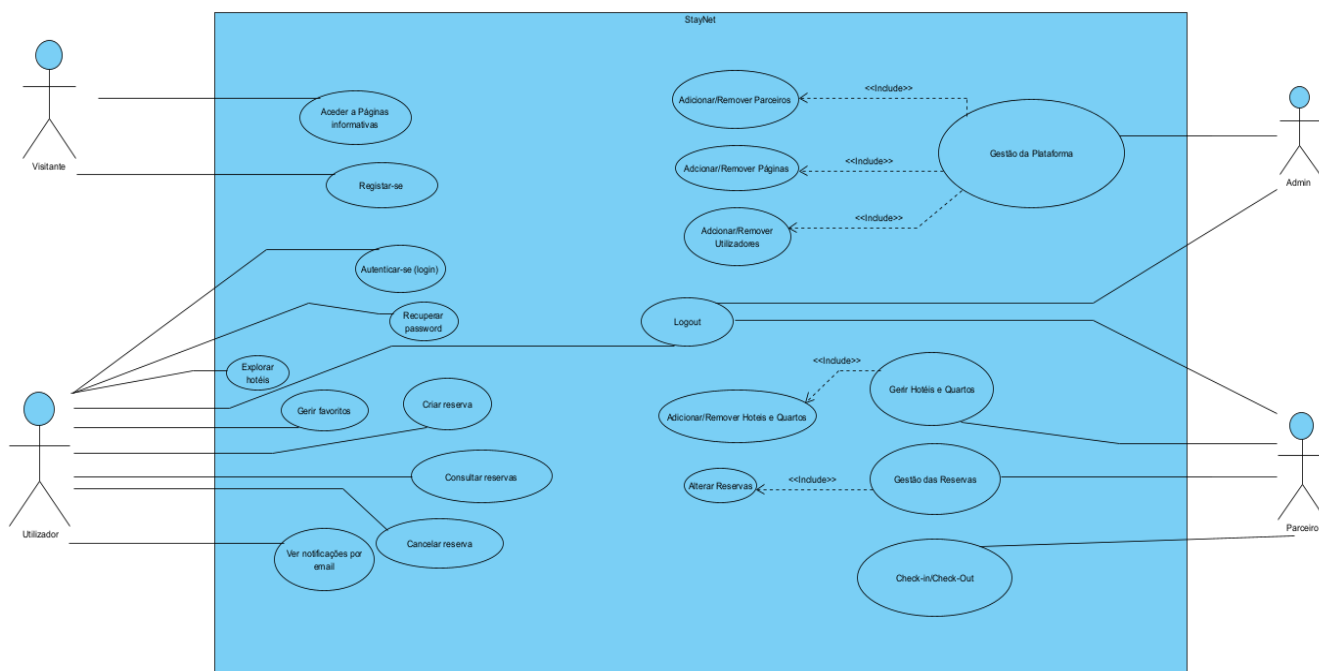


Figura 1 - Diagrama de Casos de Uso

Visitante: é um utilizador sem conta e não logado, podendo, por isso, ter acesso apenas às páginas informativas, é um utilizador limitado na plataforma.

Utilizador: é um utilizador registado e autenticado, podendo pesquisar, seleccionar datas, efetuar reservas e realizar o pagamento integrado. Além disso, consegue consultar e acompanhar o estado das suas reservas e gerir dados básicos da sua conta.

Parceiro: é o utilizador responsável por gerir os hotéis dentro da plataforma, podendo criar/editar informação do hotel, gerir quartos (preços, quantidade e descrições) e acompanhar pedidos de reserva. Também pode confirmar pedidos e tratar operações relacionadas com a estadia, como check-in e check-out.

Administrador: é o perfil com permissões mais amplas, responsável pela gestão global da plataforma. Pode supervisionar utilizadores e parceiros, controlar conteúdos/dados do sistema e garantir que a plataforma funcione de forma correta e segura.

3.2 Descrição da Arquitetura

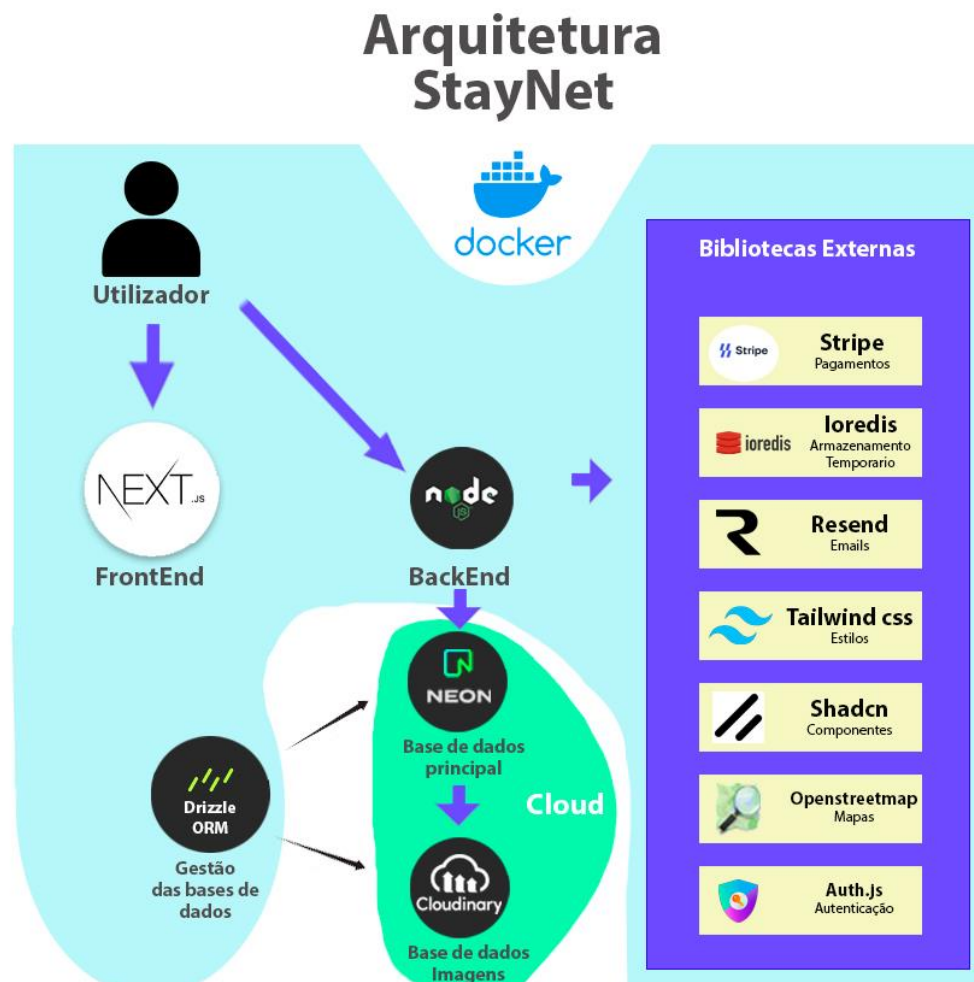


Figura 2 - Arquitetura da StayNet

A Figura 2 representa a arquitetura da StayNet que foi desenvolvida com o objetivo de assegurar escalabilidade, organização e acelerar o processo desenvolvimento, recorrendo a ferramentas modernas e profissionais.

A Plataforma foi desenvolvida em Next.js, agindo como uma solução full stack, que desenvolve a interface do utilizador e a lógica de backend. O node.js, nesta arquitetura é utilizado apenas como ambiente de execução, tornando o responsável pela gestão e processamento das Server Actions e das APIs, permitindo a comunicação entre o frontend e os serviços externos com segurança.

Para garantir consistência e compatibilidade com todos os sistemas quer em desenvolvimento quer em produção, decidimos usar o Docker. Através do Docker são geridos a partir de contentores os serviços essenciais como a aplicação Next.js, o serviço

Redis, utilizado como mecanismo de armazenamento temporário, o Drizzle Kit, usado para a gestão e visualização da base de dados através de uma dashboard local e por fim o Stripe, usado para tratar de todos os processos e lógica de pagamento.

Esta arquitetura recorre à utilização de duas bases de dados serverless, ou seja, são geridas na cloud [18]. Como base de dados principal utilizámos a Neon Database, esta trata e armazena toda a lógica e dados principais do servidor. Como base de dados secundária, optámos pela Cloudinary, esta base de dados foi desenvolvida especificamente para a organização e armazenamento de imagens. Esta escolha revelou-se adequada às necessidades da plataforma, devido á elevada quantidade de fotos de cada hotel, garantindo melhor desempenho, escalabilidade e otimização no carregamento das imagens.

De forma a melhorar a experiência do utilizador, também foram usadas algumas bibliotecas externas como o Stripe e o Redis que já foram mencionados. Além destes foram utilizados:

- **Resend** – Esta biblioteca permitiu-nos tratar de todos os envios de emails da plataforma, especialmente emails de confirmação de conta, recuperação de palavra-passe e notificações ao utilizador.
- **Tailwind CSS** – Esta biblioteca é uma biblioteca muito conhecida e utilizada na estilização dos componentes, pois permite acelerar o processo de desenvolvimento [19].
- **Shadcn** – É uma biblioteca que disponibiliza componentes pré feitos, que são facilmente alterados para funcionarem em sintonia com o projeto.
- **OpenStreetMap** – Como alternativa gratuita ao google maps, decidimos utilizar o OpenStreetMap para visualização e localização dos hotéis na plataforma. Esta solução é de código aberto e dispensa da necessidade de pagamento.
- **Auth.js** - Esta biblioteca foi utilizada para gerir o sistema de autenticação e as sessões da plataforma. O Auth.js permite gerir de forma segura o início de sessão, registo e persistência do utilizador autenticado [20].

3.3 Modelo de dados

Para a modelação da base de dados da plataforma StayNet, foi elaborado um modelo Entidade-Relacionamento (ER), com o objetivo de representar de forma clara e estruturada a informação necessária ao funcionamento da aplicação. Este modelo permitiu organizar e planear os dados envolvidos, facilitando a implementação numa base de dados relacional PostgreSQL (Neon), garantindo consistência e suporte adequado às operações principais da plataforma.

A construção do modelo ER revelou-se fundamental para identificar as entidades centrais do sistema e as respetivas relações, assegurando que as funcionalidades e os fluxos definidos (por exemplo, gestão de hotéis, disponibilização de quartos, processo de reserva e pagamento) estavam corretamente refletidos ao nível da estrutura de dados. Através desta modelação, foi possível confirmar que as interações previstas para cada perfil (Visitante, Utilizador, Parceiro e Administrador) eram suportadas por relações e restrições adequadas na base de dados.

Neste contexto, identificaram-se como entidades principais da StayNet o Utilizador, o Hotel, o Quarto, a Reserva e as Imagens do Hotel, complementadas por entidades de apoio como a Candidatura a Parceiro e os registos associados à verificação/validação (por exemplo, tokens de verificação).

Além disso, a plataforma inclui mecanismos como favoritos e carrinho, que permitem ao utilizador assinalar hotéis que pretende guardar para mais tarde e reunir/organizar a reserva antes da confirmação e do pagamento, tornando o processo mais simples e estruturado.

A Figura 3 apresenta o modelo ER completo da base de dados da StayNet, representando visualmente os relacionamentos entre as diferentes entidades, com destaque para a gestão de hotéis pelos parceiros, a associação entre hotéis e quartos, o registo de reservas por utilizadores e a componente de apoio ao processo de reserva (carrinho, favoritos e validações associadas ao pagamento).

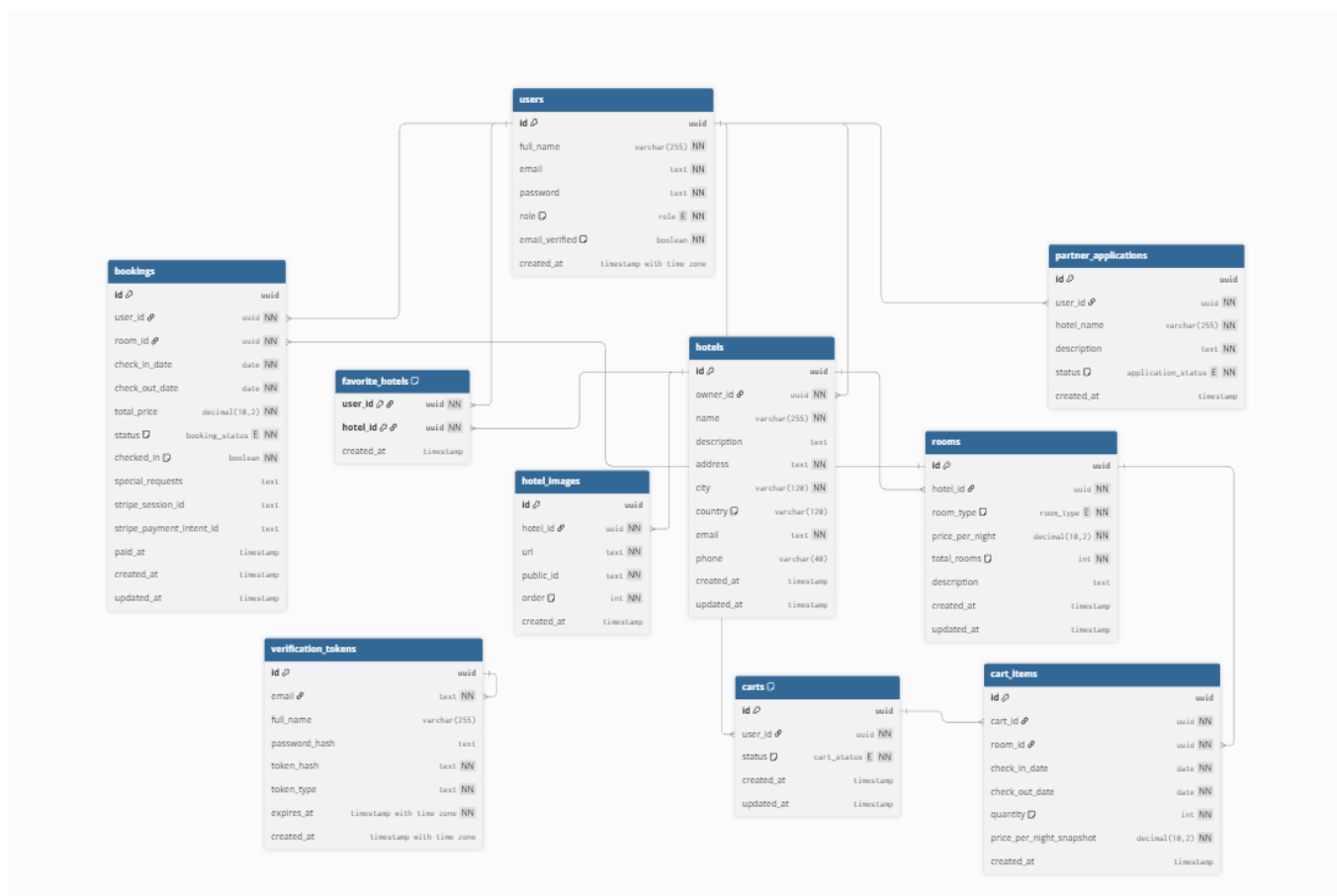


Figura 3 - Modelo ER

3.4 Mockups

Com o objetivo de garantir uma visão clara da interface da plataforma digital StayNet, foram desenvolvidos vários mockups que representam as principais páginas e fluxos do sistema. A ferramenta escolhida para a criação dos mockups foi o Figma, pela sua facilidade de utilização, colaboração em equipa e por permitir estruturar e validar percursos de navegação de forma consistente.

Apesar de se tratar de representações visuais estáticas, estes mockups permitiram antecipar decisões de design, organizar a disposição dos elementos e alinhar as funcionalidades com os requisitos definidos. Foram criados mockups para algumas das páginas principais da plataforma, nomeadamente:

- Página de Registo
- Página de Login
- Página Inicial StayNet
- Página Explorar Hotéis
- Página de detalhes do Hotel
- Página de Reservas
- Página de Favoritos
- Página de Notificações
- Área do Parceiro
- Área do Administrador

Cada mockup é apresentado individualmente nas figuras seguintes, funcionando como referência visual e apoio à implementação da interface ao longo do desenvolvimento.

STAYNET

Cria agora a tua conta StayNet

Email
Introduz o teu email

Nome completo
Introduz o teu Nome

Palavra-Passe
Introduz a tua palavra-passe

Volta a introduzir a tua palavra-passe

Registrar

Já tenho conta! [Inicia sessão.](#)

Tem um hotel e quer começar a usar a StayNet?
Preencha o nosso formulário e entraremos em contacto para ativar a sua conta empresarial! [Aderir como hotel](#)

Figura 4 - Página de Registo



**Bem-vindo(a) de volta à
StayNet 🙌**

Email

Palavra-Passe

LOGIN

[Esqueci-me da palavra-passe?](#)
Ainda não tens conta? [Cria uma conta](#)

Figura 5 - Página de Login



Figura 6 - Página Inicial StayNet

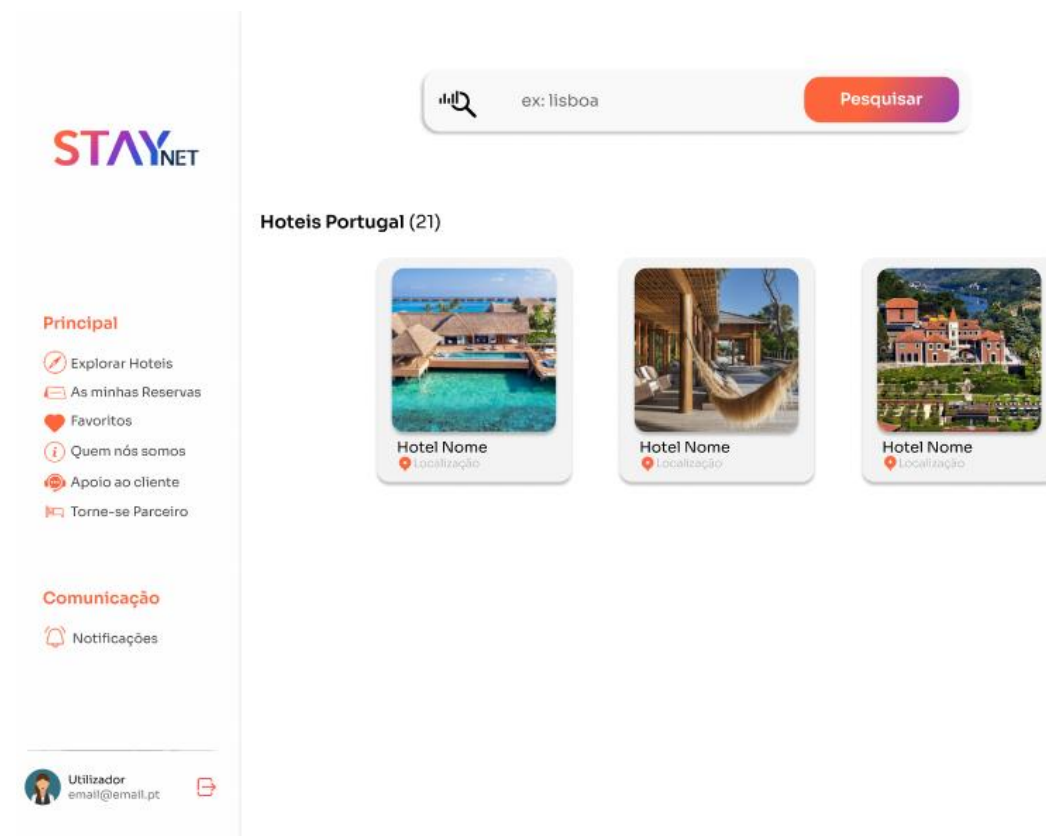


Figura 7 - Página Explorar Hotéis

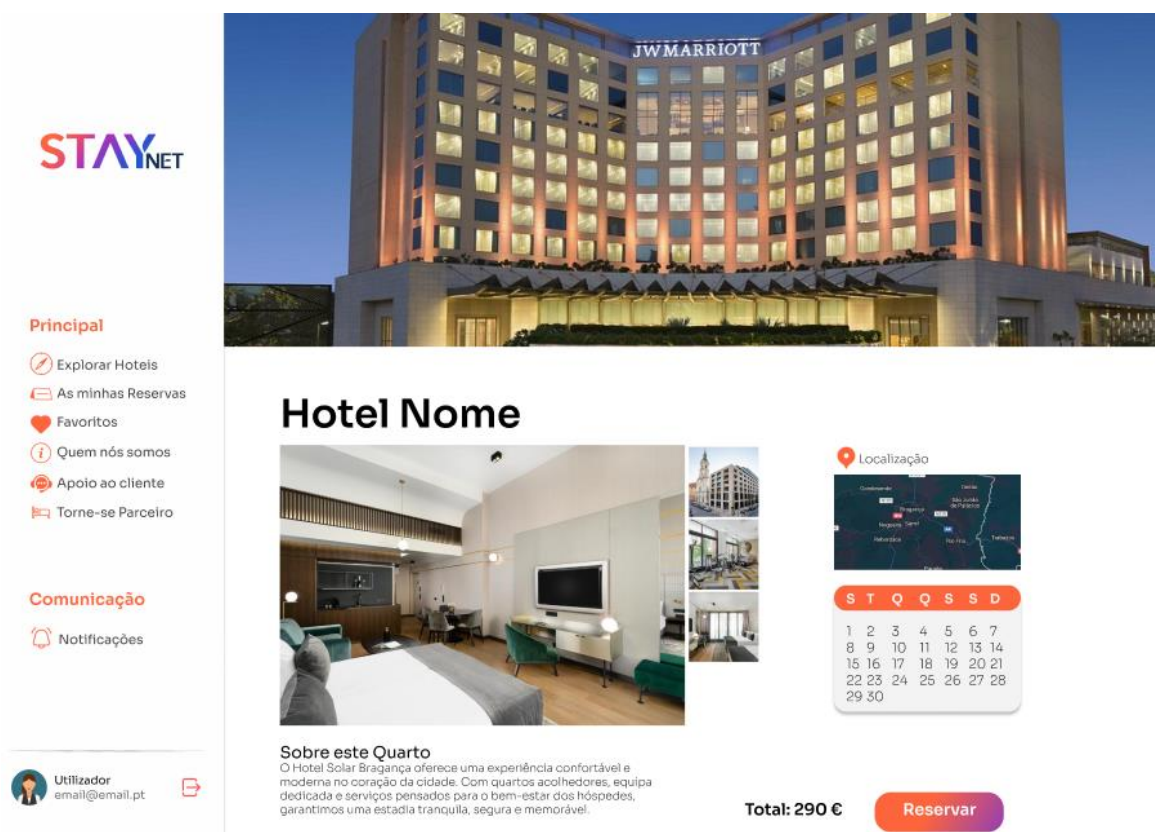


Figura 8 - Página de Detalhes do Hotel

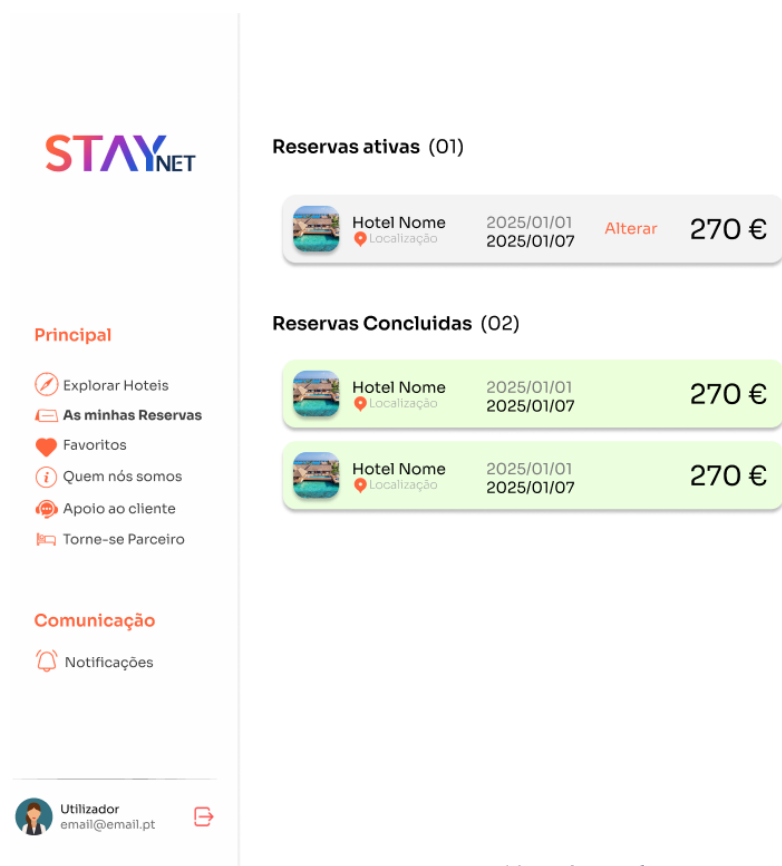


Figura 10 - Página de Reservas

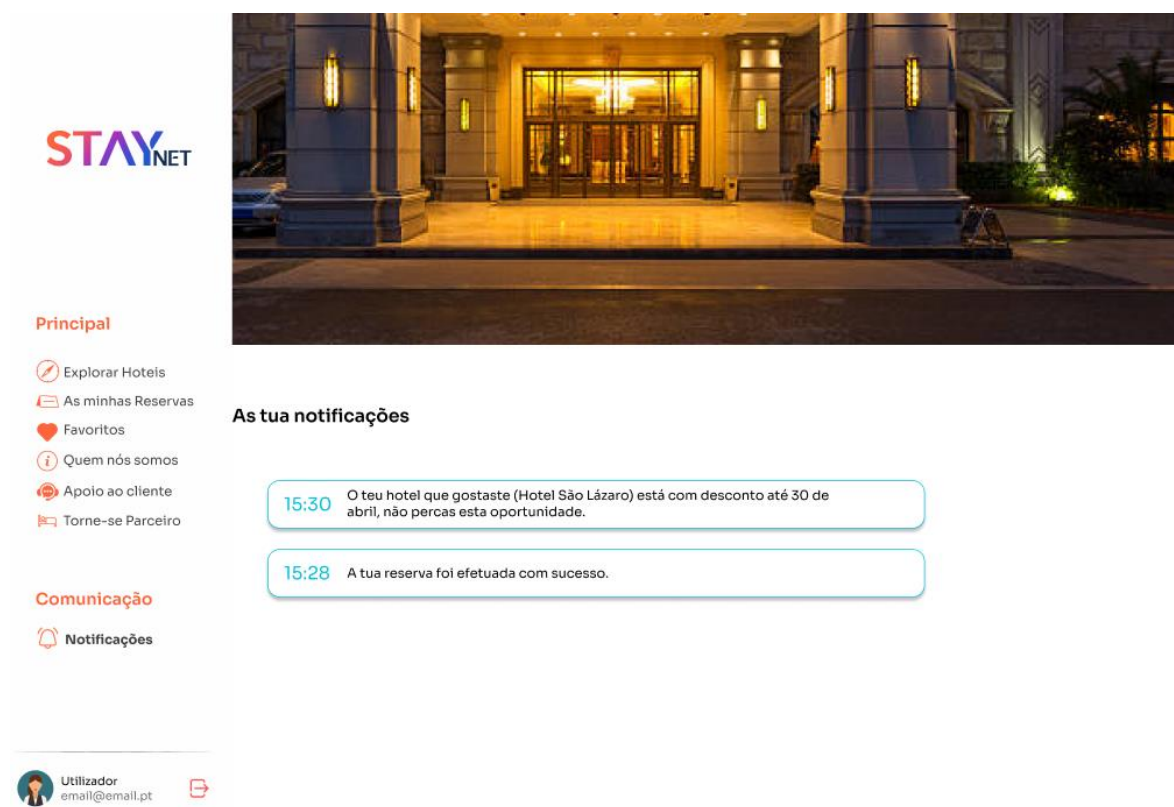


Figura 11 - Página de Notificações

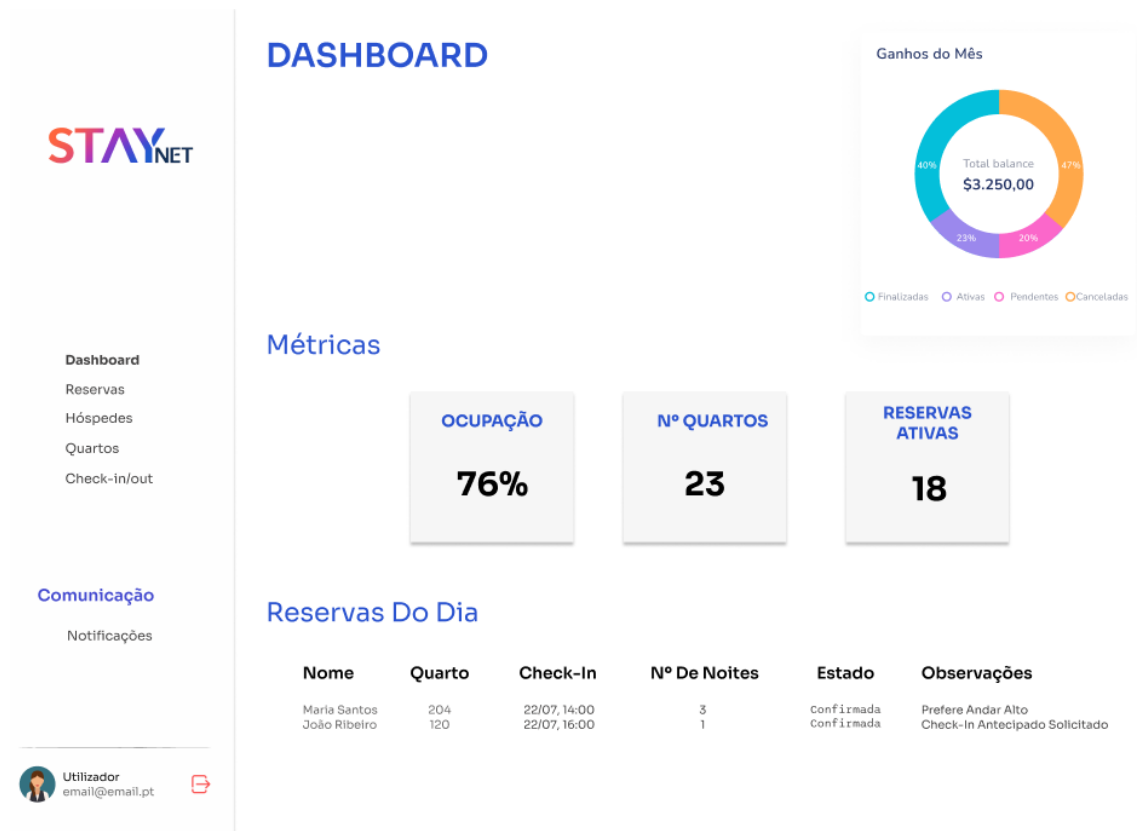


Figura 12 - Área do Parceiro

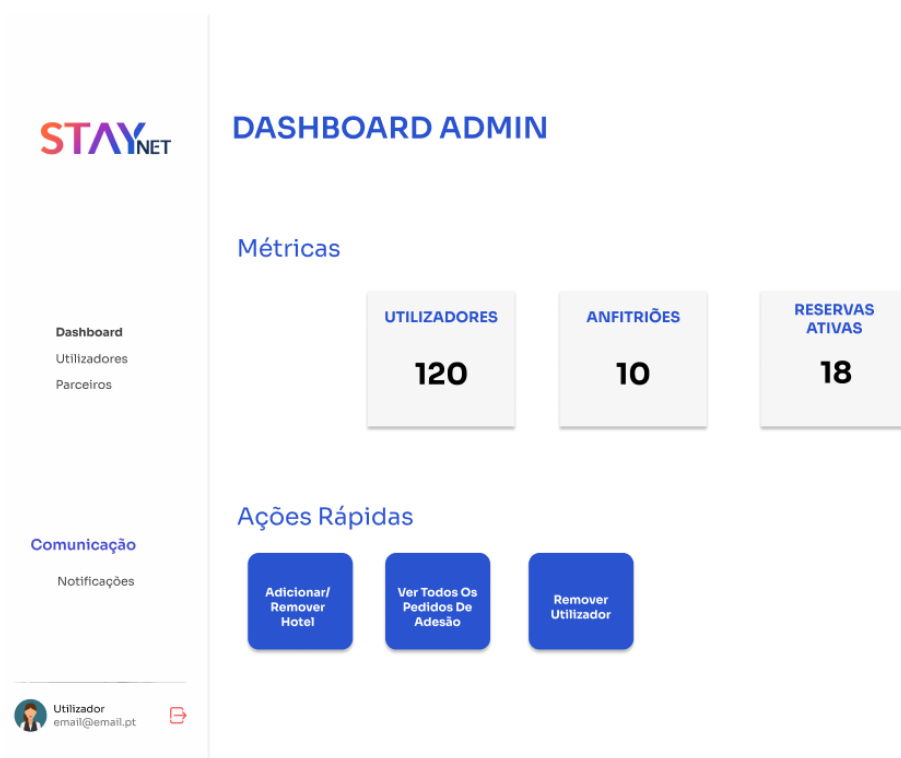


Figura 13 - Área do Administrador

3.5 Metodologia e Organização de Recursos

Para que o desenvolvimento da StayNet fosse mais simples de gerir e não se tornasse um processo desorganizado, foi importante definir desde cedo uma forma de trabalhar organizada. Ao longo do projeto, as tarefas foram distribuídas e planeadas por etapas, o que ajudou a manter o foco no que era prioritário e a controlar melhor o tempo disponível.

O trabalho começou pela análise, onde se identificaram as necessidades da plataforma e se clarificaram os perfis de utilizador e as funcionalidades principais. De seguida, passou-se para a modelação, sobretudo da base de dados e dos fluxos mais importantes (reservas, carrinho, favoritos e pagamento), para garantir que a estrutura fazia sentido antes de implementar.

Depois, avançou-se para o desenvolvimento, implementando as funcionalidades de forma progressiva e validando cada parte à medida que ficava concluída. Por fim, foram realizados testes para confirmar que tudo funcionava como esperado, com especial atenção aos pontos mais críticos do sistema, como a disponibilidade, a criação de reservas e o processo de pagamento.

Ao longo de todas as fases, a organização foi ajustada sempre que necessário, com base no que ia surgindo durante a implementação, garantindo que o projeto se mantinha consistente e alinhado com os objetivos definidos para a StayNet.

Capítulo 4

Desenvolvimento/Implementação

No contexto da StayNet, fez sentido criar uma solução digital moderna e prática que ajudasse em duas frentes: por um lado, tornar o processo de reserva mais simples para quem procura alojamento; por outro, dar aos hotéis uma forma mais organizada de gerir pedidos e a própria estadia. A plataforma foi pensada para diferentes perfis, Visitante, Utilizador, Parceiro e Administrador, garantindo que cada um só vê e usa o que realmente lhe diz respeito.

O objetivo principal deste projeto foi desenvolver uma plataforma de reservas e gestão hoteleira que permitisse ao utilizador pesquisar hotéis e quartos, confirmar disponibilidade, guardar favoritos, preparar a reserva no carrinho e finalizar tudo com pagamento integrado.

Do lado do hotel, o Parceiro tem acesso a funcionalidades de gestão, como criar e atualizar informação do hotel e dos quartos, confirmar pedidos e acompanhar o processo da estadia, incluindo operações de check-in e check-out.

Este capítulo apresenta o desenvolvimento da StayNet de forma progressiva, desde a criação da estrutura base da aplicação e definição do modelo de dados, até à implementação das funcionalidades principais. São também descritas as integrações com serviços externos e os testes realizados, explicando como cada parte foi construída para responder aos objetivos definidos e garantir uma experiência consistente para todos os utilizadores.

4.1 Implementação do Frontend (Next.js)

4.1.1 Layout da Landing Page

A página de Layout permite definir a estrutura para todas as páginas e subpáginas da Landing Page, esta página é onde são compiladas as páginas e os componentes, navbar e o footer.

```
import React, { ReactNode } from "react";
import Image from "next/image";
import Link from "next/link";
import { auth } from "@auth";
import { redirect } from "next/navigation";
import Footer from "@components/footer"

const Layout = async ({ children }: { children: ReactNode }) => {
  const session = await auth();
  if (session) redirect("/discover");
  return (
    <div className="flex min-h-screen flex-col">
      <nav className="sticky top-0 z-50 border-b border-gray-200 bg-white shadow-sm">
        <div className="mx-auto max-w-7xl px-4 sm:px-6 lg:px-8">
          <div className="flex h-16 items-center justify-between">
            <div className="flex items-center gap-2">
              <Image src={"/logo.png"} alt={"logo"} width={125} height={125} />
            </div>

            <div className="hidden md:flex items-center gap-6">
              <a
                href="#features"
                className="text-gray-700 hover:text-[var(--brand2)] transition-colors"
              >
                Funcionalidades
              </a>
              <a
                href="#aboutus"
                className="text-gray-700 hover:text-[var(--brand2)] transition-colors"
              >
                Sobre Nós
              </a>
              <a
                href="#partners"
                className="text-gray-700 hover:text-[var(--brand2)] transition-colors"
              >
                Parceiros
              </a>
            </div>

            <div className="flex items-center gap-4">
              <Link href="/sign-in">
                <button className="hidden sm:block px-4 py-2 text-gray-700 hover:text-[var(--brand2)] transition-colors">
```

```

        Entrar
      </button>
    </Link>
    <Link href="/sign-up" passHref>
      <button className="px-4 py-2 bg-[var(--brand2)] text-white rounded-lg hover:bg-[var(--brandhover2)]
transition-colors font-medium">
        Começar{" "}
      </button>
    </Link>
  </div>
</div>
</div>
</nav>

<main className="flex-1">{children}</main>
<Footer/>
</div>
);
};

export default Layout;

```

Código 1 - Layout da Landing Page

4.1.2 Landing Page

Corresponde à página inicial de qualquer tipo de utilizador, nesta página é apresentada uma visão geral da plataforma, a qual mostra todas as funcionalidades da plataforma, lista os hotéis populares, mostra as informações sobre a plataforma e tem como ações principais o início de sessão e o registo.

```

import React from "react";
import Image from "next/image";
import { Hotel, Zap, Medal } from "lucide-react";
import Link from "next/link";
import { HotelCard } from "@components/hotels/hotelcard";
import { getHotels } from "@lib/actions/hotel.action";

export default async function Page() {
  const hotels = await getHotels();

  return (
    <div>
      <header className="flex flex-col md:flex-row items-start md:items-center justify-between p-6 md:p-8 lg:p-12 gap-8 max-w-7xl mx-auto">
        <div className="max-w-2xl text-left">
          <h1 className="text-4xl sm:text-5xl md:text-6xl lg:text-7xl font-black leading-tight">
            Explora as melhores estadias na StayNet
          </h1>
          <p className="mt-4 text-muted-foreground text-base md:text-lg">
            A forma mais simples e rápida de encontrares a tua próxima estadia.
          </p>
        </div>

        <div className="hidden md:block flex-shrink-0">
          <Image
            src="/header-image.svg"
            alt="Header Image"
            width={600}
            height={400}
            className="object-contain w-full h-auto"
          />
        </div>
      </header>

      <section id="features" className="bg-secondary py-16 px-6">
        <div className="max-w-7xl mx-auto">
          <h2 className="text-3xl md:text-4xl font-bold text-left mb-12">
            Porque escolher a StayNet?
          </h2>

          <div className="grid grid-cols-1 md:grid-cols-3 gap-8">
            <div className="bg-card p-6 rounded-2xl shadow-sm hover:shadow-md transition-shadow">
              <div className="w-12 h-12 bg-[var(--brand)/15] text-[var(--brand)] rounded-xl flex items-center justify-center mb-4">
                <Hotel />
              </div>
              <h3 className="text-xl font-bold mb-2">Variedade de Opções</h3>
              <p className="text-muted-foreground">
                Milhares de Hotéis verificados em todo o mundo.
              </p>
            </div>

            <div className="bg-card p-6 rounded-2xl shadow-sm hover:shadow-md transition-shadow">
              <div className="w-12 h-12 bg-[var(--brand)/15] text-[var(--brand)] rounded-xl flex items-center justify-center mb-4">
                <Zap />
              </div>
              <h3 className="text-xl font-bold mb-2">Reserva Rápida</h3>
              <p className="text-muted-foreground">

```

```

        Processo de reserva simples e seguro. Confirma a tua estadia em
        poucos minutos.
      </p>
    </div>

    <div className="bg-card p-6 rounded-2xl shadow-sm hover:shadow-md transition-shadow">
      <div className="w-12 h-12 bg-[var(--brand)/15] text-[var(--brand)] rounded-xl flex items-center justify-center
mb-4">
        <Medal />
      </div>
      <h3 className="text-xl font-bold mb-2">Melhor Preço Garantido</h3>
      <p className="text-muted-foreground">
        Preços competitivos e transparentes. Sem taxas escondidas.
      </p>
    </div>
  </div>
</div>
</section>

<section id="aboutus" className="py-16 px-6">
  <div className="max-w-7xl mx-auto">
    <h2 className="text-3xl md:text-4xl font-bold text-left mb-12">
      Sobre Nós
    </h2>
    <p className="text-lg text-muted-foreground leading-relaxed max-w-4xl">
      A StayNet nasceu da vontade de simplificar a forma como hotéis e
      hóspedes interagem. Acreditamos que a tecnologia deve tornar o
      processo de reserva e gestão mais rápido, eficiente e humano. A
      nossa plataforma foi desenvolvida para oferecer uma experiência
      fluida — tanto para quem gere hotéis como para quem reserva —
      centralizando todas as operações num único sistema intuitivo.
    </p>
  </div>
</section>

<section id="hotels" className="py-16 px-6">
  <div className="max-w-7xl mx-auto">
    <h2 className="text-3xl md:text-4xl font-bold mb-8 text-left">
      Descobre os nossos hotéis
    </h2>

    <div className="flex flex-wrap gap-6">
      {hotels.map((hotel) => (
        <div
          key={hotel.id}
          className="flex-1 min-w-[280px] max-w-[320px]"
        >
          <HotelCard hotel={hotel} />
        </div>
      ))}
    </div>
  </div>
</section>

<section
  id="partners"
  className="bg-[var(--brand)] py-16 px-6 text-white"
>
  <div className="flex flex-col md:flex-row items-start gap-8 max-w-7xl mx-auto">
    <h2 className="text-3xl sm:text-4xl md:text-5xl font-bold flex-shrink-0 md:max-w-md">

```

```

    Como posso ser Parceiro da StayNet?
  </h2>

  <div className="flex flex-col">
    <p className="text-white/90 text-base md:text-lg mb-8 leading-relaxed">
      Se tens um hotel, podes juntar-te à StayNet e começar a receber
      reservas de forma simples, rápida e segura. Basta preencher um
      pequeno formulário e a nossa equipa entrará em contacto para
      ativar a tua conta.
    </p>

    <Link href="/partner">
      <button className="px-8 py-4 bg-[var(--brand2)] rounded-xl hover:bg-[var(--brandhover2)] transition font-bold
text-lg self-start">
        Começar Agora
      </button>
    </Link>
  </div>
</div>
</section>
</div>
);
}

```

Código 2 - Landing Page

4.1.3 Layout de Autenticação

Esta página do Layout agrupa todas as páginas referentes à autenticação, como as páginas de início de sessão, registo, recuperar palavra-passe.

Escolhemos criar um layout distinto para a autenticação, para simplificar, organizar e devido às políticas de rotas que escolhemos para cada utilizador.

```

import type { ReactNode } from "react";

const Layout = async ({ children }: { children: ReactNode }) => {
  return (
    <main className="max-w-full">
      <section className="flex min-h-screen justify-center items-center">
        <div className="bg--color-background w-full max-w-md p-6 shadow-lg">
          {children}
        </div>
      </section>
    </main>
  );
};

```

```
</main>
);
};
export default Layout;
```

Código 3 - Layout de Autenticação

4.1.4 Página de Login

Corresponde à página de login que é visível apenas para os utilizadores que não têm sessão iniciada, nesta página é apresentado um formulário com os campos de email e palavra-passe.

```
"use client";

import React from "react";
import AuthForm from "@/components/auth/AuthForm";
import { signInSchema } from "@/lib/validations";
import { signInWithCredentials } from "@/lib/actions/auth";

const Page = () => (
  <AuthForm
    type="SIGN_IN"
    schema={signInSchema}
    defaultValues={{
      email: "",
      password: "",
    }}
    onSubmit={signInWithCredentials}
  />
);

export default Page;
```

Código 4 - Página de Login

4.1.5 Página de Registo

Corresponde à página de registo que como a página de início de sessão, é apenas visível para os utilizadores que não têm sessão iniciada, nesta página é apresentado um formulário com os campos de nome, email e palavra-passe.

```
"use client";

import React from "react";
import AuthForm from "@/components/auth/AuthForm";
```

```
import { signUpSchema } from "@lib/validations";
import { signUp } from "@lib/actions/auth";

const Page = () => (
  <AuthForm
    type="SIGN_UP"
    schema={signUpSchema}
    defaultValues={{
      fullName: "",
      email: "",
      password: "",
    }}
    onSubmit={signUp}
  />
);
export default Page;
```

Código 5 - Página de Registo

4.1.6 Página de Recuperar password

Corresponde à página de Recuperar password, nesta página é apresentado um formulário com o campo email, com a função do utilizador receber um email para recuperar a sua palavra-passe esquecida.

```
"use client";

import React from "react";
import AuthForm from "@components/auth/AuthForm";
import { recoverPasswordSchema } from "@lib/validations";
import { sendPasswordResetLink } from "@lib/actions/password.actions";

const Page = () => (
  <AuthForm
    type="RECOVER_PASSWORD"
    schema={recoverPasswordSchema}
    defaultValues={{
      email: "",
    }}
    onSubmit={(data) => sendPasswordResetLink(data.email)}
  />
);

export default Page;
```

Código 6 - Página de Recuperar Password

4.1.7 Dashboard de utilizador

Esta dashboard apenas é acessível a utilizadores com sessão iniciada. Tal como a estrutura da autenticação também decidimos fazer um layout exclusivo para este conjunto de páginas, no layout desta dashboard encontram-se os componentes do menu lateral, o footer e o toaster que faz parte da biblioteca shadcn, que nos permite adicionar os pop-ups de aviso á plataforma, quando alguma operação é efetuada com sucesso, ou existe algum erro.

Na página inicial da dashboard é onde se encontram listados todos os hotéis disponíveis, inclusive a barra de procura e os filtros relacionados. Nesta página ao utilizador carregar sobre um hotel, será redirecionado para uma página que irá exibir todos os detalhes daquele hotel em específico, tais como:

- Nome e localização do hotel.
- Carrossel de imagens, com as imagens referente ao hotel.
- Breve informação sobre o hotel.
- Pequeno menu com duas páginas, onde o utilizador consegue escolher entre o quarto individual e o quarto duplo, se o mesmo existir.
- Preço referente a cada quarto.
- Informações sobre cada quarto.
- Componente de calendário onde o utilizador pode selecionar as datas que deseja reservar, e com validação para verificar as datas que já foram reservadas.
- Por fim integramos a biblioteca do Openstreetmap num componente para exibir a localização de cada hotel.

A dashboard consta com um menu lateral, com os seguintes atalhos:

- **Início** – Página inicial do dashboard, mencionada anteriormente.
- **Carrinho** – Esta página é onde se encontram todas as reservas nas quais o pagamento ainda não foi realizado. Também encontramos o componente do botão de apagar, uma ou todas as reservas e também o botão que redireciona o utilizador para o pagamento.

- **Minhas reservas** – Nesta página encontramos um componente que lista todas as reservas feitas pelo utilizador, e mostra o seu estado. O estado confirmado é para as reservas aceites e o estado completado é referente às reservas que já foram finalizadas.
- **Favoritos** – Nesta página encontramos a lista de todos os hotéis marcados como favorito pelo utilizador.
- **Torne-se Parceiro** – Esta página possui um formulário com o intuito do utilizador candidatar-se a ser um parceiro da plataforma. Ser parceiro da plataforma dá a permissão de anunciar hotéis e conseguir aceder ao dashboard de Parceiro.
- **Quem nós somos** – Esta página é apenas uma página informativa com conteúdo estático, onde tem algumas informações sobre a plataforma StayNet.
- **Sub-menu** – No extremo inferior do menu lateral, existe um submenu que apresenta o nome do utilizador e com os seguintes atalhos:
 - **Conta** – Redireciona o utilizador para a página da conta do utilizador, onde ele pode terminar sessão e alterar a sua password.
 - **Terminar sessão** – É apenas um botão que termina sessão da plataforma.

```

import "@app/globals.css";
import React, { ReactNode } from "react";

import { SidebarProvider, SidebarTrigger } from "@components/ui/sidebar";
import { SideMenu } from "@components/sidemenu";
import { Toaster } from "@components/ui/sonner";
import { auth } from "@auth";
import Footer from "@components/footer";

const Layout = async ({ children }: { children: ReactNode }) => {
  const session = await auth();

  return (
    <SidebarProvider>
      <SideMenu session={session} variant="discover" />
      <main className="flex min-h-screen flex-1 flex-col bg-background">
        <div className="flex h-14 items-center px-4">
          <SidebarTrigger />
        </div>
        <div className="flex-1 w-full p-5">{children}</div>
        <Footer />
      </main>
      <Toaster />
    </SidebarProvider>
  );
};

export default Layout;

```

Código 7 - Layout Dashboard Utilizador

```

import React from "react";
import { HotelCard } from "@components/hotels/hotelcard";
import { getHotels } from "@lib/actions/hotel.action";
import HotelFilters from "@components/hotels/hotel-filters";
import { Search } from "lucide-react";

interface SearchParams {
  search?: string;
  city?: string;
  country?: string;
}

export default async function HotelsPage({
  searchParams,
}: {
  searchParams: Promise<SearchParams>;
}) {
  const hotels = await getHotels();
  const params = await searchParams;

  if (hotels.length === 0) {
    return (

```

```

<div className="min-h-screen">
  <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
    <h1 className="text-3xl font-bold text-foreground mb-6">Hotéis</h1>
    <p className="text-muted-foreground">
      Ainda não há hotéis disponíveis.
    </p>
  </div>
</div>
);
}

```

```

const searchTerm = params.search?.toLowerCase() || "";
const selectedCity = params.city || "all";
const selectedCountry = params.country || "all";

```

```

const filteredHotels = hotels.filter((hotel) => {
  const matchesSearch =
    searchTerm === "" ||
    hotel.name.toLowerCase().includes(searchTerm) ||
    hotel.city.toLowerCase().includes(searchTerm) ||
    hotel.country?.toLowerCase().includes(searchTerm) ||
    hotel.description?.toLowerCase().includes(searchTerm);

  const matchesCity = selectedCity === "all" || hotel.city === selectedCity;
  const matchesCountry =
    selectedCountry === "all" || hotel.country === selectedCountry;

  return matchesSearch && matchesCity && matchesCountry;
});

```

```

const cities = Array.from(new Set(hotels.map((h) => h.city))).sort();
const countries = Array.from(
  new Set(hotels.map((h) => h.country).filter(Boolean)),
).sort() as string[];

```

```

const hasActiveFilters =
  searchTerm !== "" || selectedCity !== "all" || selectedCountry !== "all";

```

```

return (
  <div className="min-h-screen bg-background">
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
      <div className="mb-8">
        <h1 className="text-3xl font-bold text-foreground mb-2">
          Descubra o seu Hotel Ideal
        </h1>
        <p className="text-muted-foreground">
          {hotels.length} {" "}
          {hotels.length === 1 ? "Hotel disponível" : "hotéis disponíveis"}
        </p>
      </div>

      <HotelFilters
        cities={cities}
        countries={countries}
        totalHotels={hotels.length}
        filteredCount={filteredHotels.length}
        hasActiveFilters={hasActiveFilters}
      />
    </div>
  </div>
)

```

```

{filteredHotels.length > 0 ? (
  <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4 gap-6">
    {filteredHotels.map((hotel) => (
      <HotelCard key={hotel.id} hotel={hotel} />
    ))}
  </div>
) : (
  <div className="text-center py-16">
    <div className="inline-flex items-center justify-center w-16 h-16 bg-muted rounded-full mb-4">
      <Search className="h-8 w-8 text-muted-foreground" />
    </div>
    <h3 className="text-lg font-semibold text-foreground mb-2">
      Nenhum hotel encontrado
    </h3>
    <p className="text-muted-foreground mb-4">
      Tente ajustar os seus filtros de pesquisa
    </p>
    {hasActiveFilters && (
      <a
        href="/discover"
        className="inline-flex items-center justify-center rounded-md text-sm font-medium border border-input
bg-background hover:bg-accent hover:text-accent-foreground h-10 px-4 py-2"
      >
        Limpar todos os filtros
      </a>
    )}
  </div>
)}
</div>
</div>
);
}

```

Código 8 - Página Inicial

```

import { getHotelById } from "@lib/actions/hotel.action";
import { ImageSlider } from "@components/hotels/imageslider";
import { ReservasCard } from "@components/hotels/reservasCard";
import dynamic from "next/dynamic";
import { FavoriteButton } from "@components/favorite/favoriteButton";

const Map = dynamic(() => import("@components/Map"), {
  loading: () => (
    <p className="text-sm text-muted-foreground">A carregar mapa...</p>
  ),
});

export default async function HotelDetailsPage({
  params,
}: {
  params: Promise<{ hotelId: string }>;
}) {
  const { hotelId } = await params;
  const hotel = await getHotelById(hotelId);

  if (!hotel) {
    return (
      <div className="container mx-auto px-4 py-8">
        <p className="text-center text-muted-foreground">
          Hotel não encontrado
        </p>
      </div>
    );
  }

  return (
    <div className="container mx-auto px-4 py-8 max-w-7xl">
      <div className="mb-8">
        <h1 className="capitalize text-3xl md:text-4xl font-bold mb-2 text-foreground">
          {hotel.name}
        </h1>
        <p className="capitalize text-muted-foreground text-lg">
          {hotel.city}, {hotel.country}
        </p>
      </div>

      <div className="mb-8">
        <ImageSlider images={hotel.images} hotelName={hotel.name} />
      </div>

      {hotel.description && (
        <div className="flex mb-8">
          <div>
            <h2 className="text-2xl font-semibold mb-4 text-foreground">
              Sobre o hotel
            </h2>
            <p className="capitalize text-muted-foreground leading-relaxed">
              {hotel.description}
            </p>
          </div>

          <FavoriteButton
            className="with-30px"
            hotelId={hotel.id}
          />
        </div>
      )}
    </div>
  );
}

```

```

        variant="icon"
      />
    </div>
  )}
</div>
  <ReservasCard hotel={hotel} />
</div>

<div className="border-t border-border pt-6">
  <h2 className="text-2xl font-semibold mb-4 text-foreground">
    Contactos
  </h2>
  <div className="space-y-2">
    <p className="text-foreground">
      <span className="font-medium">Morada:</span>{" "}
      <p className="capitalize text-small text-muted-foreground leading-relaxed">
        {hotel.address}
      </p>
      <div className="flex justify-center mt-4 h-64 w-full rounded-lg overflow-hidden border-border">
        <Map location={hotel.address} />
      </div>
    </p>
    <p className="text-foreground">
      <span className="font-medium">Email:</span>{" "}
      <a
        href={`mailto:${hotel.email}`}
        className="text-primary hover:underline"
      >
        {hotel.email}
      </a>
    </p>
    {hotel.phone && (
      <p className="text-foreground">
        <span className="font-medium">Telefone:</span>{" "}
        <a
          href={`tel:${hotel.phone}`}
          className="text-primary hover:underline"
        >
          {hotel.phone}
        </a>
      </p>
    )}
  </div>
</div>
</div>
);
}

```

Código 9 - Página de detalhes do Hotel

```

"use client";

import { Suspense, useEffect, useState } from "react";
import { useRouter } from "next/navigation";
import { CartComponent } from "@components/cart/cartComponent";
import { Loader2 } from "lucide-react";
import { getCart } from "@lib/cart";

function CartContent() {
  const router = useRouter();
  const [cartItems, setCartItems] = useState<CartItem[]>([]);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    const items = getCart();
    setCartItems(items);
    setIsLoading(false);

    if (items.length === 0) {
    }
  }, []);
  const handleRefresh = () => {
    setCartItems(getCart());
  };
  if (isLoading) {
    return (
      <div className="min-h-screen flex items-center justify-center">
        <div className="text-center">
          <Loader2 className="h-8 w-8 animate-spin mx-auto mb-4" />
          <p className="text-muted-foreground">A carregar carrinho...</p>
        </div>
      </div>
    );
  }
  return (
    <div className="min-h-screen bg-background py-12">
      <CartComponent cartItems={cartItems} onRefresh={handleRefresh} />
    </div>
  );
}

export default function CartPage() {
  return (
    <Suspense
      fallback={
        <div className="min-h-screen flex items-center justify-center">
          <div className="text-center">
            <Loader2 className="h-8 w-8 animate-spin mx-auto mb-4" />
            <p className="text-muted-foreground">A carregar carrinho...</p>
          </div>
        </div>
      }
    >

```



```

    <CartContent />
  </Suspense>
);
}

```

Código 10 - Página Carrinho

```

import { getUserBookings } from "@lib/actions/booking.actions";
import { redirect } from "next/navigation";
import MyBookingsList from "@components/mybookings/myBookingsList";

export default async function MyBookingsPage() {
  const result = await getUserBookings();

  if (!result.success) {
    redirect("/discover");
  }

  return <MyBookingsList bookings={result.bookings || []} />;
}

```

Código 11 Página Minhas Reservas

```

import React from "react";
import { FavoritesList } from "@components/favorite/favoriteList";

const Page = () => {
  return (
    <div className="max-h-1xl mx-auto">
      <h1 className="text-2xl font-bold mb-4">Favoritos</h1>
      <FavoritesList />
    </div>
  );
};
export default Page;

```

Código 12 - Página Favoritos

```

import React from "react";
import { PartnerApplicationForm } from "@components/partner/createPartnerApplication";

const Page = () => {
  return (
    <div className="flex flex-col min-h-screen w-full p-5 bg-background text-foreground">
      <div className="max-w-2xl w-full mb-8">
        <h1 className="text-4xl font-bold mb-2">Torne-se Parceiro StayNet</h1>
      </div>

      <div className="w-full items-center max-w-2xl">
        <PartnerApplicationForm />
      </div>
    </div>
  );
};

```

```
export default Page;
```

Código 13 - Página Torne-se Parceiro

```
import React from "react";

const Page = () => {
  return (
    <div className="min-h-screen bg-background text-foreground">
      <div className="max-w-4xl text-center mx-auto px-6 py-16">
        <h1 className="text-4xl font-bold mb-6 text-center">Quem Somos</h1>

        <p className="text-lg leading-relaxed mb-6">
          A StayNet nasceu com a missão de tornar a experiência de reservas e
          gestão de hotéis mais simples e eficiente.
        </p>

        <p className="text-lg leading-relaxed mb-6">
          A nossa plataforma centraliza todas as operações num único sistema
          intuitivo, oferecendo uma experiência fluida tanto para quem gere
          hotéis como para quem reserva. Com a StayNet, a gestão e a reserva de
          quartos tornam-se rápidas, claras e descomplicadas.
        </p>

        <p className="text-lg leading-relaxed">
          O nosso objetivo é conectar hotéis e hóspedes de forma inteligente,
          garantindo eficiência, transparência e satisfação em cada interação.
        </p>
      </div>
    </div>
  );
};

export default Page;
```

Código 14 - Página Quem nós somos

```

import React from "react";
import { User2 } from "lucide-react";
import { SignOutButton } from "@components/auth/SignOutButton";
import { auth } from "@auth";
import Link from "next/link";

const Page = async () => {
  const session = await auth();

  return (
    <div className="flex min-h-screen items-center justify-center">
      <div className="w-full max-w-xl rounded-3xl border bg-background p-10 shadow-xl text-center bg-muted/40">
        <div className="mb-8 flex justify-center">
          <div className="flex h-24 w-24 items-center justify-center rounded-full bg-primary/10 ring-1 ring-primary/20">
            <User2 className="h-12 w-12 text-primary" />
          </div>
        </div>

        <h1 className="mb-2 text-4xl font-semibold tracking-tight">
          A minha conta
        </h1>

        <p className="mb-8 text-sm text-muted-foreground">
          Gerir sessão e informações da conta
        </p>

        <div className="mb-10 flex flex-col items-center gap-2">
          <p className="text-lg font-medium capitalize text-foreground">
            {session?.user?.name}
          </p>

          <Link
            href="/recover-password"
            className="text-sm font-medium text-primary underline-offset-4 hover:underline"
          >
            Alterar palavra-passe
          </Link>
        </div>

        <SignOutButton />
      </div>
    </div>
  );
};

export default Page;

```

Código 15 - Página a minha conta

4.1.8 Dashboard de Parceiro

Esta dashboard, é a página de controlo onde os gerentes dos hotéis conseguem controlar as reservas, hóspedes e visualizar o estado dos pagamentos. Esta apenas é visível a utilizadores com as devidas permissões, as quais são geridas pelo administrador da StayNet, após a submissão da candidatura do utilizador. À semelhança da dashboard de utilizador, esta também tem um layout exclusivo para este conjunto de páginas, permitindo ter um menu lateral adaptado á dashboard de parceiro. Esta abordagem facilita também a gestão das rotas e permissões associadas á dashboard do parceiro.

Na página inicial da dashboard é onde se encontram algumas métricas que mostram imediatamente a informação que o gerente ou administrador do hotel necessita. No menu lateral existem vários itens que redirecionam para as seguintes páginas:

- **Adicionar Hotel** – É a página onde o gerente do hotel consegue inserir toda a informação referente ao seu hotel e quartos. Nesta página há um formulário, o qual tem como principal objetivo permitir o registo e a configuração de um hotel na plataforma. Inicialmente o formulário pede os dados gerais do hotel, como o nome, uma descrição, localização e as informações de contacto. De seguida vem a secção da configuração de Quartos, o gestor é obrigado a inserir pelo menos um tipo de quarto, individual ou duplo, sendo possível indicar o número total de quartos, o preço e uma descrição para cada tipo de quarto.
- **Hotéis** - Uma página onde o gerente do hotel consegue listar e remover todos os seus hotéis inseridos na plataforma.
- **Reservas** – Esta página tem como objetivo e listar e mostrar todos os detalhes das reservas feitas em todos os hotéis do gerente. Mostra informações como, o estado da reserva, nome do hóspede, check-in/out, tipo de quarto, email do hóspede e o preço total da reserva.
- **Hóspedes** – Página simples onde o gerente dos hotéis consegue listar e analisar todos os hóspedes dos seus hotéis.
- **Check-in/out** – Página dedicada à gestão dos check-ins e dos check-outs. Nesta página o gerente consegue fazer os check-ins e os check-outs manualmente, quando é necessário.

```

import React from "react";
import { BookingChart } from "@components/controlpanel/chartinicio";

const Page = () => {
  return (
    <div>
      <h1 className="text-2xl font-bold mb-4">
        {" "}
        Pagina de Controlo de Parceiro
      </h1>
      <BookingChart />
    </div>
  );
};
export default Page;

```

Código 16 - Página Inicial Parceiro

```

import React from "react";
import { CreateHotelForm } from "@components/hotels/hotelForm";

const Page = () => {
  return (
    <div className="w-full max-w-3xl px-4 py-6 md:px-6 md:py-8">
      <div className="mb-6">
        <h1 className="text-2xl md:text-3xl font-semibold text-left">
          Novo Hotel
        </h1>
        <p className="mt-1 text-sm md:text-base text-muted-foreground">
          Adiciona o teu hotel à plataforma e torna-o visível para os clientes.
        </p>
      </div>

      <CreateHotelForm />
    </div>
  );
};
export default Page;

```

Código 17 - Página Adicionar Hotel

```

import { UserHotelsList } from "@components/controlpanel/PartnerHotelsList";
import { getUserHotels } from "@lib/actions/hotel.action";

export default async function ControlPanelHotels() {
  const hotels = await getUserHotels();

  return (
    <section className="space-y-4">
      <h2 className="text-2xl md:text-3xl font-semibold text-left pb-6">
        Os meus hotéis
      </h2>
      <UserHotelsList hotels={hotels} />
    </section>
  );
}

```

Código 18 - Página Hotéis

```

import { getPartnerBookings } from "@lib/actions/booking.actions";
import { PartnerBookingsList } from "@components/controlpanel/partnerBookingList";

export default async function OwnerBookingsPage() {
  const result = await getPartnerBookings();

  if (!result.success) {
    return (
      <div className="container mx-auto px-4 py-8">
        <h1 className="text-2xl font-bold mb-4">Erro</h1>
        <p className="text-red-600">{result.error}</p>
      </div>
    );
  }

  const bookings = result.bookings || [];

  if (bookings.length === 0) {
    return (
      <div className="container mx-auto px-4 py-8">
        <h1 className="text-2xl md:text-3xl font-semibold text-left">
          Reservas dos Meus Hotéis
        </h1>
        <p className="text-gray-600">Ainda não há reservas nos seus hotéis.</p>
      </div>
    );
  }

  return (
    <div className="container mx-auto px-4 py-8">
      <h1 className="text-2xl md:text-3xl font-semibold text-left pb-6">
        Reservas dos Meus Hotéis
      </h1>
      <PartnerBookingsList bookings={bookings} />
    </div>
  );
}

```

Código 19 - Página Reservas

```

import React from "react";
import { GuestsList } from "@components/controlpanel/guestsList";

const Page = () => {
  return (
    <div className="space-y-6">
      <GuestsList />
    </div>
  );
};
export default Page;

```

Código 20 - Página Hospedes

```

import React from "react";
import { CheckInManagement } from "@components/controlpanel/checkInManagement";
import { CheckOutManagement } from "@components/controlpanel/checkOutManagement";

const Page = () => {
  return (
    <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
      <CheckInManagement />
      <CheckOutManagement />
    </div>
  );
};
export default Page;

```

Código 21 - Página check-in e check-out

4.1.9 Dashboard de Administrador

Esta dashboard, é a página de controlo do administrador de toda a plataforma StayNet. Esta apenas é visível pelos administradores.

Na página inicial da dashboard de administrador, encontram-se algumas métricas que mostram algumas das informações mais importantes. No menu lateral existem várias páginas, tais como:

- **Parceiros** – Nesta página é onde são geridos os pedidos realizados pelos utilizadores para se tornarem parceiros da StayNet. Conforme os pedidos são realizados, inicialmente ficam com o estado de pendentes podendo ser aprovados ou rejeitados por um administrador. Esta página também possui um componente, onde são listados todos os parceiros da plataforma.
- **Hotéis** – Diferente da dashboard do parceiro, que apenas lista os hotéis de cada um, esta mostra todos os hotéis da plataforma.

```
import React from "react";
import { DashboardStats } from "@components/dashboard/dashboardMetrics";

const Page = () => {
  return (
    <div className="container mx-auto py-8 px-4">
      <div className="mb-8">
        <h1 className="text-3xl font-bold tracking-tight">Dashboard</h1>
        <p className="text-muted-foreground">
          Visão geral das estatísticas da plataforma
        </p>
      </div>

      <DashboardStats />
    </div>
  );
};

export default Page;
```

Código - 22 Página Inicial do Dashboard do Administrador


```

import React from "react";
import { AdminPartnerApplications } from "@components/partner/dashboardPartnerApplications";
import { PartnersList } from "@components/partner/partnerList";

const Page = () => {
  return (
    <div>
      <div className="pb-15">
        <h1 className="font-medium text-2xl pb-6">
          Lista de candidaturas pendentes
        </h1>
        <AdminPartnerApplications />
      </div>

      <div>
        <h1 className="font-medium text-2xl pb-6">
          Lista de Parceiros registados
        </h1>

        <PartnersList />
      </div>
    </div>
  );
};
export default Page;

```

Código - 23 Página Parceiros

```

import React from "react";
import { HotelsList } from "@components/dashboard/adminHotelsList";

const Page = () => {
  return (
    <div>
      <h1 className="font-medium text-2xl pb-6">Lista de hoteis</h1>
      <HotelsList />
    </div>
  );
};
export default Page;

```

Código - 24 Página Hotéis

4.2 Implementação do Backend

Para que o frontend da StayNet funcionasse como era esperado, foi necessário implementar também a parte do backend do sistema e a ligação à base de dados. No nosso caso, a StayNet é full-stack em Next.js, ou seja, a própria aplicação trata tanto da interface como da lógica do servidor (através das server Actions, rotas e lógica no lado do servidor), comunicando com as bases de dados Neon e Cloudinary através do Drizzle.

O backend é o que garante que a plataforma funcione a sério, porque é aqui que ficam as tarefas mais importantes:

- **Lógica da plataforma:** validações e regras do sistema, como autenticação, permissões por perfil (Utilizador/Parceiro/Admin), criação de reservas, confirmação de pedidos e regras de disponibilidade.
- **Gestão da base de dados:** é o backend que faz as operações na base de dados (criar, editar, consultar e remover). O frontend apenas faz pedidos e mostra resultados; quem decide e executa é o backend.
- **Segurança:** tratamento de dados sensíveis e controlo de acessos, garantindo que só quem tem autorização consegue aceder a certas áreas (por exemplo, dashboard de parceiro e operações de gestão).
- **Integrações externas:** ligação a serviços como o pagamento (Stripe) e envio de notificações por email através do (Resend), garantindo que estes processos acontecem de forma controlada e consistente.

4.2.1 Estrutura Geral da Plataforma

Para facilitar a compreensão da estrutura da nossa plataforma com o apoio do plugin (FileTree Pro) instalado no Visual Studio Code, criamos o diagrama da figura 14.

Com a ajuda deste diagrama conseguimos analisar como foram estruturadas as pastas do Next.js, o que é imprescindível, pois é pela estrutura das páginas que o Next.js cria as rotas automaticamente. As rotas e as páginas principais são:

- (auth) - Contém todas as páginas da autenticação, como o login (sign-in), registo (sign-up), recuperação de palavra-passe (recover-password) e gestão de parceiros (partner). Esta estrutura permitiu controlar toda a lógica de autenticação, tornando o desenvolvimento das proteções de rotas mais fácil.
- (controlpanel) - Reúne todas as páginas dedicadas à gestão dos hotéis do parceiro.
- (dashboard) - Reúne todas as páginas dedicadas à gestão da plataforma da StayNet.
- (discover) – Esta pasta é onde estão todas as páginas que são acessíveis pelos utilizadores que pretendem reservar um hotel. Nesta estrutura incluímos uma funcionalidade do Next.js na pasta [hotelId], que é as rotas dinâmicas, que recebe o id do hotel pelo URL, o que permite exibir os detalhes de cada hotel sem criar páginas para cada um.

Na pasta (api), é onde ficam as rotas da autenticação do Auth.js, e a lógica da API do Stripe que trata do processamento de pagamentos.

Também temos a pasta componentes, esta é onde organizamos todos os componentes customizados da plataforma, que podemos usar em qualquer página.

Também temos a pasta (database) e a pasta (Drizzle), nestas pastas é onde está a integração da base de dados e o schema, que é onde estão definidas todas as tabelas e as suas relações.

Dentro da pasta (lib) é onde ficam todas as Server Actions que permitem que a todas as funcionalidades do backend sejam executadas sem necessidade de servidores separados.

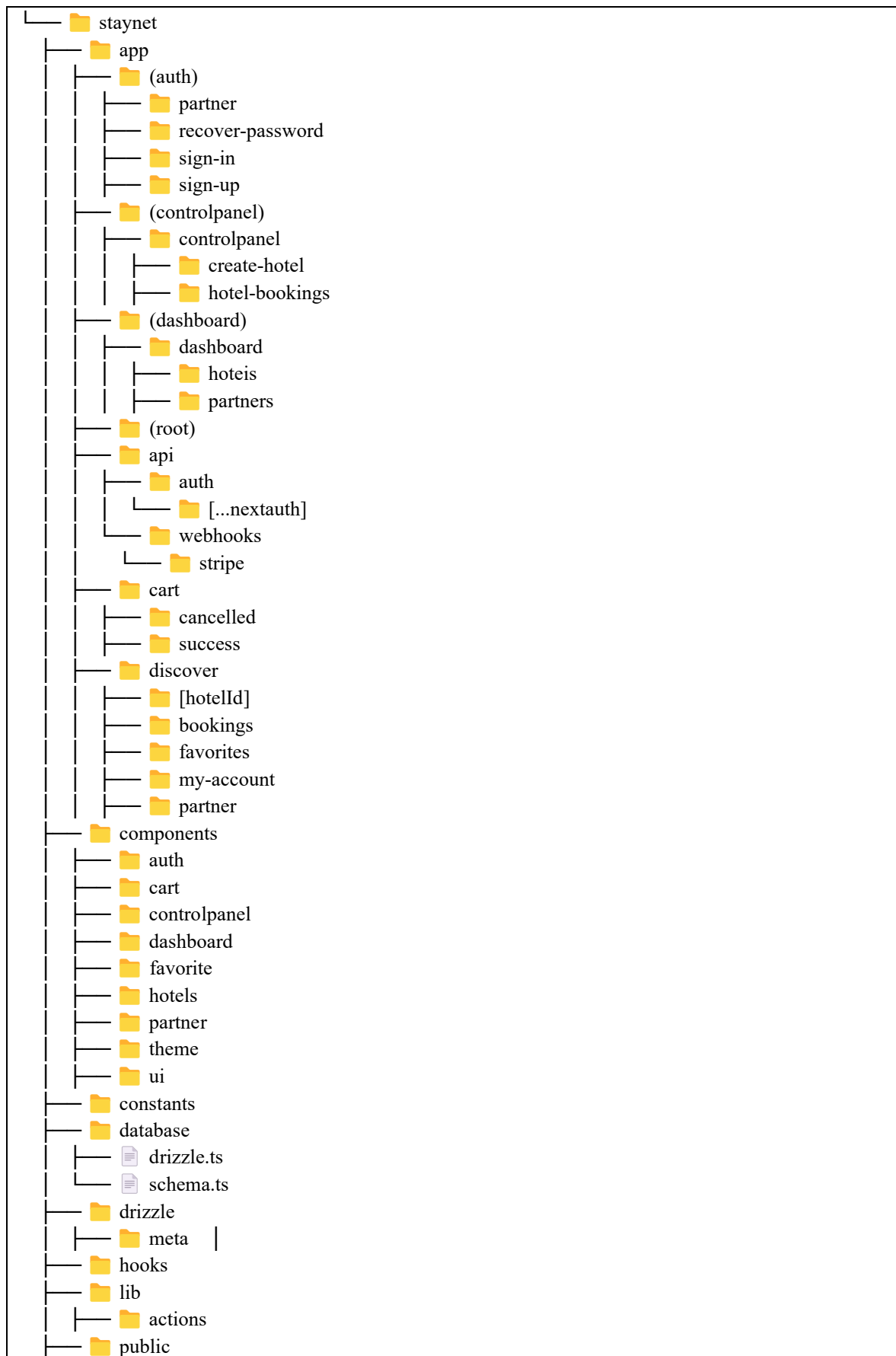


Figura 14 - Estrutura da plataforma

4.3 Criação da Base de Dados

A base de dados da StayNet foi criada no Neon Database, a qual foi estruturada para armazenar nomeadamente utilizadores, hotéis, quartos, reservas, pagamentos e parceiros.

Para a implementação, devido a esta base de dados ser na cloud, tivemos de aceder á plataforma online da neon onde criamos conta e copiamos a APi, após declararmos a API nas nossas variáveis de ambiente, utilizámos o ficheiro schema disponibilizado pelo Drizzle ORM, o que permite definir todas as tabelas e relações. Estas duas aplicações trabalham muito bem em conjunto pois após a definição das tabelas no schema o Drizzle também nos permite gerar e migrar a base de dados para o Neon.

```
import {
  uuid,
  text,
  pgTable,
  varchar,
  timestamp,
  pgEnum,
  integer,
  decimal,
  date,
  boolean,
} from "drizzle-orm/pg-core";
import { relations } from "drizzle-orm";

export const ROLE_ENUM = pgEnum("role", [
  "Admin",
  "Partner",
  "Pending",
  "User",
]);

export const applicationStatusEnum = pgEnum("application_status", [
  "Pending",
  "Approved",
  "Rejected",
]);

export const ROOM_TYPE_ENUM = pgEnum("room_type", ["Single", "Double"]);

export const BOOKING_STATUS_ENUM = pgEnum("booking_status", [
  "Pending",
  "Confirmed",
  "Cancelled",
  "Completed",
]);

export const users = pgTable("users", {
  id: uuid("id").primaryKey().defaultRandom(),
```

```

fullName: varchar("full_name", { length: 255 }).notNull(),
email: text("email").notNull().unique(),
password: text("password").notNull(),
role: ROLE_ENUM("role").notNull().default("User"),
emailVerified: boolean("email_verified").notNull().default(true),
createdAt: timestamp("created_at", {
  withTimezone: true,
}).defaultNow(),
});

export const verificationTokens = pgTable("verification_tokens", {
  id: uuid("id").primaryKey().defaultRandom(),
  email: text("email").notNull().unique(),

  fullName: varchar("full_name", { length: 255 }),
  passwordHash: text("password_hash"),

  tokenHash: text("token_hash").notNull().unique(),
  tokenType: text("token_type").notNull(),
  expiresAt: timestamp("expires_at", { withTimezone: true }).notNull(),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow(),
  nextEmailAt: timestamp("next_email_at").defaultNow(),
});

export const hotels = pgTable("hotels", {
  id: uuid("id").primaryKey().defaultRandom(),

  ownerId: uuid("owner_id")
    .notNull()
    .references(() => users.id, { onDelete: "cascade" }),

  name: varchar("name", { length: 255 }).notNull(),
  description: text("description"),
  address: text("address").notNull(),
  city: varchar("city", { length: 120 }).notNull(),
  country: varchar("country", { length: 120 }).default("Portugal"),
  email: text("email").notNull(),
  phone: varchar("phone", { length: 40 }),

  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});

export const hotelImages = pgTable("hotel_images", {
  id: uuid("id").primaryKey().defaultRandom(),

  hotelId: uuid("hotel_id")
    .notNull()
    .references(() => hotels.id, { onDelete: "cascade" }),

  url: text("url").notNull(),
  publicId: text("public_id").notNull(),
  order: integer("order").notNull().default(0),

  createdAt: timestamp("created_at").defaultNow(),
});

export const rooms = pgTable("rooms", {
  id: uuid("id").primaryKey().defaultRandom(),

```

```

hotelId: uuid("hotel_id")
  .NotNull()
  .references(() => hotels.id, { onDelete: "cascade" }),

roomType: ROOM_TYPE_ENUM("room_type").NotNull(),
pricePerNight: decimal("price_per_night", {
  precision: 10,
  scale: 2,
}).NotNull(),
totalRooms: integer("total_rooms").NotNull().default(0),
description: text("description"),

createdAt: timestamp("created_at").defaultNow(),
updatedAt: timestamp("updated_at").defaultNow(),
});

export const favorites = pgTable("favorites", {
  id: uuid("id").primaryKey().defaultRandom(),

  userId: uuid("user_id")
    .NotNull()
    .references(() => users.id, { onDelete: "cascade" }),

  hotelId: uuid("hotel_id")
    .NotNull()
    .references(() => hotels.id, { onDelete: "cascade" }),

  createdAt: timestamp("created_at").defaultNow(),
});

export const bookings = pgTable("bookings", {
  id: uuid("id").primaryKey().defaultRandom(),
  userId: uuid("user_id")
    .NotNull()
    .references(() => users.id, { onDelete: "cascade" }),
  roomId: uuid("room_id")
    .NotNull()
    .references(() => rooms.id, { onDelete: "cascade" }),
  checkInDate: date("check_in_date").NotNull(),
  checkOutDate: date("check_out_date").NotNull(),
  totalPrice: decimal("total_price", {
    precision: 10,
    scale: 2,
  }).NotNull(),
  status: BOOKING_STATUS_ENUM("status").NotNull().default("Pending"),
  checkedIn: boolean("checked_in").NotNull().default(false),
  specialRequests: text("special_requests"),

  stripeSessionId: text("stripe_session_id"),
  stripePaymentIntentId: text("stripe_payment_intent_id"),
  paidAt: timestamp("paid_at"),

  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});

export const partnerApplications = pgTable("partner_applications", {
  id: uuid("id").primaryKey().defaultRandom(),

```

```

    userId: uuid("user_id")
      .notNull()
      .unique()
      .references(() => users.id, { onDelete: "cascade" }),

    hotelName: varchar("hotel_name", { length: 255 }).notNull(),
    description: text("description").notNull(),

    status: applicationStatusEnum("status").notNull().default("Pending"),

    createdAt: timestamp("created_at").defaultNow(),
  });

export const usersRelations = relations(users, ({ one, many }) => ({
  hotels: many(hotels),
  bookings: many(bookings),
  partnerApplication: one(partner.Applications),
  favorites: many(favorites),
}));

export const hotelsRelations = relations(hotels, ({ one, many }) => ({
  owner: one(users, {
    fields: [hotels.ownerId],
    references: [users.id],
  }),
  rooms: many(rooms),
  images: many(hotelImages),
  favorites: many(favorites),
}));

export const favoritesRelations = relations(favorites, ({ one }) => ({
  user: one(users, {
    fields: [favorites.userId],
    references: [users.id],
  }),
  hotel: one(hotels, {
    fields: [favorites.hotelId],
    references: [hotels.id],
  }),
}));

export const hotelImagesRelations = relations(hotelImages, ({ one }) => ({
  hotel: one(hotels, {
    fields: [hotelImages.hotelId],
    references: [hotels.id],
  }),
}));

export const roomsRelations = relations(rooms, ({ one, many }) => ({
  hotel: one(hotels, {
    fields: [rooms.hotelId],
    references: [hotels.id],
  }),
  bookings: many(bookings),
}));

export const bookingsRelations = relations(bookings, ({ one }) => ({
  user: one(users, {
    fields: [bookings.userId],
    references: [users.id],
  }),
}));

```



```

    }),
    room: one(rooms, {
      fields: [bookings.roomId],
      references: [rooms.id],
    }),
  }));

export const partnerApplicationsRelations = relations(
  partnerApplications,
  ({ one }) => ({
    user: one(users, {
      fields: [partnerApplications.userId],
      references: [users.id],
    }),
  }),
);

```

Código 25 - Código do Schema

4.4 Ligação á Base de Dados

A ligação á base de dados é realizada no ficheiro `drizzle.ts`, este

```

import config from "@lib/config";
import { drizzle } from "drizzle-orm/neon-http";
import { neon } from "@neondatabase/serverless";

const sql = neon(config.env.databaseUrl);
export const db = drizzle(process.env.DATABASE_URL!);

```

Código 26 - Conexão Drizzle e Neon Data Base

4.5 Armazenamento e Upload de Imagens

Nesta Plataforma decidimos armazenar todas as imagens da Plataforma StayNet numa base de dados secundaria, com o intuito de não saturar a base de dados principal.

Para esta tarefa optamos pela Cloudinary, esta ferramenta é uma base de dados na cloud que permite armazenar e gerir imagens. A conexão foi feita através da base de dados da Neon que guarda apenas a referência às imagens, através da tabela (`hotel_images`), onde são armazenados:

- URL público da imagem;
- identificador interno do Cloudinary (publicId);
- a relação com o hotel correspondente (hotelId);
- a ordem das imagens.

Desta forma, a Cloudinary é responsável pelo armazenamento, enquanto a Neon apenas guarda as referências de cada imagem.

O upload de imagens é realizado pelo componente (CldUploadWidget), o qual é disponibilizado pela própria Cloudinary, e que permite ao utilizador inserir todas as imagens de forma organizada e segura.

```
export const hotelImages = pgTable("hotel_images", {  
  id: uuid("id").primaryKey().defaultRandom(),  
  
  hotelId: uuid("hotel_id")  
    .notNull()  
    .references(() => hotels.id, { onDelete: "cascade" }),  
  
  url: text("url").notNull(),  
  publicId: text("public_id").notNull(),  
  order: integer("order").notNull().default(0),  
  
  createdAt: timestamp("created_at").defaultNow(),  
});
```

Código 27 - Armazenamento de Fotos

```

<CldUploadWidget
  uploadPreset={process.env.NEXT_PUBLIC_CLOUDINARY_UPLOAD_PRESET}
  options={{
    multiple: true,
    maxFiles: 20,
  }}
  onSuccess={(result: any) => {
    const info = result?.info;
    const url = info?.secure_url;
    const publicId = info?.public_id;

    if (url && publicId) addImage(url, publicId);
  }}
>
  {{{ open }}} => (
    <button
      type="button"
      onClick={() => open()}
      className="w-full h-32 border-2 border-dashed rounded-lg flex flex-col items-center justify-center
      hover:border-gray-400 transition-colors"
    >
      <ImagePlus className="h-8 w-8 text-gray-400" />
      <p className="text-sm text-gray-500 mt-2">
        Clica para adicionar imagens
      </p>
      <p className="text-xs text-gray-400 mt-1">
        ({images.length} selecionada(s))
      </p>
    </button>
  )}
</CldUploadWidget>

```

Código 28 - Lógica de Upload

4.6 Autenticação de Utilizadores e Gestão de Sessões

A autenticação da plataforma da StayNet foi disponibilizada pela biblioteca Auth.js (NextAuth), que integramos diretamente no Next.js. O processo de autenticação, é realizado através da verificação e comparação das credenciais inseridas, com as que foram guardadas na base de dados. Para os dados de autenticação não serem guardados em texto plano utilizámos o Bcryptjs, que guarda as credenciais em hashes únicas, e adiciona uma camada extra de proteção a ataques.

A Gestão de sessões é feita pelo JWT (JSON Web Tokens), que permite que os utilizadores fiquem autenticados de forma segura, sem ser necessário guardar informações no servidor. É guardado o seu id e o seu role, o que permite depois para controlar o acesso a cada página distinguindo, utilizadores Admin, Partner, Pending e User, assegurando que apenas conseguem aceder às funcionalidades permitidas.

Também foi utilizado o ficheiro (proxy.ts) disponibilizado em novas versões do Next.js, que permite gerir as rotas e as chamadas às APIs de forma organizada e segura.

```
import NextAuth, { User } from "next-auth";
import { compare } from "bcryptjs";
import CredentialsProvider from "next-auth/providers/credentials";
import { db } from "@/database/drizzle";
import { users } from "@/database/schema";
import { eq } from "drizzle-orm";

export const { handlers, signIn, signOut, auth } = NextAuth({
  session: { strategy: "jwt" },
  providers: [
    CredentialsProvider({
      async authorize(credentials) {
        if (!credentials?.email || !credentials?.password) {
          return null;
        }

        const user = await db
          .select()
          .from(users)
          .where(eq(users.email, credentials.email.toString()))
          .limit(1);

        if (user.length === 0) return null;

        const isValid = await compare(
          credentials.password.toString(),
          user[0].password,
        );
        if (!isValid) return null;

        return {
          id: user[0].id.toString(),
          email: user[0].email,
          name: user[0].fullName,
          role: user[0].role,
        } as User & { role: string };
      },
    },
  ],
  pages: {
    signIn: "/sign-in",
  },
  callbacks: {
    async jwt({ token, user }) {
      if (user) {
        token.id = user.id;
        token.name = user.name;
        token.role = (user as User & { role: string }).role;
      }
      return token;
    },
    async session({ session, token }) {
```

```

    if (session.user) {
      session.user.id = token.id as string;
      session.user.name = token.name as string;
      session.user.role = token.role as
        | "Admin"
        | "Partner"
        | "Pending"
        | "User";
    }
    return session;
  },
},
});
});

```

Código 29 - Autenticação

```

import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";
import { getToken } from "next-auth/jwt";

const secret = process.env.AUTH_SECRET;

export async function proxy(request: NextRequest) {
  const url = request.nextUrl.clone();
  const pathname = url.pathname;

  const token = await getToken({ req: request, secret });

  if (pathname === "/sign-in" || pathname === "/sign-up") {
    if (token) {
      url.pathname = "/discover";
      return NextResponse.redirect(url);
    }
    return NextResponse.next();
  }

  const privateRoutes = ["/dashboard", "/controlpanel", "/cart", "/discover"];
  if (privateRoutes.some((route) => pathname.startsWith(route))) {
    if (!token) {
      url.pathname = "/sign-in";
      return NextResponse.redirect(url);
    }

    if (pathname.startsWith("/dashboard") && token.role !== "Admin") {
      url.pathname = "/";
      return NextResponse.redirect(url);
    }

    if (
      pathname.startsWith("/controlpanel") &&
      token.role !== "Partner" &&
      token.role !== "Admin"
    ) {
      url.pathname = "/";
      return NextResponse.redirect(url);
    }
  }

  return NextResponse.next();
}

```

```

}

export const config = {
  matcher: [
    "/dashboard/:path*",
    "/controlpanel/:path*",
    "/cart/:path*",
    "/discover/:path*",
    "/sign-in",
    "/sign-up",
  ],
};

```

Código 30 - Proxy

4.7 Rotas e API

Todas as rotas da plataforma são feitas através do sistema de rotas automático do Next.js, que permite organizar melhor as páginas no desenvolvimento do código, e ser mais fácil de pôr em prática.

Quanto às APIs, utilizamos a API do Stripe para fazer o processamento de pagamentos (Stripe Webhooks).

```

import { NextRequest, NextResponse } from "next/server";
import Stripe from "stripe";
import { confirmBookingPayment } from "@lib/actions/checkout.actions";
import { releaseHold } from "@lib/redis";

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!, {
  apiVersion: "2025-12-15.clover",
});

const webhookSecret = process.env.STRIPE_WEBHOOK_SECRET!;

export async function POST(request: NextRequest) {
  const body = await request.text();
  const signature = request.headers.get("stripe-signature")!;

  let event: Stripe.Event;

  try {
    event = stripe.webhooks.constructEvent(body, signature, webhookSecret);
  } catch (err) {
    console.error("Webhook signature verification failed:", err);
    return NextResponse.json({ error: "Invalid signature" }, { status: 400 });
  }

  try {
    switch (event.type) {
      case "checkout.session.completed":
        const session = event.data.object as Stripe.Checkout.Session;

```

```

    await confirmBookingPayment(session.id);

    if (session.metadata?.holdIds) {
      const holdIds = session.metadata.holdIds.split(",");
      for (const holdId of holdIds) {
        await releaseHold(holdId);
      }
    }

    console.log("Payment confirmed:", session.id);
    break;

  case "checkout.session.expired":
    const expiredSession = event.data.object as Stripe.Checkout.Session;

    if (expiredSession.metadata?.holdIds) {
      const holdIds = expiredSession.metadata.holdIds.split(",");
      for (const holdId of holdIds) {
        await releaseHold(holdId);
      }
    } else if (expiredSession.metadata?.holdId) {
      await releaseHold(expiredSession.metadata.holdId);
    }

    console.log("Session expired and holds released:", expiredSession.id);
    break;

  case "payment_intent.succeeded":
    const paymentIntent = event.data.object as Stripe.PaymentIntent;
    console.log("Payment succeeded:", paymentIntent.id);
    break;

  case "payment_intent.payment_failed":
    const failedPayment = event.data.object as Stripe.PaymentIntent;
    console.log("Payment failed:", failedPayment.id);
    break;

  default:
    console.log("Unhandled event type:", event.type);
}

return NextResponse.json({ received: true });
} catch (error) {
  console.error("Error processing webhook:", error);
  return NextResponse.json(
    { error: "Webhook processing failed" },
    { status: 500 },
  );
}
}

```

Código 31 - Stripe

4.8 Armazenamento Temporário

Para tratarmos do armazenamento temporário decidimos utilizar a biblioteca Ioredis. Esta biblioteca ajuda a integrar o sistema Redis, que é um sistema de armazenamento em memória, utilizado principalmente para cache e dados temporários.

Na Staynet o Redis está a funcionar no Docker dentro de um container, com a função de gerir reservas temporárias de quartos, permitindo bloquear um quarto enquanto o utilizador finaliza o pagamento.

```
import { Redis } from "ioredis";

const getRedisUrl = () => {
  if (process.env.REDIS_URL) {
    return process.env.REDIS_URL;
  }

  return "redis://localhost:6379";
};

export const redis = new Redis(getRedisUrl(), {
  maxRetriesPerRequest: 3,
  retryStrategy(times) {
    const delay = Math.min(times * 50, 2000);
    return delay;
  },
  lazyConnect: true,
});

redis.connect().catch((err) => {
  console.error("Erro ao conectar ao Redis:", err);
});

redis.on("error", (err) => {
  console.error("Redis Client Error", err);
});

redis.on("connect", () => {
  console.log("Redis conectado com sucesso");
});

export async function createTemporaryHold(
  roomId: string,
  checkIn: string,
  checkOut: string,
  userId: string,
): Promise<string> {
  const holdId = `hold:${roomId}:${Date.now()}:${userId}`;
  const holdData = {
    roomId,
    checkIn,
    checkOut,
    userId,
  };
}
```



```

    createdAt: new Date().toISOString(),
  };

  await redis.setex(holdId, 1800, JSON.stringify(holdData));
  return holdId;
}

export async function getActiveHolds(
  roomId: string,
  checkIn: string,
  checkOut: string,
): Promise<number> {
  const pattern = `hold:${roomId}.*`;
  const keys = await redis.keys(pattern);

  let count = 0;
  for (const key of keys) {
    const data = await redis.get(key);
    if (data) {
      const hold = JSON.parse(data);
      if (hold.checkIn < checkOut && hold.checkOut > checkIn) {
        count++;
      }
    }
  }

  return count;
}

export async function releaseHold(holdId: string): Promise<void> {
  await redis.del(holdId);
}

export async function extendHold(holdId: string): Promise<boolean> {
  const ttl = await redis.ttl(holdId);
  if (ttl > 0) {
    await redis.expire(holdId, 1800);
    return true;
  }
  return false;
}

```

Código 32 - Redis

4.9 Envio de Emails Automático

O envio de emails na StayNet é gerido através da biblioteca Resend, esta biblioteca foi integrada com as Server Actions, possibilitando o envio dos emails de forma automática quando é chamada uma determinada função, como por exemplo:

- Confirmação de email de novos utilizadores.
- Recuperação e alteração de palavra-passe.

```
const resend = new Resend(process.env.RESEND_API_KEY);

export async function sendPasswordResetLink(email: string): Promise<{
  success: boolean;
  error?: string;
}> {
  constresetUrl=`${process.env.NEXT_PUBLIC_APP_URL}/reset-
password?token=TOKEN_DE_EXEMPLO`;

  const { error: emailError } = await resend.emails.send({
    from: "StayNet <onboarding@resend.dev>",
    to: [email.toLowerCase()],
    subject: "Recuperação de Password - StayNet",
    react: PasswordResetEmail({
      resetUrl,
      expiresInMinutes: 60,
    }),
  });

  if (emailError) {
    console.error("Erro ao enviar email:", emailError);
    return { success: false, error: "Erro ao enviar email. Tente novamente." };
  }

  return { success: true };
}
```

Código 33 - Alterar e recuperar password

```
const resend = new Resend(process.env.RESEND_API_KEY);

const verificationUrl = `${process.env.NEXT_PUBLIC_APP_URL}/verify-email?token=${token}`;

const { error: emailError } = await resend.emails.send({
  from: "StayNet <onboarding@resend.dev>",
  to: [email.toLowerCase()],
  subject: "Verifique o seu email - StayNet",
  react: VerifyEmailTemplate({
    fullName,
    verificationUrl,
  }),
});

if (emailError) {
  console.error("Erro ao enviar email:", emailError);
}
```

Código 34 - Email de verificação

4.10 Docker

Toda a plataforma StayNet corre dentro de contentores separados no Docker, como, o Redis, o Stripe e o ambiente web.

Para garantir a consistência, compatibilidade com todos os sistemas operativos e permitir uma produção mais fácil da plataforma, foi criada uma imagem Docker, permitindo executar a aplicação de forma idêntica em qualquer máquina.

```
services:
  redis:
    image: redis:7-alpine
    container_name: hotel-redis
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data
    command: redis-server --appendonly yes
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5
      restart: unless-stopped

  web:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: hotel-web
    ports:
      - "3000:3000"
    volumes:
      - ./app
      - /app/node_modules
      - /app/.next
    env_file:
      - .env
    environment:
      NODE_ENV: development
      REDIS_URL: redis://redis:6379
      NEXT_PUBLIC_APP_URL: http://localhost:3000
    depends_on:
```

```
redis:
  condition: service_healthy
  command: npm run dev
  restart: unless-stopped

stripe-cli:
  image: stripe/stripe-cli:latest
  container_name: hotel-stripe-cli
  env_file:
    - .env
  command: >
    listen
    --api-key ${STRIPE_SECRET_KEY}
    --device-name ${STRIPE_DEVICE_NAME}
    --forward-to web:3000/api/webhooks/stripe
  depends_on:
    - web
  restart: unless-stopped

volumes:
  redis_data:
```

Código 35 - Docker compose

4.11 Validação e Segurança

Ao longo do desenvolvimento da StayNet foram adicionados vários métodos de validação, tanto no frontend quanto no backend. Assim permitindo verificar que os dados inseridos pelos utilizadores são corretos e seguros. Foram adicionadas medidas como verificação de campos obrigatórios, validação de emails, hashing de passwords com o bcryptjs, controlo de permissões com JWT e proteção das rotas e APIs.

No desenvolvimento também garantimos o uso correto das diretivas "use server" e "use client", para separar o código do frontend do backend. Garantido que certas funções e validações, corram apenas no backend e não sejam visualizadas por nenhum utilizador no frontend.

```

import { z } from "zod";

export const signUpSchema = z.object({
  fullName: z.string().min(3),
  email: z.string().email(),
  password: z.string().min(8),
});

export const signInSchema = z.object({
  email: z.string().email(),
  password: z.string().min(8),
});

export const recoverPasswordSchema = z.object({
  email: z.string().email(),
});

export const imageSchema = z.object({
  url: z.string().min(1),
  publicId: z.string().min(1),
});

export const createHotelSchema = z
  .object({
    name: z.string().min(2, "Indica o nome do hotel."),
    description: z.string().optional(),
    address: z.string().min(5, "Indica a morada."),
    city: z.string().min(2, "Indica a cidade."),
    country: z.string().min(2, "Indica o país."),
    email: z.string().email("Email inválido."),
    phone: z.string().optional(),
    images: z.array(imageSchema).default([]),

    singleRoomTotal: z.number().int().min(0, "Quantidade inválida").default(0),
    singleRoomPrice: z.number().positive("Preço deve ser positivo").optional(),
    singleRoomDescription: z.string().optional(),

    doubleRoomTotal: z.number().int().min(0, "Quantidade inválida").default(0),
    doubleRoomPrice: z.number().positive("Preço deve ser positivo").optional(),
    doubleRoomDescription: z.string().optional(),
  })
  .refine(
    (data) => {
      return data.singleRoomTotal > 0 || data.doubleRoomTotal > 0;
    },
    {
      message: "É obrigatório configurar pelo menos um tipo de quarto",
      path: ["singleRoomTotal"],
    },
  )
  .refine(
    (data) => {
      return !(data.singleRoomTotal > 0 && !data.singleRoomPrice);
    },
    {
      message: "Preço é obrigatório quando há quartos single configurados",
      path: ["singleRoomPrice"],
    },
  )
  .refine(

```

```
(data) => {  
  return !(data.doubleRoomTotal > 0 && !data.doubleRoomPrice);  
},  
{  
  message: "Preço é obrigatório quando há quartos double configurados",  
  path: ["doubleRoomPrice"],  
},  
);
```

Código 36 – Validações de formulários

Capítulo 5

Testes/Avaliação/Discussão

5.1 Apresentação de Resultados

Com o backend e o frontend da StayNet já implementados, com a base de dados ligada e tudo a comunicar como o pretendido, a próxima etapa é pôr a plataforma à prova. Ou seja, testar os fluxos mais importantes para garantir que a pesquisa, carrinho, favoritos, reservas, pagamento e a parte de gestão do parceiro funcionam bem.

Depois disso, é altura de mostrar os resultados e explicar o que foi alcançado. No fundo, comprovar que aquilo que começou como uma ideia acabou por se transformar numa plataforma real, funcional e pronta a ser usada, tanto por quem reserva como por quem gere os hotéis.

5.1.1 Resultados Página Inicial

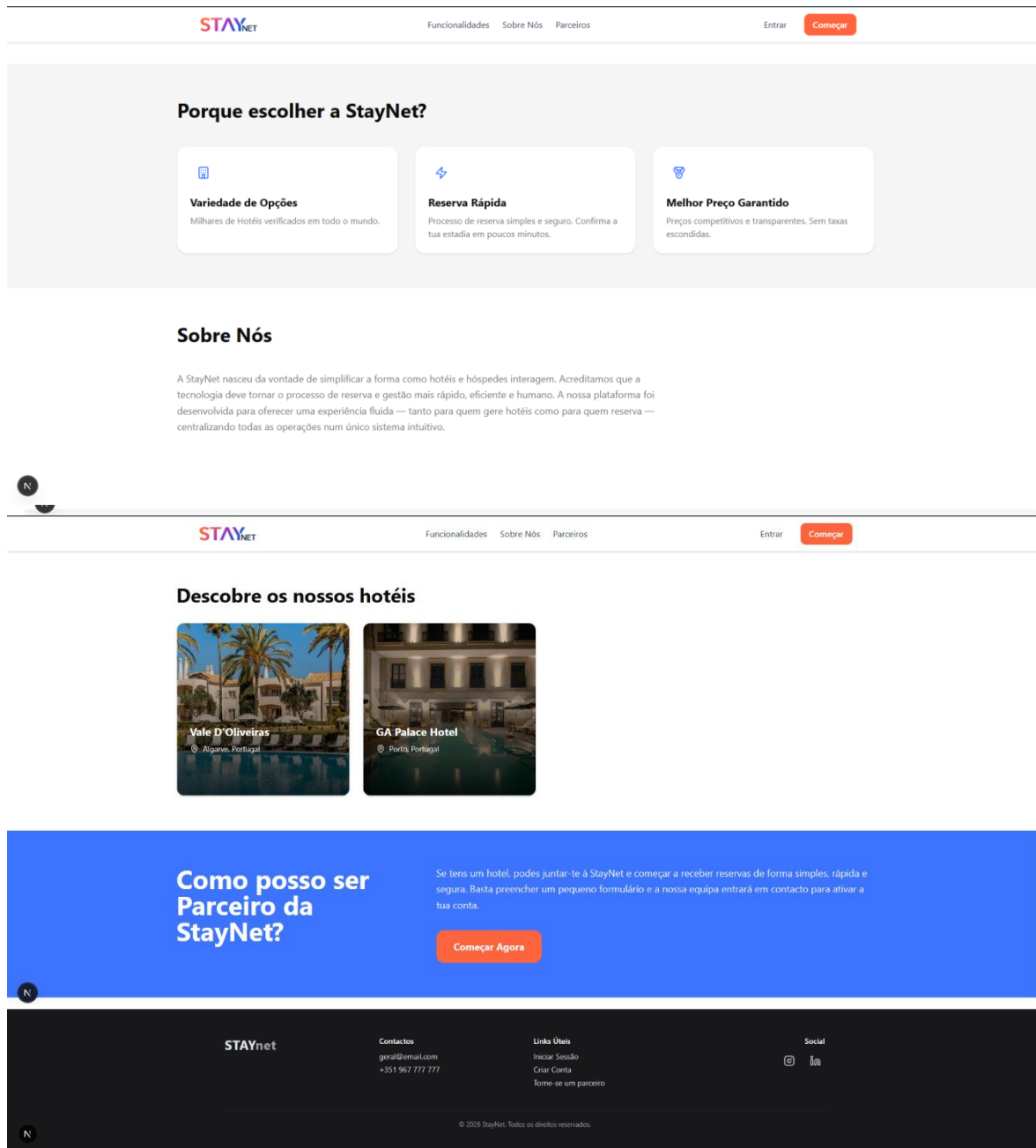


Figura 15 - Resultados Página Inicial

5.1.2 Login

Na página de login, foram introduzidas credenciais incorretas de forma intencional, para confirmar o comportamento do sistema perante um acesso inválido. Como esperado, foi apresentada uma mensagem de erro, indicando que os dados introduzidos não eram válidos.

De seguida, ao inserir credenciais válidas já registadas na base de dados, foi possível iniciar sessão com sucesso e aceder à plataforma, confirmando o correto funcionamento do processo de autenticação. Caso o utilizador não saiba a palavra-passe, recebe um email de recuperação.

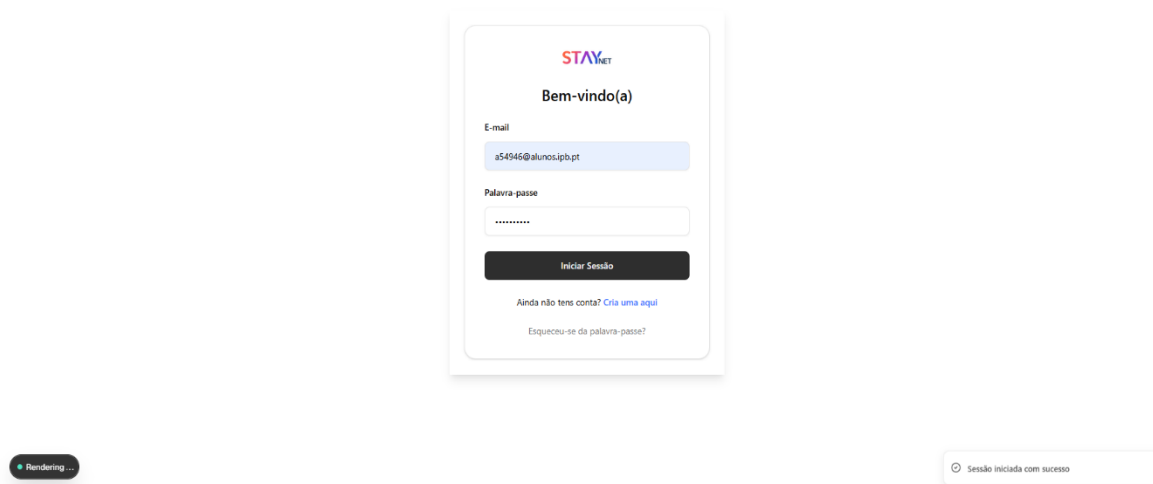


Figura 16 - Página de Login com credenciais corretas



Bem-vindo(a)

E-mail

a54946@alunos.ipb.pt

Palavra-passe

Iniciar Sessão

Ainda não tens conta? [Cria uma aqui](#)

[Esqueceu-se da palavra-passe?](#)

⊗ Erro ao iniciar sessão

Figura 17 - Página de Login com credenciais incorretas



Recuperar Password

E-mail

a54946@alunos.ipb.pt

Recuberá um email com um link seguro para redefinir a sua password.

Enviar Link de Recuperação

Já tens conta? [Iniciar sessão](#)

Figura 18 - Recuperar palavra-passe

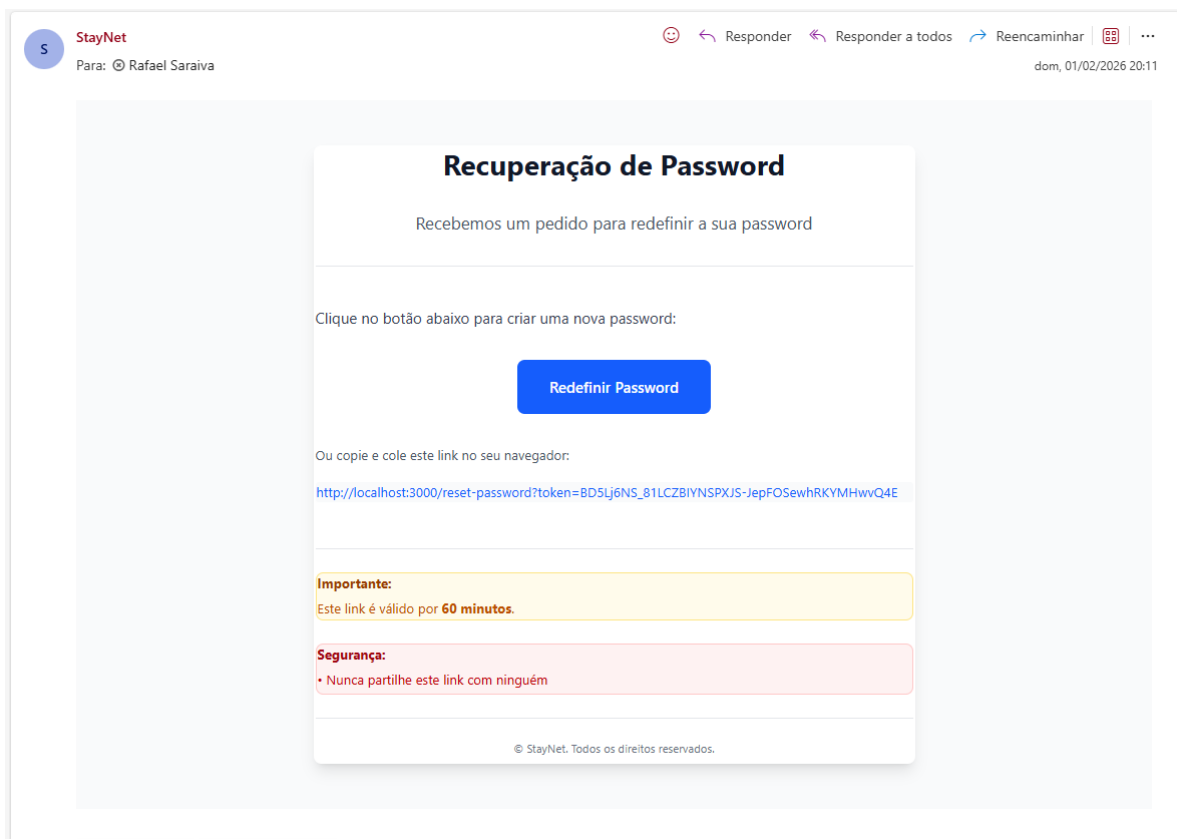


Figura 19 - Email de Recuperação

5.1.3 Registrar

Na página de registrar o utilizador cria a conta com as credenciais que pretender, o processo só fica concluído após a receção de um email a pedir para verificar conta.

O formulário 'Criar Conta' do StayNet contém os seguintes campos: 'Nome Completo' com o valor 'gustavo silva', 'E-mail' com o valor 'a51848@alunos.ipb.pt', e 'Palavra-passe' com caracteres ocultos por pontos. Um botão 'Criar Conta' está visível, assim como um link 'Já tens conta? Iniciar sessão'.

📧 Já foi enviado um email de verificação. Verifique a sua caixa de entrada.

Figura 20 - Conta criada com sucesso

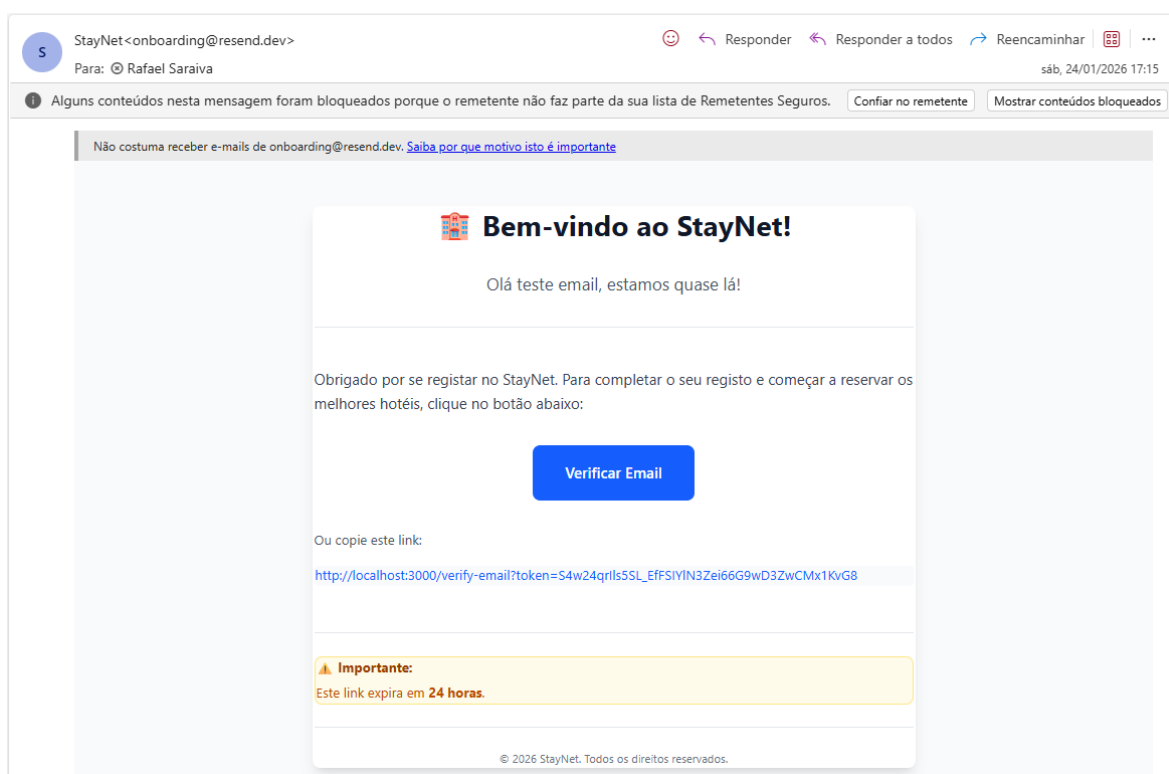


Figura 21 - Email de verificação de conta

5.1.4 Área de utilizador

Nesta secção apresentam-se os resultados obtidos com a implementação desta área, demonstrando o comportamento da plataforma com diferentes contas e confirmando que a informação apresentada é sempre a correta para cada utilizador. Também são validadas as funcionalidades essenciais do utilizador tais como:

- Página de início com pesquisa
- Detalhes sobre os hotéis
- Carrinho de reservas
- Processo de pagamento
- Minhas reservas
- Favoritos
- Candidatura a parceiro
- Página informativa
- Ver conta

Esta área também conta com uma navbar que implementamos com as funcionalidades já mencionadas e que também conta com um botão que permite alterar o tema da plataforma entre dark, light ou predefinido do sistema.

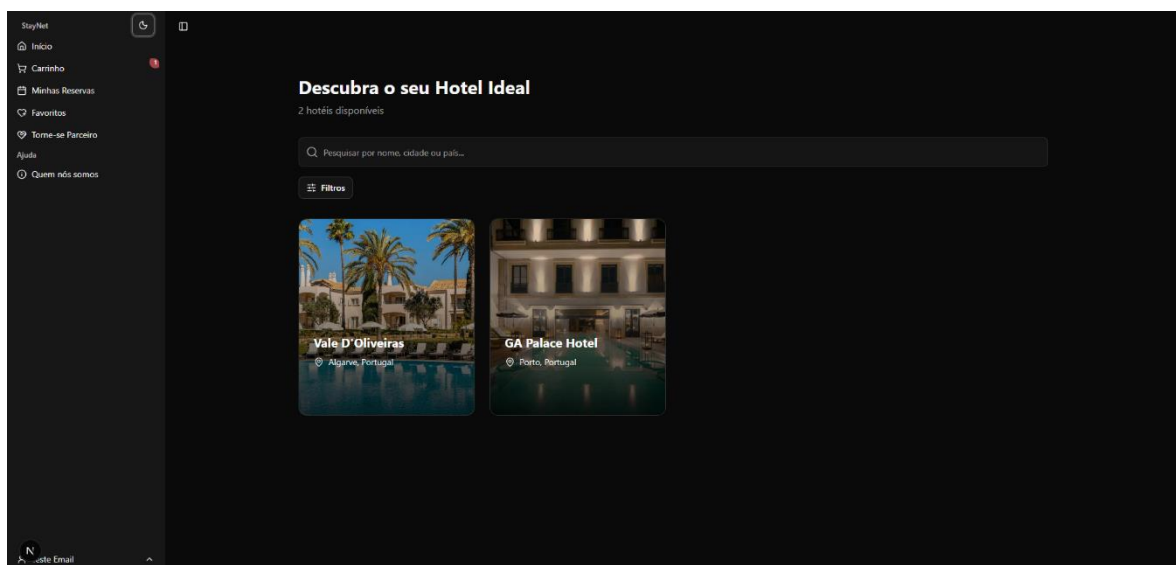


Figura 22 - Página de início com pesquisa

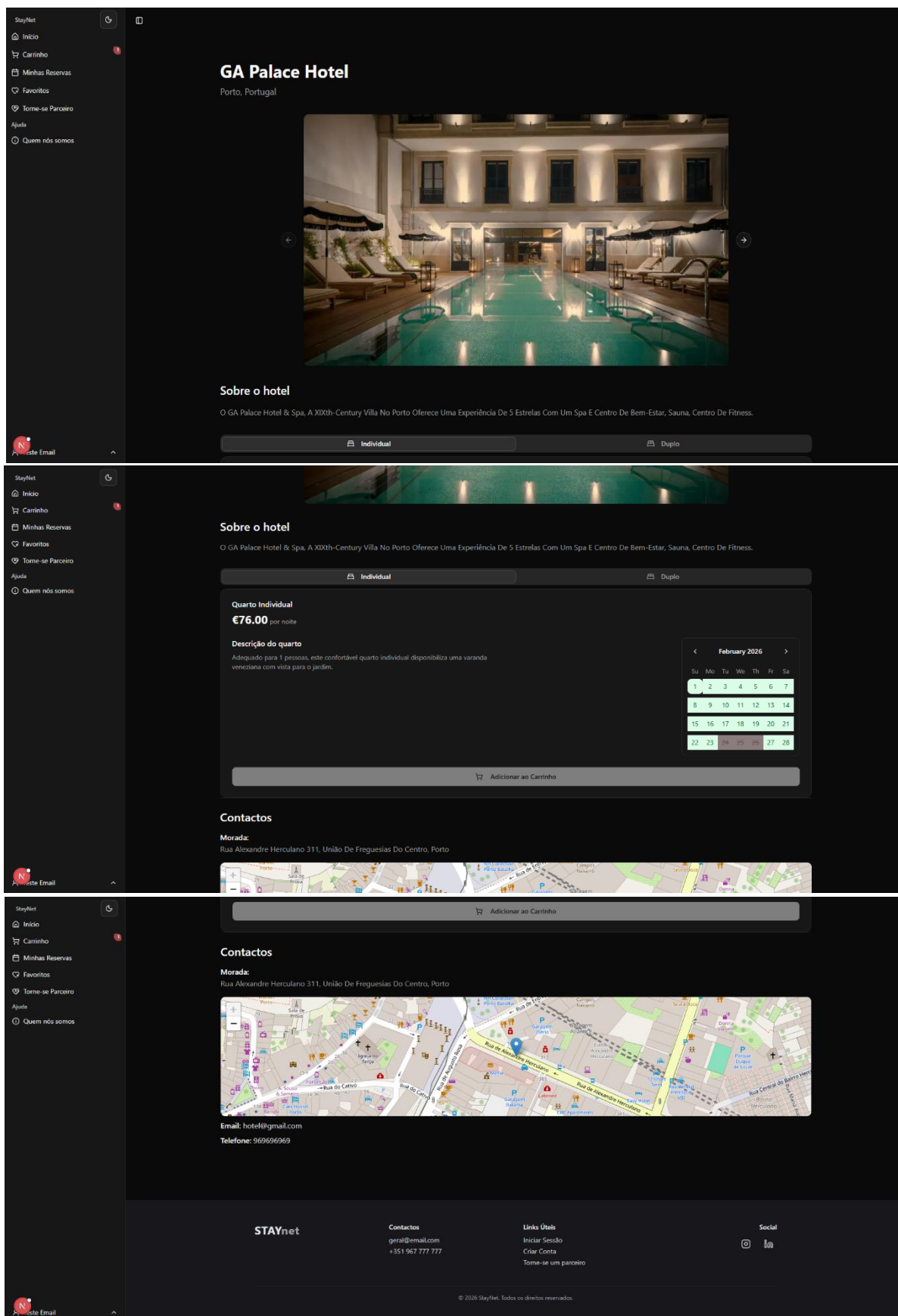


Figura 23 - Página de detalhes dos hotéis

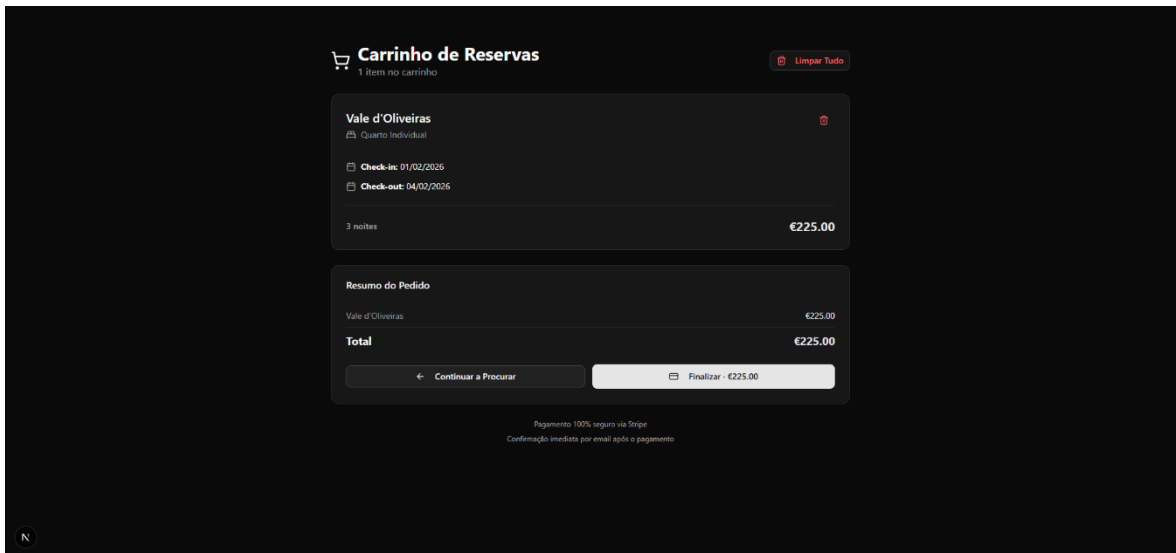


Figura 24 Carrinho de Reservas

← Área restrita de staynet

GA Palace Hotel - Quarto Individual

€ 228,00

3 noites (03/02/2026 - 06/02/2026)

Pagar com link

OU

Dados para contato

E-mail: a54946@alunos.ipb.pt

Forma de pagamento

Dados do cartão

1234 1234 1234 1234

MM / AA CVC

Nome do titular do cartão

Nome completo

País ou região

Portugal

☐ Salve minhas informações para um checkout mais rápido

Pague com segurança em Área restrita de staynet e em todos os lugares que aceitam o link

Pagar

Powered by stripe | Termos | Privacidade

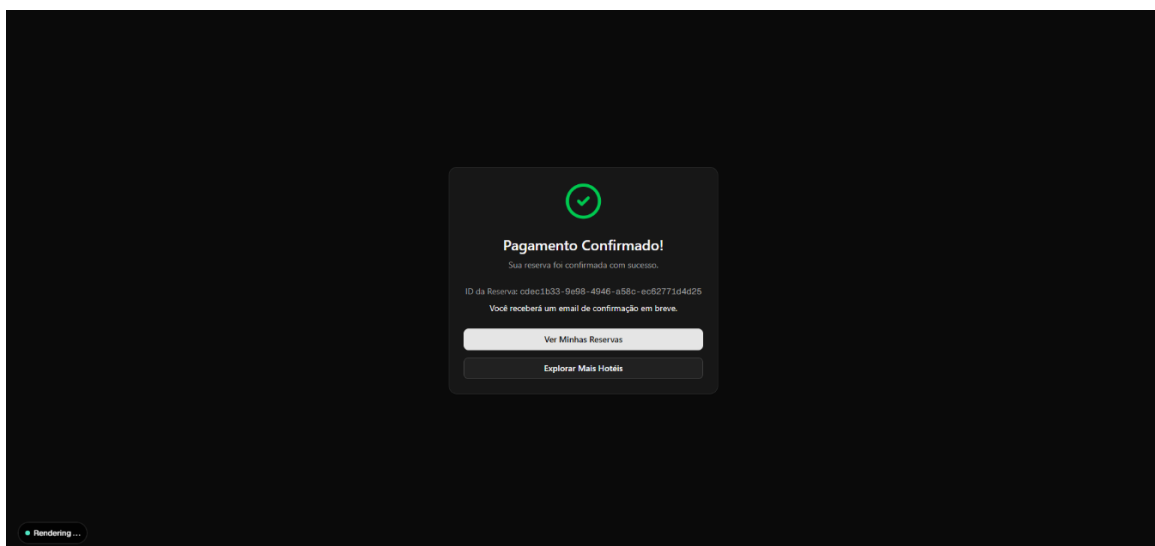


Figura 25 – Processo de Pagamento

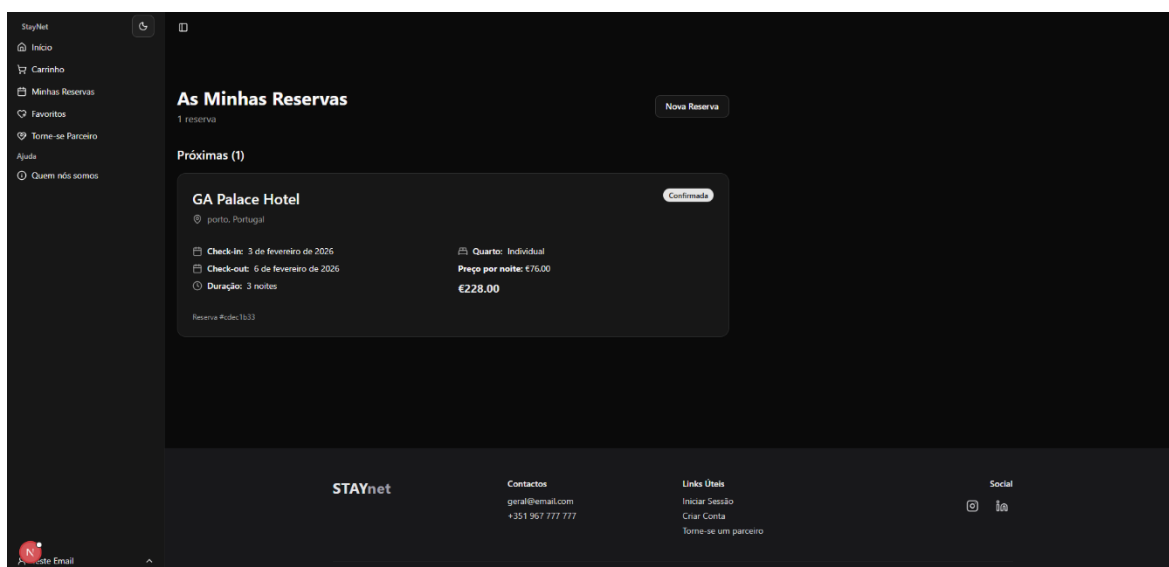


Figura 26 - As minhas reservas

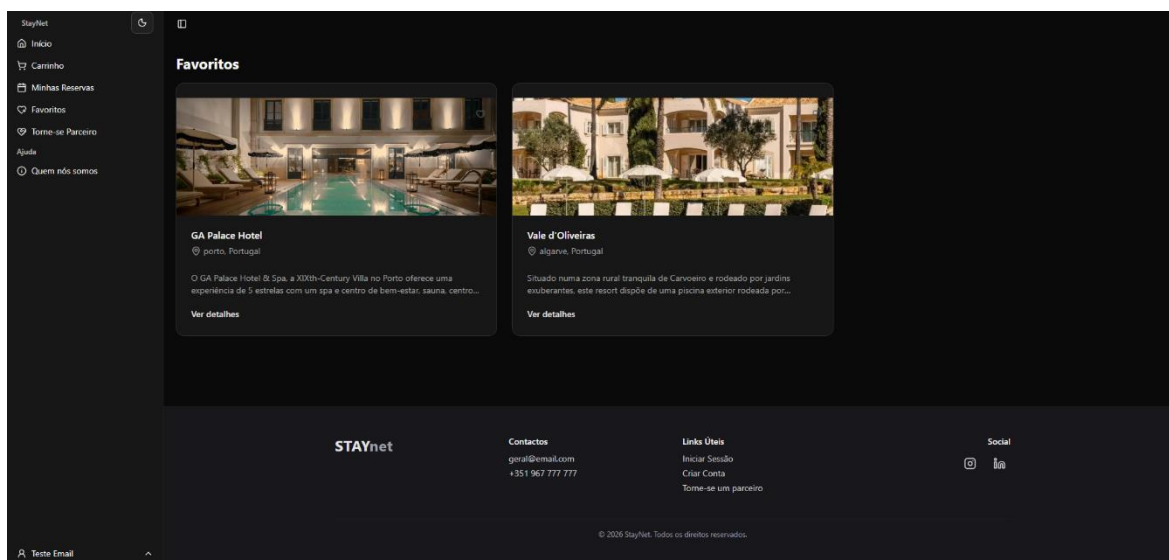


Figura 27 - Favoritos

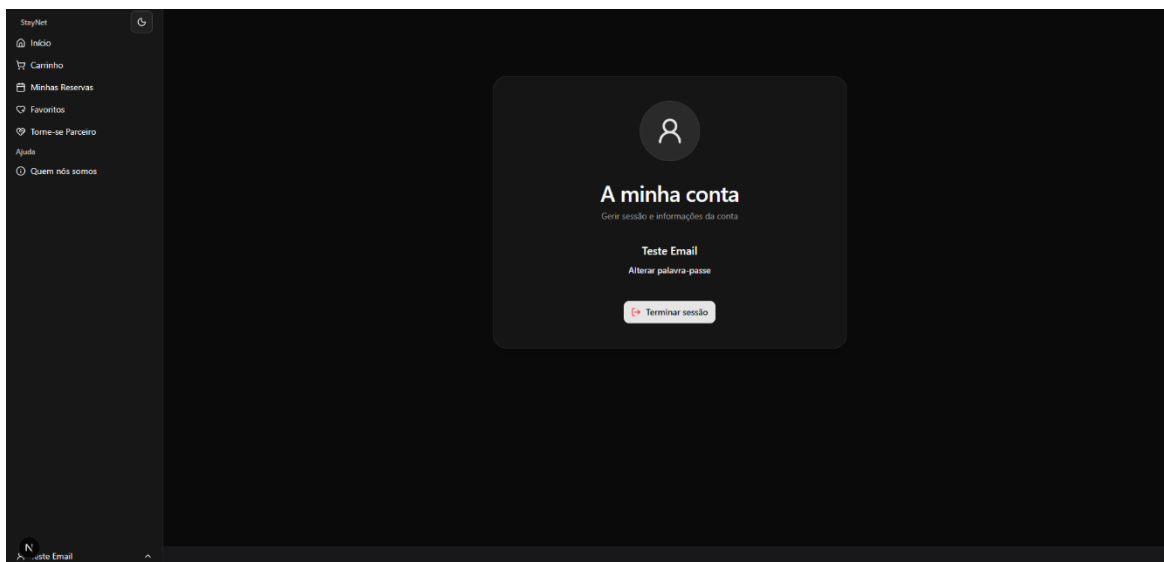


Figura 30 - Ver conta

5.1.5 Área de Parceiro

A Área de Parceiro é o painel onde o parceiro trata do lado “do hotel” na StayNet. Aqui consegue gerir hotéis e quartos, acompanhar e confirmar reservas, ver hóspedes e registar check-in/check-out, tudo no mesmo sítio.

Nesta secção mostram-se os resultados desta implementação, comprovando que estas ações funcionam como esperado e que a gestão fica mais simples e organizada para quem opera o hotel.

Este painel também conta com uma navbar com as funcionalidades do parceiro, como mostra na figura 31.

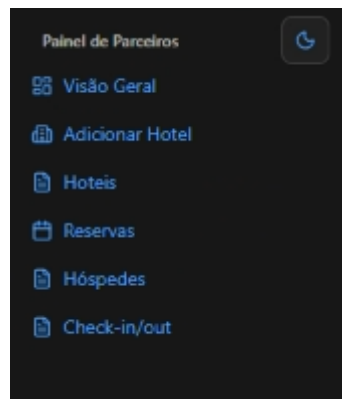


Figura 31 - Navbar do Parceiro

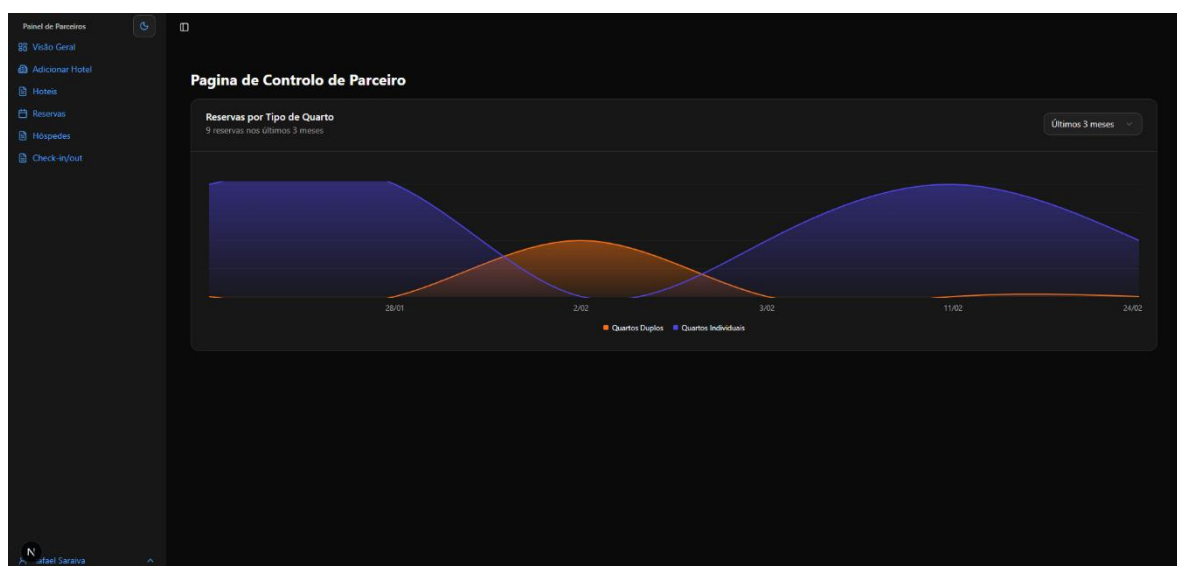


Figura 32 - Painel de Parceiro

Panel de Painéis

Visão Geral

Adicionar Hotel

Hotes

Reservas

Hóspedes

Check-in/out

N. Rafael Saraiva

Novo Hotel

Adiciona o teu hotel à plataforma e torna-o visível para os clientes.

Informações Básicas

Dados gerais do hotel

Nome do hotel

Ex: Hotel Bragança

Descrição

Breve descrição do hotel

Cidade

Ex: Bragança

País

Portugal

Morada

Rua... nº...-...

Email de contacto

hotel@email.com

Telefone (opcional)

+351 123 456 789

Configuração de Quartos

Configure os tipos de quartos disponíveis (obrigatório pelo menos um tipo)

Quartos Individuais

Quantidade de quartos

0

Preço por noite (€)

0.00

Número total de quartos individuais

Descrição

Descrição dos quartos individuais...

Quartos Duplos (Opcional)

Quantidade de quartos

0

Preço por noite (€)

0.00

Número total de quartos duplos

Descrição

Descrição dos quartos duplos...

É obrigatório configurar pelo menos um tipo de quarto para adicionar o hotel!

Imagens do Hotel

Adicione fotos do hotel (opcional)

Clica para adicionar imagens

(0 seleccionada)

Criar hotel

Figura 33 - Adicionar Hotel

92

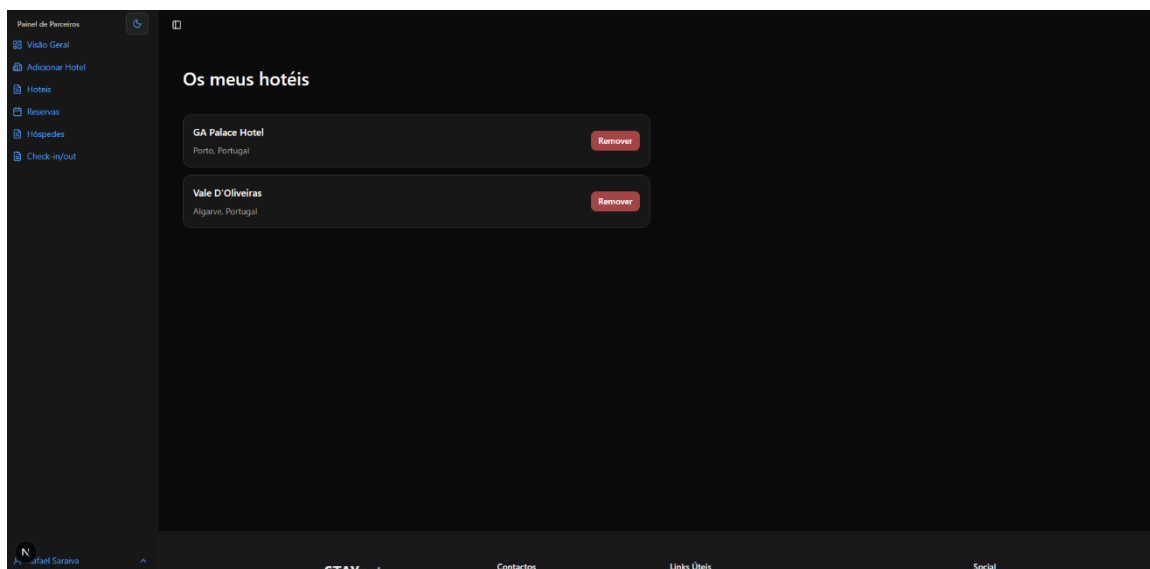


Figura 34 - Os meus hotéis

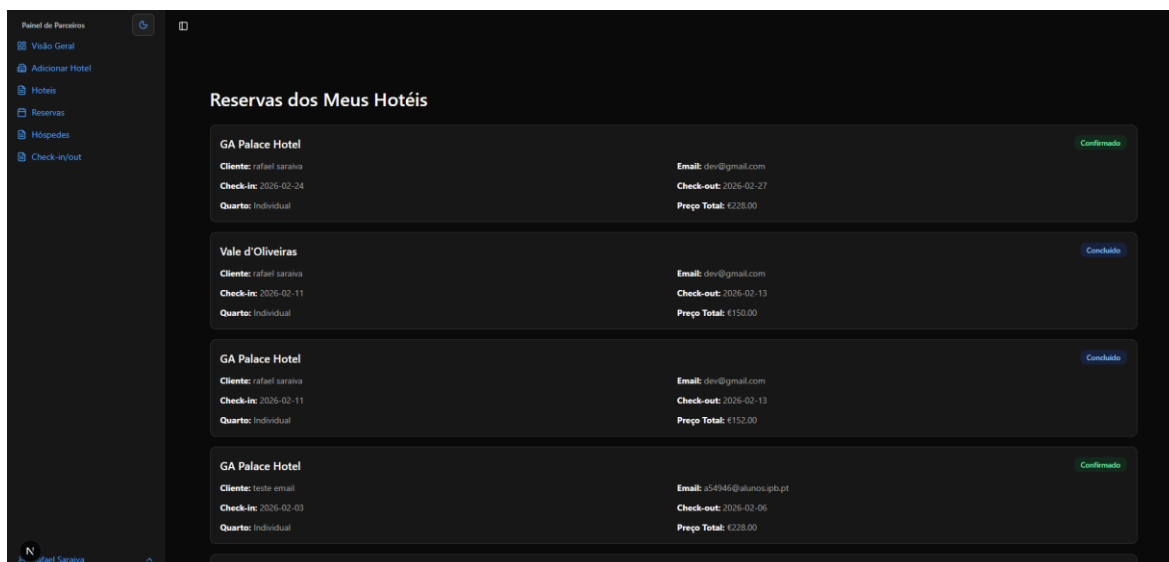


Figura 35 - Reservas dos meus Hoteis

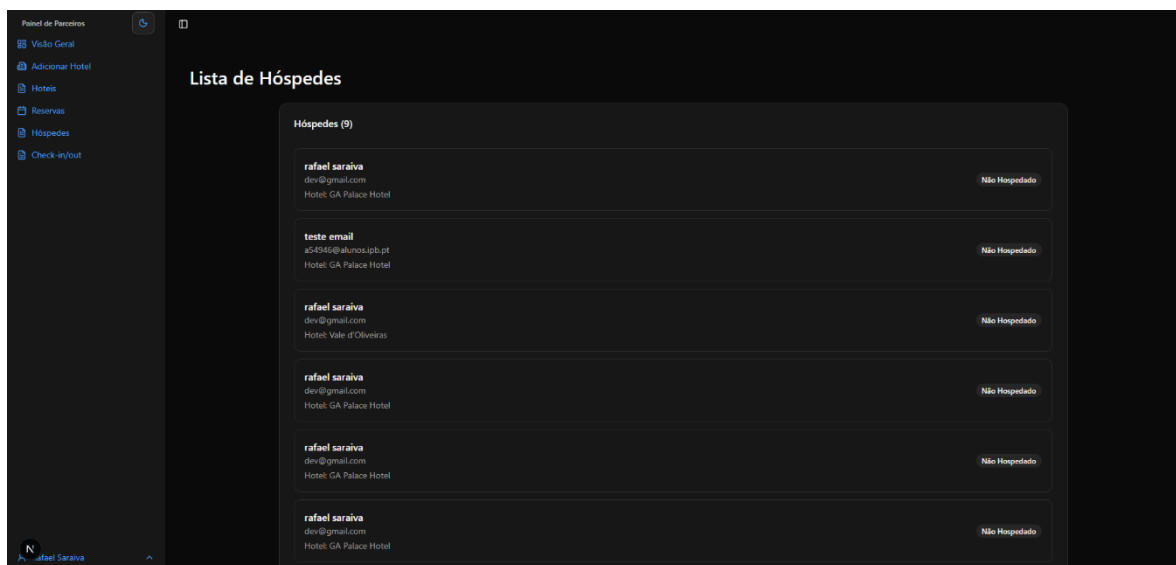


Figura 36 - Lista de Hospedes

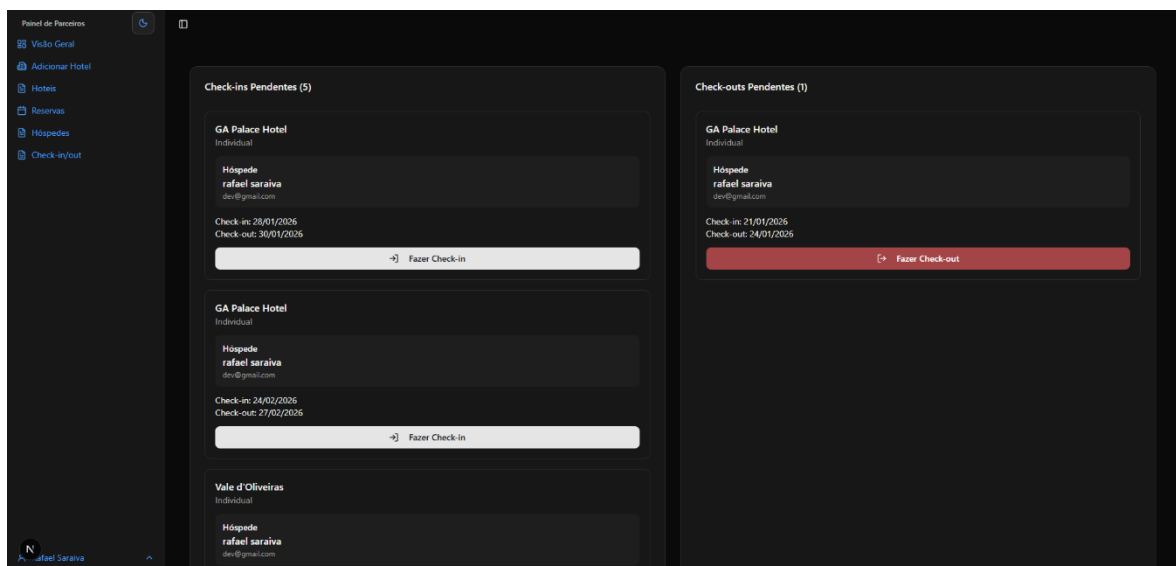


Figura 37 - Check-ins e Check-outs

5.1.6 Área de Administrador

Na Área de Administrador é possível ter uma visão geral das estatísticas da plataforma visualizando o número e dados dos utilizadores, parceiros e hotéis.

Também é possível analisar as reservas ativas da plataforma e com apoio da navbar conseguimos navegar para a gestão dos parceiros e dos hotéis anunciados na plataforma.

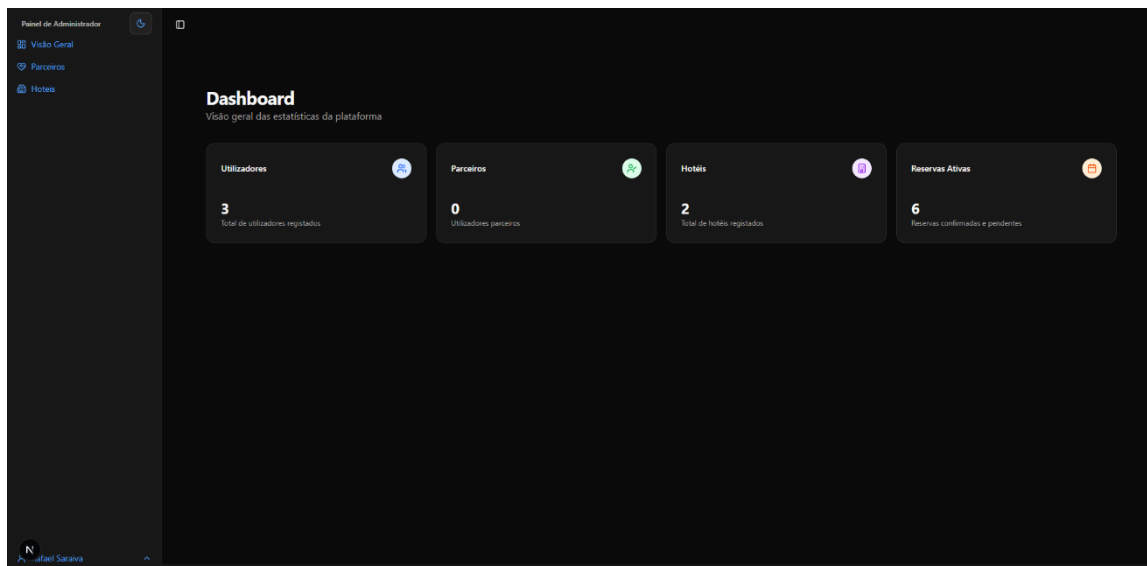


Figura 38 - Painel de Admin

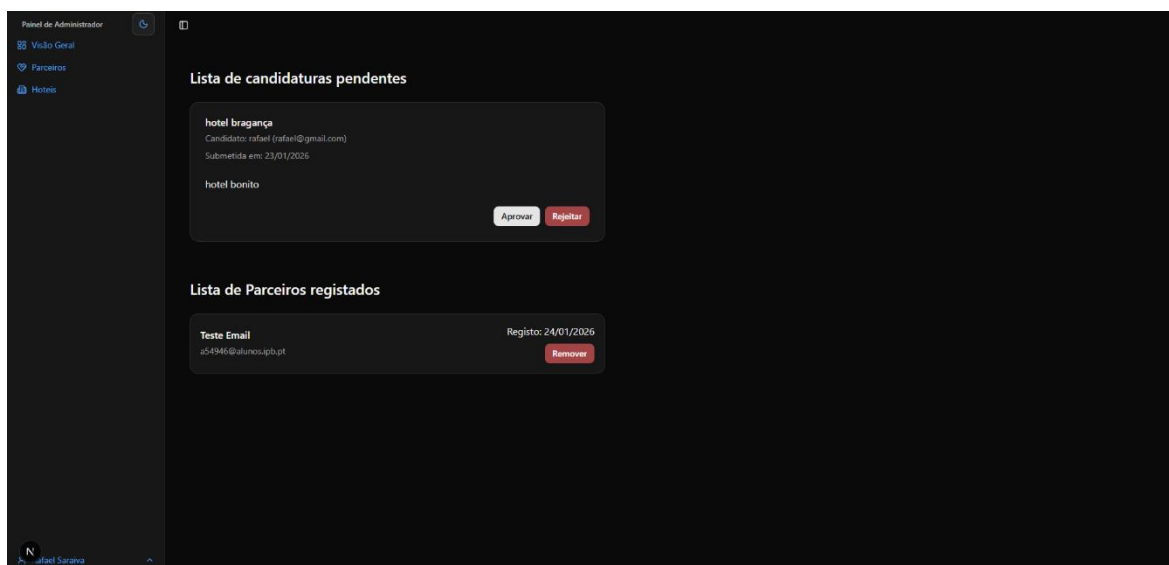


Figura 39 - Gestão de Parceiros

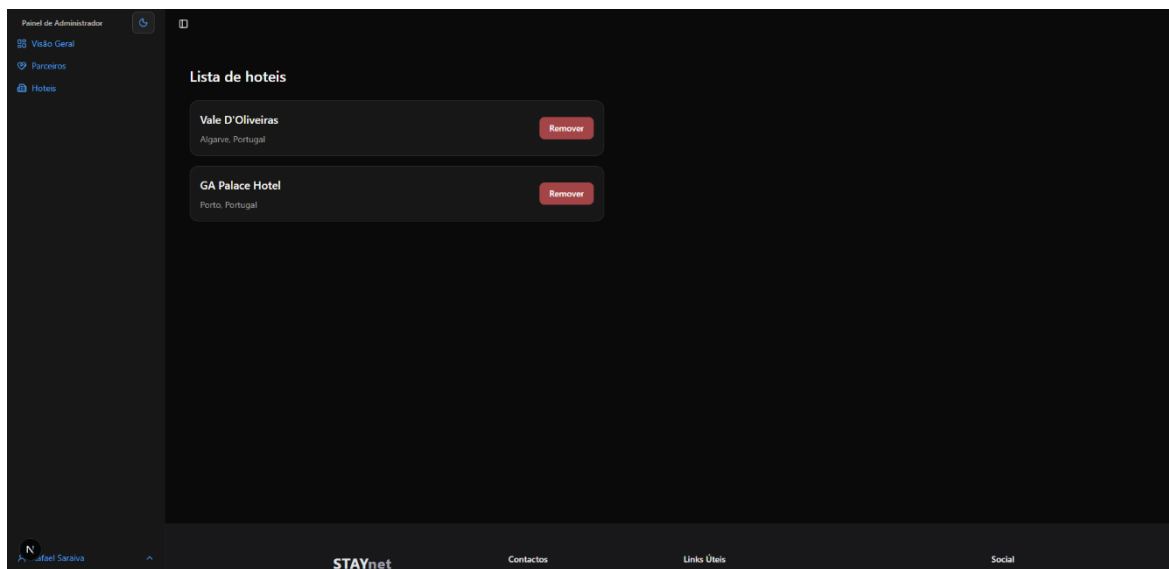


Figura 40 - Lista de Hotéis anunciados

Capítulo 6

Conclusões

Ao longo deste projeto foi desenvolvida a plataforma StayNet, com o objetivo de criar uma solução de reservas e gestão hoteleira que fosse simples de usar para quem procura alojamento e, ao mesmo tempo, útil para quem gere hotéis. A plataforma foi pensada para diferentes perfis (Visitante, Utilizador, Parceiro e Administrador), garantindo que cada um tem acesso apenas às funcionalidades necessárias.

Do ponto de vista funcional, foi possível implementar os principais fluxos previstos: pesquisa e consulta de hotéis/quartos, favoritos, carrinho, criação de reservas e pagamento integrado, bem como uma área dedicada ao parceiro para gerir hotéis, quartos e pedidos, incluindo ações relacionadas com check-in e check-out. A estrutura de dados foi organizada através de um modelo Entidade-Relacionamento, permitindo suportar estes processos de forma consistente e preparada para evolução.

Em termos técnicos, a opção por uma abordagem full-stack com Next.js, ligada a duas bases de dados (Neon), e a (Cloudinary) dedicada a imagens, com o Drizzle, permitiu manter uma base coerente, organizada e relativamente fácil de manter, tanto na parte de interface como na parte de lógica e acesso a dados. A integração com serviços externos, em particular no processo de pagamento, foi um ponto central para aproximar a plataforma de um cenário real de utilização.

Apesar dos resultados alcançados, existiram limitações naturais do tempo disponível em relação à quantidade de trabalho que estava projetado. Algumas melhorias futuras passam por reforçar a validação de disponibilidade em cenários de concorrência (reservas simultâneas), melhorar a experiência de gestão do parceiro com mais métricas e relatórios, acrescentar funcionalidades como cancelamentos com regras mais detalhadas, notificações automáticas, um histórico mais completo de pagamentos e operações e também adicionar mais ferramentas de ajuda e de apoio ao utilizador como um bot de suporte.

No final, considera-se que os objetivos principais foram cumpridos: a StayNet evoluiu de uma ideia para uma plataforma funcional, com um conjunto sólido de funcionalidades e com uma base técnica preparada para futuras melhorias e expansão.

Bibliografia

- [1] “Booking,” [Online]. Available: <https://www.booking.com/>
- [2] “Airbnb,” [Online]. Available: <https://www.airbnb.pt/>
- [3] “Trivago,” [Online]. Available: <https://www.trivago.pt/>
- [4] “Nextjs,” [Online]. Available: <https://nextjs.org/>.
- [5] “nodejs,” [Online]. Available: <https://nodejs.org/en>.
- [6] “neon.com,” [Online]. Available: <https://neon.com/docs/guides/node>.
- [7] “drizzle,” [Online]. Available: <https://orm.drizzle.team/>.
- [8] “Cloudinary,” [Online]. Available: <https://cloudinary.com/>.
- [9] “Docker,” [Online]. Available: <https://www.docker.com/>.
- [10] “TypeScript,” [Online]. Available: <https://www.typescriptlang.org/>.
- [11] “shadcn,” [Online]. Available: <https://ui.shadcn.com/>.
- [12] “stripe,” [Online]. Available: https://stripe.com/en-pt?utm_matchtype=e&utm_adposition=&gad_campaignid=23373237280.
- [13] “openstreetmap,” [Online]. Available: <https://www.openstreetmap.org/#map=7/39.602/-7.839>.
- [14] “Resend,” [Online]. Available: <https://resend.com/>.
- [15] “Redis,” [Online]. Available: <https://redis.io/>.
- [16] “Visual Paradigm,” [Online]. Available: <https://www.visual-paradigm.com/>.
- [17] “Figma,” [Online]. Available: <https://www.figma.com>.
- [18] “ibm,” [Online]. Available: <https://www.ibm.com/br-pt/products/cloud-databases>.
- [19] “Tailwind,” [Online]. Available: <https://tailwindcss.com/>.
- [20] “Auth.js,” [Online]. Available: <https://authjs.dev/>.

Apêndice A.

Proposta Original do Projeto

Proposta Original de Projeto



Curso de Licenciatura em Engenharia Informática

Projeto 3º Ano - Ano letivo de 2025/2026

Plataforma web para gestão de reservas e check-ins em hotel

☐	Inteligência artificial	☐	Multimédia/Realidade aumentada/...
☐	Aplicações móveis	☐	Redes e gestão de sistemas
☒	Plataformas web	☐	*

* - A especificar pelo proponente

Orientador: Prof. Romeu Sequeira

Coorientador: Prof. José Exposto

1 Objetivo

Desenvolver uma aplicação web em Next.js que permita a gestão de hóspedes num hotel, incluindo funcionalidades de check-in, check-out, reservas e envio automático de notificações por email. O objetivo é aplicar os conhecimentos adquiridos no curso, criando uma solução prática e próxima de um caso real de utilização.

2 Detalhes

O projeto será desenvolvido com Next.js no frontend e com rotas server-side do Next.js no backend. A base de dados será PostgreSQL (Neon), utilizando a Drizzle ORM para interação e migrações. A autenticação será implementada com Auth.js.

As principais funcionalidades da aplicação incluem a gestão de utilizadores com autenticação segura, o registo de check-in e check-out, a gestão de reservas, o envio automático de notificações e possivelmente faturas por email e uma interface intuitiva para os funcionários do hotel.

3 Metodologia de trabalho

- Levantamento e análise de requisitos;
- Modelação de dados e desenho da arquitetura da aplicação;
- Revisão do estado da arte e familiarização com as tecnologias escolhidas;
- Desenvolvimento solução, com testes unitários e de integração;
- Validação final e escrita do relatório técnico.

O desenvolvimento do projeto será realizado de forma iterativa e incremental, garantindo a evolução contínua da aplicação. Serão realizadas reuniões periódicas com os orientadores, de modo a acompanhar o progresso e validar as etapas cumpridas. O controlo de versões será assegurado através de um repositório no GitLab, com registo contínuo de commits e documentação associada.

Dimensão da equipa:	2 elementos(Rafael Saraiva (a54946) e Gustavo Soares (a51848).)
Recursos necessários:	Para a concretização do projeto serão necessários: acesso ao serviço Neon (PostgreSQL), um ambiente de desenvolvimento local (Node.js, VSCode), GitLab para alojar o repositório do projeto

Apêndice B.

Conteúdos Adicionais

B.1 Componentes

B.1.1 Componentes de Autenticação

AuthForm.tsx

```
"use client";

import { zodResolver } from "@hookform/resolvers/zod";
import {
  DefaultValues,
  FieldValues,
  Path,
  SubmitHandler,
  useForm,
  UseFormReturn,
} from "react-hook-form";
import React, { useState } from "react";
import { ZodType } from "zod";
import { Button } from "@components/ui/button";
import {
  Form,
  FormControl,
  FormField,
  FormItem,
  FormLabel,
  FormMessage,
} from "@components/ui/form";
import { Input } from "@components/ui/input";
import Link from "next/link";
import { FIELD_NAMES, FIELD_TYPES } from "@constants";
import { toast } from "sonner";
import { useRouter } from "next/navigation";
import Image from "next/image";
import { Mail, CheckCircle } from "lucide-react";

interface Props<T extends FieldValues> {
  schema: ZodType<T>;
  defaultValues: T;
  onSubmit: (data: T) => Promise<
```

```

    success: boolean;
    error?: string;
    pendingVerification?: boolean;
  }>;
  type: "SIGN_UP" | "SIGN_IN" | "RECOVER_PASSWORD";
}

const TITLES = {
  SIGN_IN: "Bem-vindo(a)",
  SIGN_UP: "Criar Conta",
  RECOVER_PASSWORD: "Recuperar Password",
};

const BUTTON_TEXT = {
  SIGN_IN: "Iniciar Sessão",
  SIGN_UP: "Criar Conta",
  RECOVER_PASSWORD: "Enviar Link de Recuperação",
};

const AuthForm = <T extends FieldValues>({
  type,
  schema,
  defaultValues,
  onSubmit,
}: Props<T>) => {
  const [pendingVerification, setPendingVerification] = useState(false);
  const [userEmail, setUserEmail] = useState("");
  const [recoverSuccess, setRecoverSuccess] = useState(false);

  const form: UseFormReturn<T> = useForm({
    resolver: zodResolver(schema),
    defaultValues: defaultValues as DefaultValues<T>,
  });

  const router = useRouter();

  const handleSubmit: SubmitHandler<T> = async (data) => {
    const result = await onSubmit(data);

    if (result.success && result.pendingVerification) {
      setPendingVerification(true);
      setUserEmail((data as any).email);
    } else if (result.success && type === "RECOVER_PASSWORD") {
      setRecoverSuccess(true);
      setUserEmail((data as any).email);
    } else if (result.success) {
      toast.success("Sessão iniciada com sucesso");
      router.push("/discover");
    } else if (result.error) {
      toast.error(result.error);
    }
  };

  if (pendingVerification || recoverSuccess) {
    const title = pendingVerification
      ? "Verifique o seu Email"
      : "Email Enviado!";
    const subtitle = pendingVerification

```

```

? "Enviámos um link de verificação para:"
: "Se existir uma conta associada a:";

return (
  <div className="mx-auto flex max-w-md flex-col gap-6 rounded-2xl border bg-card p-8 text-center shadow-sm">
    <Image
      src="/logo.png"
      alt="logo"
      width={96}
      height={96}
      className="mx-auto"
    />

    <CheckCircle className="mx-auto h-14 w-14 text-primary" />

    <h2 className="text-2xl font-semibold">{title}</h2>

    <div className="rounded-lg border bg-muted p-4">
      <Mail className="mx-auto mb-2 h-6 w-6 text-muted-foreground" />
      <p className="text-sm text-muted-foreground">{subtitle}</p>
      <p className="mt-1 text-sm font-medium">{userEmail}</p>
    </div>

    <p className="text-sm text-muted-foreground">
      O link expira em breve. Verifique também a pasta de spam.
    </p>

    <Button variant="outline" onClick={() => router.push("/sign-in")}>
      Voltar ao Login
    </Button>
  </div>
);
}

return (
  <div className="mx-auto flex max-w-md flex-col gap-6 rounded-2xl border bg-card p-8 shadow-sm">
    <Image
      src="/logo.png"
      alt="logo"
      width={96}
      height={96}
      className="mx-auto"
    />

    <h2 className="text-center text-2xl font-semibold">{TITLES[type]}</h2>

    <Form {...form}>
      <form onSubmit={form.handleSubmit(handleSubmit)} className="space-y-6">
        {Object.keys(defaultValues).map((field) => (
          <FormField
            key={field}
            control={form.control}
            name={field as Path<T>}
            render={({ field }) => (
              <FormItem className="space-y-1">
                <FormLabel className="text-sm font-medium">
                  {FIELD_NAMES[field.name as keyof typeof FIELD_NAMES]}
                </FormLabel>

```

```

    <FormControl>
      <Input
        type={FIELD_TYPES[field.name as keyof typeof FIELD_TYPES]}
        className="h-11 px-4"
        {...field}
      />
    </FormControl>

    <FormMessage />
  </FormItem>
)}
/>
)})

{type === "RECOVER_PASSWORD" && (
  <div className="rounded-lg border bg-muted px-4 py-3 text-sm text-muted-foreground">
    Receberá um email com um link seguro para redefinir a sua
    password.
  </div>
)}

<Button className="h-11 w-full" type="submit">
  {BUTTON_TEXT[type]}
</Button>
</form>
</Form>

{type === "SIGN_IN" && (
  <
    <p className="text-center text-sm">
      Ainda não tens conta? {" "}
      <Link
        className="font-medium text-brand hover:underline"
        href="/sign-up"
      >
        Cria uma aqui
      </Link>
    </p>

    <Link
      className="text-center text-sm text-muted-foreground hover:underline"
      href="/recover-password"
    >
      Esqueceu-se da palavra-passe?
    </Link>
  </>
)}

{type !== "SIGN_IN" && (
  <p className="text-center text-sm">
    Já tens conta? {" "}
    <Link
      className="font-medium text-brand hover:underline"
      href="/sign-in"
    >
      Iniciar sessão
    </Link>
  </p>
)}

```

```

    ))
  </div>
);
};

export default AuthForm;

```

reset-passwordForm.tsx

```

"use client";

import { useState } from "react";
import { useRouter } from "next/navigation";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Label } from "@components/ui/label";
import { resetPassword } from "@lib/actions/password.actions";

interface ResetPasswordFormProps {
  token: string;
}

export function ResetPasswordForm({ token }: ResetPasswordFormProps) {
  const router = useRouter();
  const [newPassword, setNewPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError("");

    if (newPassword !== confirmPassword) {
      setError("As passwords não coincidem");
      return;
    }

    setLoading(true);
    const result = await resetPassword(token, newPassword);
    setLoading(false);

    if (result.success) {
      alert("Password redefinida com sucesso!");
      router.push("/sign-in");
    } else {
      setError(result.error || "Erro ao redefinir password");
    }
  };

  return (
    <div className="p-6 max-w-md w-full">
      <h2 className="text-2xl font-bold mb-6">Nova Password</h2>

      <form onSubmit={handleSubmit} className="space-y-4">
        <div>
          <Label htmlFor="newPassword">Nova Password</Label>
          <Input

```

```

        id="newPassword"
        type="password"
        value={newPassword}
        onChange={(e) => setNewPassword(e.target.value)}
        required
        minLength={6}
        disabled={loading}
      />
    </div>

    <div>
      <Label htmlFor="confirmPassword">Confirmar Password</Label>
      <Input
        id="confirmPassword"
        type="password"
        value={confirmPassword}
        onChange={(e) => setConfirmPassword(e.target.value)}
        required
        minLength={6}
        disabled={loading}
      />
    </div>

    {error && <p className="text-red-600 text-sm">{error}</p>}

    <Button type="submit" className="w-full bg-brand disabled={loading}>
      {loading ? "A redefinir..." : "Redefinir Password"}
    </Button>
  </form>
</div>
);
}

```

SignoutButton.tsx

```

"use client";

import { Button } from "@components/ui/button";
import { DropdownMenuItem } from "@components/ui/dropdown-menu";
import { handleSignOut } from "@lib/actions/auth";
import { useTransition } from "react";
import { LogOut } from "lucide-react";

import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
  AlertDialogTrigger,
} from "@components/ui/alert-dialog";

interface SignOutButtonProps {

```

```

variant?: "default" | "dropdown";
className?: string;
}

export function SignOutButton({
  variant = "default",
  className,
}: SignOutButtonProps) {
  const [isPending, startTransition] = useTransition();

  const handleClick = () => {
    startTransition(async () => {
      await handleSignOut();
    });
  };

  if (variant === "dropdown") {
    return (
      <DropdownMenuItem
        onClick={handleClick}
        disabled={isPending}
        className={`text-red-500 cursor-pointer focus:text-red-600 ${className}`}
      >
        {isPending ? "A sair..." : "Terminar Sessão"}
      </DropdownMenuItem>
    );
  }

  return (
    <Button
      variant="default"
      onClick={handleClick}
      disabled={isPending}
      className={className}
    >
      <LogOut className="text-red-500 cursor-pointer focus:text-red-600" />
      {isPending ? "A sair..." : "Terminar sessão"}
    </Button>
  );
}

```

B.1.2 Componentes da Dashboard de Utilizador

calendarioReservas.tsx

```

"use client";

import * as React from "react";
import { type DateRange } from "react-day-picker";
import { Calendar } from "@/components/ui/calendar";
import { getUnavailableDates } from "@/lib/actions/booking.actions";

interface CalendarioProps {
  dateRange: DateRange | undefined;
  setDateRange: (date: DateRange | undefined) => void;
}

```

```

    roomId: string;
  }

export function Calendario({
  dateRange,
  setDateRange,
  roomId,
}: CalendarioProps) {
  const today = React.useMemo(() => {
    const d = new Date();
    d.setHours(0, 0, 0, 0);
    return d;
  }, []);

  const [currentMonth, setCurrentMonth] = React.useState<Date>(
    dateRange?.from || today,
  );
  const [unavailableDates, setUnavailableDates] = React.useState<string[]>([]);
  const [isLoading, setIsLoading] = React.useState(false);

  React.useEffect(() => {
    const fetchUnavailableDates = async () => {
      if (!roomId) return;

      setIsLoading(true);
      const year = currentMonth.getFullYear();
      const month = currentMonth.getMonth() + 1;

      const firstDay = new Date(year, month - 1, 1);
      const lastDay = new Date(year, month, 0);

      const checkIn = firstDay.toISOString().split("T")[0];
      const checkOut = lastDay.toISOString().split("T")[0];

      const result = await getUnavailableDates(roomId, checkIn, checkOut);

      if (result.success && result.unavailableDates) {
        setUnavailableDates(result.unavailableDates);
      } else {
        setUnavailableDates([]);
      }

      setIsLoading(false);
    };

    fetchUnavailableDates();
  }, [roomId, currentMonth]);

  const isDayUnavailable = React.useCallback(
    (date: Date): boolean => {
      const dateStr = date.toISOString().split("T")[0];
      return unavailableDates.includes(dateStr);
    },
    [unavailableDates],
  );

  const modifiers = React.useMemo(
    () => ({

```



```

    available: (date: Date) => date >= today && !isDayUnavailable(date),
    unavailable: (date: Date) => date >= today && isDayUnavailable(date),
  }},
  [today, isDayUnavailable],
);

const modifiersClassNames = {
  available: "bg-green-100 text-green-900 hover:bg-green-200 font-semibold",
  unavailable:
    "bg-red-100 text-red-900 hover:bg-red-200 line-through opacity-50",
};

return (
  <div className="flex flex-col gap-2">
    {isLoading && (
      <div className="text-center text-sm text-muted-foreground">
        A carregar disponibilidade...
      </div>
    )}
    <div className="flex justify-center">
      <Calendar
        mode="range"
        month={currentMonth}
        onMonthChange={setCurrentMonth}
        selected={dateRange}
        onSelect={setDateRange}
        numberOfMonths={1}
        disabled={(date) => date < today || isDayUnavailable(date)}
        modifiers={modifiers}
        modifiersClassNames={modifiersClassNames}
        className="rounded-lg border shadow-sm"
      />
    </div>
  </div>
);
}

```

hotelCard.tsx

```

import Link from "next/link";
import { Card } from "@components/ui/card";
import { MapPin } from "lucide-react";
import Image from "next/image";

export function HotelCard({ hotel }: { hotel: HotelCardProps }) {
  return (
    <Link href={`/${discover}/${hotel.id}`} className="group">
      <Card className="relative overflow-hidden border-border bg-card hover:border-primary/50 transition-all duration-300 h-80">
        <div className="absolute inset-0">
          {hotel.images.length > 0 ? (
            <Image
              src={hotel.images[0].url}
              alt={`Imagem do ${hotel.name}`}
              fill
              className="object-cover group-hover:scale-110 transition-transform duration-500"
              sizes="(max-width: 640px) 100vw, (max-width: 1024px) 50vw, 33vw"
            />
          ) : (
            <div className="w-full h-full bg-muted flex items-center justify-center">

```

```

        <span className="text-muted-foreground">Sem imagem</span>
      </div>
    )}
  </div>

  <div className="absolute inset-0 bg-gradient-to-t from-black/80 via-black/40 to-transparent group-
hover:from-black/90 transition-all duration-300" />

  <div className="absolute inset-0 p-6 flex flex-col justify-end text-white">
    <h3 className="capitalize text-xl font-bold mb-2 line-clamp-2 group-hover:text-[var(--brand2)]
transition-colors">
      {hotel.name}
    </h3>

    <div className="flex items-center gap-2 text-sm text-white/90">
      <MapPin className="h-4 w-4 flex-shrink-0" />
      <span className="capitalize line-clamp-1">
        {hotel.city}, {hotel.country}
      </span>
    </div>

    {hotel.description && (
      <p className="text-sm text-white/80 line-clamp-2 mt-3 opacity-0 group-hover:opacity-100
transition-opacity duration-300">
        {hotel.description}
      </p>
    )}
  </div>
</Card>
</Link>
);
}

```

ImageSlider.tsx

```

import * as React from "react";
import Image from "next/image";

import {
  Carousel,
  CarouselContent,
  CarouselItem,
  CarouselNext,
  CarouselPrevious,
} from "@components/ui/carousel";

export function ImageSlider({ images, hotelName = "Hotel" }: ImageSliderProps) {
  if (!images || images.length === 0) {
    return (
      <div className="w-full max-w-4xl aspect-[16/9] flex items-center justify-center bg-gray-100 rounded-
lg">
        <span className="text-gray-400">Sem imagens disponíveis</span>
      </div>
    );
  }

  return (
    <Carousel className="w-full max-w-4xl mx-auto">

```

```

<CarouselContent>
  {images.map((image, index) => (
    <CarouselItem key={image.id}>
      <div className="relative aspect-[16/9] w-full overflow-hidden rounded-lg">
        <Image
          src={image.url}
          alt={`$ {hotelName} - Imagem $ {index + 1} `}
          fill
          className="object-cover"
          priority={index === 0}
          sizes="(max-width: 768px) 100vw, (max-width: 1200px) 80vw, 1200px"
        />
      </div>
    </CarouselItem>
  ))}
</CarouselContent>
<CarouselPrevious />
<CarouselNext />
</Carousel>
);
}

```

reservasCard.tsx

```

"use client";

import * as React from "react";
import { type DateRange } from "react-day-picker";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@components/ui/tabs";
import {
  Card,
  CardContent,
  CardDescription,
  CardFooter,
  CardHeader,
  CardTitle,
} from "@components/ui/card";
import { Button } from "@components/ui/button";
import { BedDouble, BedSingle, Loader2, ShoppingCart } from "lucide-react";
import { Calendario } from "../calendarioReservas";
import {
  calculateBookingPrice,
  checkRoomAvailability,
} from "@lib/actions/booking.actions";
import { useRouter } from "next/navigation";
import { addToCart } from "@lib/cart";
import { toast } from "sonner";

export function ReservasCard({ hotel }: ReservasCardProps) {
  const [dateRange, setDateRange] = React.useState<DateRange | undefined>();
  const [selectedTab, setSelectedTab] = React.useState<"single" | "double">(
    hotel.singleRoom ? "single" : "double",
  );
  const [isBooking, setIsBooking] = React.useState(false);
  const [priceInfo, setPriceInfo] = React.useState<{
    nights: number;
    total: string;
  } | null>(null);
  const router = useRouter();

```

```

React.useEffect(() => {
  const fetchPrice = async () => {
    if (!dateRange?.from || !dateRange?.to) {
      setPriceInfo(null);
      return;
    }

    const currentRoom =
      selectedTab === "single" ? hotel.singleRoom : hotel.doubleRoom;
    if (!currentRoom?.id) return;

    const checkIn = dateRange.from.toISOString().split("T")[0];
    const checkOut = dateRange.to.toISOString().split("T")[0];

    const result = await calculateBookingPrice(
      currentRoom.id,
      checkIn,
      checkOut,
    );

    if (
      result.success &&
      result.nights !== undefined &&
      result.total !== undefined
    ) {
      setPriceInfo({
        nights: result.nights,
        total: result.total,
      });
    } else {
      setPriceInfo(null);
    }
  };

  fetchPrice();
}, [dateRange, selectedTab, hotel.singleRoom?.id, hotel.doubleRoom?.id]);

const handleAddToCart = async () => {
  if (!dateRange?.from || !dateRange?.to) {
    toast.error("Por favor, selecione as datas");
    return;
  }

  const currentRoom =
    selectedTab === "single" ? hotel.singleRoom : hotel.doubleRoom;

  if (!currentRoom?.id || !priceInfo) {
    toast.error("Quarto não encontrado");
    return;
  }

  setIsBooking(true);

  try {
    const checkIn = dateRange.from.toISOString().split("T")[0];
    const checkOut = dateRange.to.toISOString().split("T")[0];

    const availability = await checkRoomAvailability(

```

```

        currentRoom.id,
        checkIn,
        checkOut,
    );

    if (!availability.available) {
        toast.error("Este quarto está indisponível para as datas selecionadas");
        setIsBooking(false);
        return;
    }

    addToCart({
        roomId: currentRoom.id,
        checkInDate: checkIn,
        checkOutDate: checkOut,
        totalPrice: priceInfo.total,
        nights: priceInfo.nights,
        roomType: currentRoom.roomType,
        hotelName: hotel.name,
        hotelId: hotel.id,
    });

    toast.success(`${hotel.name} foi adicionado ao carrinho!`, {
        description: `${priceInfo.nights} noite(s) • €${priceInfo.total}`,
    });

    setDateRange(undefined);
    setPriceInfo(null);
} catch (error) {
    console.error(error);
    toast.error("Erro ao adicionar ao carrinho");
} finally {
    setIsBooking(false);
}
};

return (
    <Tabs
        value={selectedTab}
        onValueChange={(value) => {
            setSelectedTab(value as "single" | "double");
            setDateRange(undefined);
            setPriceInfo(null);
        }}
        className="w-full"
    >
        <TabsList className="grid w-full grid-cols-2">
            {hotel.singleRoom && (
                <TabsTrigger value="single">
                    <BedSingle className="mr-2 h-4 w-4" />
                    Individual
                </TabsTrigger>
            )}
            {hotel.doubleRoom && (
                <TabsTrigger value="double">
                    <BedDouble className="mr-2 h-4 w-4" />
                    Duplo
                </TabsTrigger>
            )}
        </TabsList>

```

```

[["single", "double"].map((tab) => {
  const currentRoom =
    tab === "single" ? hotel.singleRoom : hotel.doubleRoom;
  if (!currentRoom) return null;

  return (
    <TabsContent key={tab} value={tab}>
      <Card>
        <CardHeader>
          <CardTitle>
            {tab === "single" ? "Quarto Individual" : "Quarto Duplo"}
          </CardTitle>
          <CardDescription>
            <span className="text-2xl font-bold text-foreground">
              €{currentRoom.pricePerNight}
            </span>
            <span className="text-sm"> por noite</span>
          </CardDescription>
        </CardHeader>

        <CardContent className="space-y-4">
          <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
            <div className="space-y-4">
              <div>
                <h4 className="font-semibold mb-2">
                  Descrição do quarto
                </h4>
                <p className="text-sm text-muted-foreground">
                  {currentRoom.description || "Sem descrição disponível."}
                </p>
              </div>

              {priceInfo && (
                <div className="bg-muted p-4 rounded-lg">
                  <p className="text-sm font-medium">
                    {priceInfo.nights} noite
                    {priceInfo.nights > 1 ? "s" : ""}
                  </p>
                  <p className="text-lg font-bold text-primary">
                    Total: €{priceInfo.total}
                  </p>
                </div>
              )}
            </div>

            <div className="flex items-start justify-center lg:justify-end">
              <Calendario
                roomId={currentRoom.id}
                dateRange={dateRange}
                setDateRange={setDateRange}
              />
            </div>
          </div>
        </CardContent>

        <CardFooter>
          <Button

```

```

        onClick={handleAddToCart}
        disabled={!dateRange?.from || !dateRange?.to || isBooking}
        className="w-full"
        size="lg"
      >
        {isBooking ? (
          <
            <Loader2 className="mr-2 h-4 w-4 animate-spin" />A
            processar...
          </>
        ) : (
          <
            <ShoppingCart className="mr-2 h-4 w-4" />
            Adicionar ao Carrinho
          </>
        )}
      </Button>
    </CardFooter>
  </Card>
</TabsContent>
);
}}}
</Tabs>
);
}

```

myBookings.tsx

```

import { Button } from "@components/ui/button";
import {
  Card,
  CardContent,
  CardDescription,
  CardFooter,
  CardHeader,
  CardTitle,
} from "@components/ui/card";
import { Badge } from "@components/ui/badge";
import { Calendar, Package, MapPin, BedDouble, Clock } from "lucide-react";
import Link from "next/link";

type Props = {
  bookings: any[];
};

export default function MyBookingsList({ bookings }: Props) {
  const formatDate = (date: string) =>
    new Date(date).toLocaleDateString("pt-PT", {
      day: "numeric",
      month: "long",
      year: "numeric",
    });

  const calculateNights = (checkIn: string, checkOut: string) =>
    Math.ceil(
      (new Date(checkOut).getTime() - new Date(checkIn).getTime()) /
      (1000 * 60 * 60 * 24),
    );
}

```

```

const getStatusBadge = (status: string) => {
  const map: Record<string, { label: string; variant: any }> = {
    Confirmed: { label: "Confirmada", variant: "default" },
    Pending: { label: "Pendente", variant: "secondary" },
    Cancelled: { label: "Cancelada", variant: "destructive" },
  };

  const cfg = map[status] ?? {
    label: status,
    variant: "secondary",
  };

  return <Badge variant={cfg.variant}>{cfg.label}</Badge>;
};

if (bookings.length === 0) {
  return (
    <div className="container max-w-4xl py-12">
      <div className="mb-8">
        <h1 className="text-3xl font-bold">As Minhas Reservas</h1>
        <p className="text-muted-foreground mt-2">
          Gerir as suas reservas de hotel
        </p>
      </div>

      <div className="text-center py-16 border-2 border-dashed rounded-lg">
        <Package className="h-12 w-12 mx-auto text-muted-foreground mb-4" />
        <h2 className="text-xl font-semibold mb-2">Nenhuma Reserva</h2>
        <p className="text-muted-foreground mb-6">
          Ainda não tem reservas. Explore os nossos hotéis!
        </p>

        <Button asChild>
          <Link href="/discover">
            <Calendar className="mr-2 h-4 w-4" />
            Explorar Hotéis
          </Link>
        </Button>
      </div>
    </div>
  );
}

const upcoming = bookings.filter(
  ({ booking }) => new Date(booking.checkInDate) >= new Date(),
);

const past = bookings.filter(
  ({ booking }) => new Date(booking.checkInDate) < new Date(),
);

return (
  <div className="container max-w-4xl py-12">
    <div className="flex items-center justify-between mb-8">
      <div>
        <h1 className="text-3xl font-bold">As Minhas Reservas</h1>
        <p className="text-muted-foreground mt-1">

```



```

    {bookings.length} {bookings.length === 1 ? "reserva" : "reservas"}
  </p>
</div>

<Button asChild variant="outline">
  <Link href="/discover">Nova Reserva</Link>
</Button>
</div>

{upcoming.length > 0 && <Section title="Próximas" bookings={upcoming} />}

{past.length > 0 && <Section title="Histórico" bookings={past} muted />}
</div>
);

function Section({
  title,
  bookings,
  muted,
}): {
  title: string;
  bookings: any[];
  muted?: boolean;
}) {
  return (
    <div className={muted ? "opacity-60" : "mb-12"}>
      <h2 className="text-xl font-semibold mb-4">
        {title} ({bookings.length})
      </h2>

      <div className="space-y-4">
        {bookings.map(({ booking, room, hotel }) => (
          <Card key={booking.id}>
            <CardHeader>
              <div className="flex items-start justify-between">
                <div>
                  <CardTitle className="text-2xl">{hotel.name}</CardTitle>
                  <CardDescription className="flex items-center gap-2 mt-2">
                    <MapPin className="h-4 w-4" />
                    {hotel.city}, {hotel.country}
                  </CardDescription>
                </div>
                {getStatusBadge(booking.status)}
              </div>
            </CardHeader>

            <CardContent className="space-y-4">
              <div className="grid md:grid-cols-2 gap-4 text-sm">
                <div className="space-y-2">
                  <Info icon={Calendar} label="Check-in">
                    {formatDate(booking.checkInDate)}
                  </Info>
                  <Info icon={Calendar} label="Check-out">
                    {formatDate(booking.checkOutDate)}
                  </Info>
                  <Info icon={Clock} label="Duração">
                    {calculateNights(
                      booking.checkInDate,
                      booking.checkOutDate,
                    )} {" "}
                </div>
              </div>
            </CardContent>
          </Card>
        ))}
      </div>
    </div>
  );
}

```

```

        noites
      </Info>
    </div>

    <div className="space-y-2">
      <Info icon={BedDouble} label="Quarto">
        {room.roomType === "Single" ? "Individual" : "Duplo"}
      </Info>

      <p>
        <strong>Preço por noite:</strong> € {room.pricePerNight}
      </p>

      <p className="text-lg font-bold text-primary">
        € {booking.totalPrice}
      </p>
    </div>
  </div>

  {booking.specialRequests && (
    <div className="pt-2 border-t">
      <p className="font-medium mb-1">Pedidos Especiais:</p>
      <p className="text-muted-foreground">
        {booking.specialRequests}
      </p>
    </div>
  )}
</CardContent>

<CardFooter className="text-xs text-muted-foreground">
  Reserva #{booking.id.slice(0, 8)}
</CardFooter>
</Card>
  )))
</div>
</div>
);
}

function Info({
  icon: Icon,
  label,
  children,
}: {
  icon: any;
  label: string;
  children: React.ReactNode;
}) {
  return (
    <div className="flex items-center gap-2">
      <Icon className="h-4 w-4 text-muted-foreground" />
      <span className="font-medium">{label}</span>
      <span>{children}</span>
    </div>
  );
}
}
}

```

createPartnerApplication.tsx

```
import { Button } from "@components/ui/button";
import {
  Card,
  CardContent,
  CardDescription,
  CardFooter,
  CardHeader,
  CardTitle,
} from "@components/ui/card";
import { Badge } from "@components/ui/badge";
import { Calendar, Package, MapPin, BedDouble, Clock } from "lucide-react";
import Link from "next/link";

type Props = {
  bookings: any[];
};

export default function MyBookingsList({ bookings }: Props) {
  const formatDate = (date: string) =>
    new Date(date).toLocaleDateString("pt-PT", {
      day: "numeric",
      month: "long",
      year: "numeric",
    });

  const calculateNights = (checkIn: string, checkOut: string) =>
    Math.ceil(
      (new Date(checkOut).getTime() - new Date(checkIn).getTime()) /
      (1000 * 60 * 60 * 24),
    );

  const getStatusBadge = (status: string) => {
    const map: Record<string, { label: string; variant: any }> = {
      Confirmed: { label: "Confirmada", variant: "default" },
      Pending: { label: "Pendente", variant: "secondary" },
      Cancelled: { label: "Cancelada", variant: "destructive" },
    };
    const cfg = map[status] ?? {
      label: status,
      variant: "secondary",
    };
    return <Badge variant={cfg.variant}>{cfg.label}</Badge>;
  };

  if (bookings.length === 0) {
    return (
      <div className="container max-w-4xl py-12">
        <div className="mb-8">
          <h1 className="text-3xl font-bold">As Minhas Reservas</h1>
          <p className="text-muted-foreground mt-2">
            Gerir as suas reservas de hotel
          </p>
        </div>

        <div className="text-center py-16 border-2 border-dashed rounded-lg">
          <Package className="h-12 w-12 mx-auto text-muted-foreground mb-4" />
          <h2 className="text-xl font-semibold mb-2">Nenhuma Reserva</h2>
        </div>
      </div>
    );
  }
}
```

```

    <p className="text-muted-foreground mb-6">
      Ainda não tem reservas. Explore os nossos hotéis!
    </p>

    <Button asChild>
      <Link href="/discover">
        <Calendar className="mr-2 h-4 w-4" />
        Explorar Hotéis
      </Link>
    </Button>
  </div>
</div>
);
}

const upcoming = bookings.filter(
  ({ booking }) => new Date(booking.checkInDate) >= new Date(),
);

const past = bookings.filter(
  ({ booking }) => new Date(booking.checkInDate) < new Date(),
);

return (
  <div className="container max-w-4xl py-12">
    <div className="flex items-center justify-between mb-8">
      <div>
        <h1 className="text-3xl font-bold">As Minhas Reservas</h1>
        <p className="text-muted-foreground mt-1">
          {bookings.length} {bookings.length === 1 ? "reserva" : "reservas"}
        </p>
      </div>

      <Button asChild variant="outline">
        <Link href="/discover">Nova Reserva</Link>
      </Button>
    </div>

    {upcoming.length > 0 && <Section title="Próximas" bookings={upcoming} />}

    {past.length > 0 && <Section title="Histórico" bookings={past} muted />}
  </div>
);

function Section({
  title,
  bookings,
  muted,
}): {
  title: string;
  bookings: any[];
  muted?: boolean;
}) {
  return (
    <div className={muted ? "opacity-60" : "mb-12"}>
      <h2 className="text-xl font-semibold mb-4">
        {title} ({bookings.length})
      </h2>

```

```

<div className="space-y-4">
  {bookings.map(({ booking, room, hotel }) => (
    <Card key={booking.id}>
      <CardHeader>
        <div className="flex items-start justify-between">
          <div>
            <CardTitle className="text-2xl">{hotel.name}</CardTitle>
            <CardDescription className="flex items-center gap-2 mt-2">
              <MapPin className="h-4 w-4" />
              {hotel.city}, {hotel.country}
            </CardDescription>
          </div>
          {getStatusBadge(booking.status)}
        </div>
      </CardHeader>

      <CardContent className="space-y-4">
        <div className="grid md:grid-cols-2 gap-4 text-sm">
          <div className="space-y-2">
            <Info icon={Calendar} label="Check-in">
              {formatDate(booking.checkInDate)}
            </Info>
            <Info icon={Calendar} label="Check-out">
              {formatDate(booking.checkOutDate)}
            </Info>
            <Info icon={Clock} label="Duração">
              {calculateNights(
                booking.checkInDate,
                booking.checkOutDate,
              )} {" "}
              noites
            </Info>
          </div>

          <div className="space-y-2">
            <Info icon={BedDouble} label="Quarto">
              {room.roomType === "Single" ? "Individual" : "Duplo"}
            </Info>

            <p>
              <strong>Preço por noite:</strong> €{room.pricePerNight}
            </p>

            <p className="text-lg font-bold text-primary">
              €{booking.totalPrice}
            </p>
          </div>

          {booking.specialRequests && (
            <div className="pt-2 border-t">
              <p className="font-medium mb-1">Pedidos Especiais:</p>
              <p className="text-muted-foreground">
                {booking.specialRequests}
              </p>
            </div>
          )}
        </CardContent>
      </Card>
    )}
  )}

```

```

        <CardFooter className="text-xs text-muted-foreground">
          Reserva #{booking.id.slice(0, 8)}
        </CardFooter>
      </Card>
    )))
  </div>
</div>
);
}

function Info({
  icon: Icon,
  label,
  children,
}): {
  icon: any;
  label: string;
  children: React.ReactNode;
}) {
  return (
    <div className="flex items-center gap-2">
      <Icon className="h-4 w-4 text-muted-foreground" />
      <span className="font-medium">{label}</span>
      <span>{children}</span>
    </div>
  );
}
}

```

cartComponent.tsx

```

"use client";

import * as React from "react";
import { useRouter } from "next/navigation";
import {
  Card,
  CardContent,
  CardDescription,
  CardFooter,
  CardHeader,
  CardTitle,
} from "@components/ui/card";
import { Button } from "@components/ui/button";
import { Separator } from "@components/ui/separator";
import {
  Loader2,
  CreditCard,
  Calendar,
  BedDouble,
  ShoppingCart,
  Trash2,
  ArrowLeft,
  ShoppingBag,
} from "lucide-react";

```

```

import { removeFromCart, clearCart, getCartTotal } from "@lib/cart";
import { createCheckoutSessionMultiple } from "@lib/actions/checkout.actions";
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
  AlertDialogTrigger,
} from "@components/ui/alert-dialog";

export interface CartComponentProps {
  cartItems: CartItem[];
  onRefresh: () => void;
}

export function CartComponent({ cartItems, onRefresh }: CartComponentProps) {
  const [isProcessing, setIsProcessing] = React.useState(false);
  const [error, setError] = React.useState<string | null>(null);
  const router = useRouter();

  const handleRemoveItem = (itemId: string) => {
    removeFromCart(itemId);
    onRefresh();
  };

  const handleClearCart = () => {
    clearCart();
    onRefresh();
  };

  const handleCheckout = async () => {
    setIsProcessing(true);
    setError(null);

    try {
      const result = await createCheckoutSessionMultiple(cartItems);

      if (!result.success) {
        throw new Error(result.error || "Erro ao criar sessão de pagamento");
      }

      if (!result.url) {
        throw new Error("URL de pagamento não retornada");
      }

      window.location.href = result.url;
    } catch (err) {
      console.error("Erro no checkout:", err);
      setError(
        err instanceof Error ? err.message : "Erro ao processar pagamento",
      );
      setIsProcessing(false);
    }
  };

  if (cartItems.length === 0) {

```

```

return (
  <div className="max-w-2xl mx-auto p-6">
    <Card>
      <CardHeader className="text-center">
        <ShoppingBag className="w-16 h-16 mx-auto mb-4 text-muted-foreground" />
        <CardTitle>Carrinho Vazio</CardTitle>
        <CardDescription>
          Ainda não adicionou nenhuma reserva ao carrinho
        </CardDescription>
      </CardHeader>
      <CardFooter className="justify-center">
        <Button onClick={() => router.push("/discover")}>
          <ArrowLeft className="mr-2 h-4 w-4" />
          Explorar Hotéis
        </Button>
      </CardFooter>
    </Card>
  </div>
);
}

const total = getCartTotal();

return (
  <div className="max-w-4xl mx-auto p-6 space-y-6">
    <div className="flex items-center justify-between">
      <div className="flex items-center gap-3">
        <ShoppingCart className="w-8 h-8" />
        <div>
          <h1 className="text-3xl font-bold">Carrinho de Reservas</h1>
          <p className="text-muted-foreground">
            {cartItems.length} {cartItems.length === 1 ? "item" : "itens"} no carrinho
          </p>
        </div>
      </div>
    </div>

    <AlertDialog>
      <AlertDialogTrigger asChild>
        <Button variant="outline" size="sm" className="text-destructive">
          <Trash2 className="w-4 h-4 mr-2" />
          Limpar Tudo
        </Button>
      </AlertDialogTrigger>
      <AlertDialogContent>
        <AlertDialogHeader>
          <AlertDialogTitle>Limpar Carrinho?</AlertDialogTitle>
          <AlertDialogDescription>
            Tem a certeza que deseja remover todas as reservas do carrinho?
          </AlertDialogDescription>
        </AlertDialogHeader>
        <AlertDialogFooter>
          <AlertDialogCancel>Cancelar</AlertDialogCancel>
          <AlertDialogAction
            onClick={handleClearCart}
            className="bg-destructive text-destructive-foreground hover:bg-destructive/90"
          >
            Sim, Limpar

```



```

        </AlertDialogAction>
        </AlertDialogFooter>
    </AlertDialogContent>
</AlertDialog>
</div>

<div className="space-y-4">
    {cartItems.map((item) => (
        <Card key={item.id}>
            <CardHeader>
                <div className="flex items-start justify-between">
                    <div className="space-y-1">
                        <CardTitle className="text-xl">{item.hotelName}</CardTitle>
                        <CardDescription className="flex items-center gap-2">
                            <BedDouble className="w-4 h-4" />
                            {item.roomType === "Single"
                                ? "Quarto Individual"
                                : "Quarto Duplo"}
                        </CardDescription>
                    </div>
                </div>
            <AlertDialog>
                <AlertDialogTrigger asChild>
                    <Button
                        variant="ghost"
                        size="icon"
                        className="text-destructive"
                    >
                        <Trash2 className="w-4 h-4" />
                    </Button>
                </AlertDialogTrigger>
                <AlertDialogContent>
                    <AlertDialogHeader>
                        <AlertDialogTitle>Remover item?</AlertDialogTitle>
                        <AlertDialogDescription>
                            Deseja remover {item.hotelName} do carrinho?
                        </AlertDialogDescription>
                    </AlertDialogHeader>
                    <AlertDialogFooter>
                        <AlertDialogCancel>Cancelar</AlertDialogCancel>
                        <AlertDialogAction
                            onClick={() => handleRemoveItem(item.id)}
                        >
                            Remover
                        </AlertDialogAction>
                    </AlertDialogFooter>
                </AlertDialogContent>
            </AlertDialog>
        </div>
    </CardHeader>

    <CardContent>
        <div className="grid gap-3 text-sm">
            <div className="flex items-center gap-2">
                <Calendar className="w-4 h-4 text-muted-foreground" />
                <span>
                    <strong>Check-in:</strong>{" "}
                    {new Date(item.checkInDate).toLocaleDateString("pt-PT")}
                </span>
            </div>
        </div>
    </CardContent>
    <div className="flex items-center gap-2">

```

```

        <Calendar className="w-4 h-4 text-muted-foreground" />
        <span>
          <strong>Check-out:</strong>{" "}
          {new Date(item.checkOutDate).toLocaleDateString("pt-PT")}
        </span>
      </div>
      <Separator className="my-2" />
      <div className="flex items-center justify-between">
        <span className="text-muted-foreground font-medium">
          {item.nights} noite{item.nights > 1 ? "s" : ""}
        </span>
        <span className="text-xl font-bold text-primary">
          €{item.totalPrice}
        </span>
      </div>
    </div>
  </CardContent>
</Card>
  )}
</div>

{error && (
  <Card className="border-destructive">
    <CardContent className="pt-6">
      <div className="flex items-start gap-2 text-destructive">
        <div className="flex-1">
          <p className="text-sm font-medium">
            Erro ao processar pagamento
          </p>
          <p className="text-sm mt-1">{error}</p>
        </div>
      </div>
    </CardContent>
  </Card>
)}

<Card>
  <CardHeader>
    <CardTitle>Resumo do Pedido</CardTitle>
  </CardHeader>
  <CardContent className="space-y-2">
    {cartItems.map((item) => (
      <div key={item.id} className="flex justify-between text-sm">
        <span className="text-muted-foreground">{item.hotelName}</span>
        <span>€{item.totalPrice}</span>
      </div>
    ))}
    <Separator className="my-2" />
    <div className="flex justify-between font-bold text-lg">
      <span>Total</span>
      <span className="text-primary">€{total.toFixed(2)}</span>
    </div>
  </CardContent>
  <CardFooter className="flex gap-3">
    <Button
      variant="outline"
      onClick={() => router.push("/discover")}
      disabled={isProcessing}
    >

```

```

        className="flex-1"
      >
        <ArrowLeft className="mr-2 h-4 w-4" />
        Continuar a Procurar
      </Button>
      <Button
        onClick={handleCheckout}
        disabled={isProcessing}
        className="flex-1"
        size="lg"
      >
        {isProcessing ? (
          <>
            <Loader2 className="mr-2 h-4 w-4 animate-spin" />A processar...
          </>
        ) : (
          <>
            <CreditCard className="mr-2 h-4 w-4" />
            Finalizar · €{total.toFixed(2)}
          </>
        )}
      </Button>
    </CardFooter>
  </Card>

  <div className="text-center space-y-2">
    <p className="text-xs text-muted-foreground">
      Pagamento 100% seguro via Stripe
    </p>
    <p className="text-xs text-muted-foreground">
      Confirmação imediata por email após o pagamento
    </p>
  </div>
</div>
);
}

```

B.1.3 Componentes da Dashboard de Parceiro

checkinManagement.tsx

```

"use client";

import React, { useState, useEffect } from "react";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Button } from "@components/ui/button";
import { Loader2, Login } from "lucide-react";
import {
  getPartnerPendingCheckIns,
  checkInBooking,
} from "@lib/actions/controlpanel.hotels.actions";
import { toast } from "sonner";

```

```

interface Booking {
  bookingId: string;
  checkInDate: string;
  checkOutDate: string;
  roomType: string;
  hotelName: string;
  guestName: string;
  guestEmail: string;
}

export function CheckInManagement() {
  const [bookings, setBookings] = useState<Booking[]>([]);
  const [loading, setLoading] = useState(true);
  const [processingId, setProcessingId] = useState<string | null>(null);

  const loadBookings = async () => {
    setLoading(true);
    const result = await getPartnerPendingCheckIns();
    if (result.success && result.bookings) {
      setBookings(result.bookings);
    }
    setLoading(false);
  };

  useEffect(() => {
    loadBookings();
  }, []);

  const handleCheckIn = async (bookingId: string) => {
    setProcessingId(bookingId);
    const result = await checkInBooking(bookingId);
    if (result.success) {
      toast.success("Check-in realizado com sucesso!");
      loadBookings();
    } else {
      toast.error(result.error || "Erro ao fazer check-in");
    }
    setProcessingId(null);
  };

  if (loading) {
    return (
      <Card>
        <CardHeader>
          <CardTitle>Check-ins Pendentes</CardTitle>
        </CardHeader>
        <CardContent>
          <div className="flex justify-center py-8">
            <Loader2 className="h-8 w-8 animate-spin" />
          </div>
        </CardContent>
      </Card>
    );
  }

  return (
    <Card>

```

```

<CardHeader>
  <CardTitle>Check-ins Pendentes ({bookings.length})</CardTitle>
</CardHeader>
<CardContent>
  {bookings.length === 0 ? (
    <p className="text-muted-foreground text-center py-8">
      Nenhum check-in pendente
    </p>
  ) : (
    <div className="space-y-4">
      {bookings.map((booking) => (
        <div
          key={booking.bookingId}
          className="border rounded-lg p-4 space-y-3"
        >
          <div>
            <p className="font-semibold">{booking.hotelName}</p>
            <p className="text-sm text-muted-foreground">
              {booking.roomType === "Single" ? "Individual" : "Duplo"}
            </p>
          </div>

          <div className="bg-muted/50 rounded-md p-3">
            <p className="text-sm font-medium">Hóspede</p>
            <p className="font-semibold">{booking.guestName}</p>
            <p className="text-xs text-muted-foreground">
              {booking.guestEmail}
            </p>
          </div>

          <div className="text-sm">
            <p>
              Check-in: {" "}
              {new Date(booking.checkInDate).toLocaleDateString("pt-PT")}
            </p>
            <p>
              Check-out: {" "}
              {new Date(booking.checkOutDate).toLocaleDateString("pt-PT")}
            </p>
          </div>

          <Button
            onClick={() => handleCheckIn(booking.bookingId)}
            disabled={processingId === booking.bookingId}
            className="w-full"
          >
            {processingId === booking.bookingId ? (
              <Loader2 className="h-4 w-4 animate-spin mr-2" />
            ) : (
              <LogIn className="h-4 w-4 mr-2" />
            )}
            Fazer Check-in
          </Button>
        </div>
      )})
    </div>
  )}
</CardContent>
</Card>

```

```
);  
}
```

checkoutManagement.tsx

```
"use client";  
  
import React, { useState, useEffect } from "react";  
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";  
import { Button } from "@components/ui/button";  
import { Loader2, Logout } from "lucide-react";  
import {  
  getPartnerActiveStays,  
  checkOutBooking,  
} from "@lib/actions/controlpanel.hotels.actions";  
import { toast } from "sonner";  
  
interface Booking {  
  bookingId: string;  
  checkInDate: string;  
  checkOutDate: string;  
  roomType: string;  
  hotelName: string;  
  guestName: string;  
  guestEmail: string;  
}  
  
export function CheckOutManagement() {  
  const [bookings, setBookings] = useState<Booking[]>([]);  
  const [loading, setLoading] = useState(true);  
  const [processingId, setProcessingId] = useState<string | null>(null);  
  
  const loadBookings = async () => {  
    setLoading(true);  
    const result = await getPartnerActiveStays();  
    if (result.success && result.bookings) {  
      setBookings(result.bookings);  
    }  
    setLoading(false);  
  };  
  
  useEffect(() => {  
    loadBookings();  
  }, []);  
  
  const handleCheckOut = async (bookingId: string) => {  
    setProcessingId(bookingId);  
    const result = await checkOutBooking(bookingId);  
    if (result.success) {  
      toast.success("Check-out realizado com sucesso!");  
      loadBookings();  
    } else {  
      toast.error(result.error || "Erro ao fazer check-out");  
    }  
    setProcessingId(null);  
  };  
}
```

```

if (loading) {
  return (
    <Card>
      <CardHeader>
        <CardTitle>Check-outs Pendentes</CardTitle>
      </CardHeader>
      <CardContent>
        <div className="flex justify-center py-8">
          <Loader2 className="h-8 w-8 animate-spin" />
        </div>
      </CardContent>
    </Card>
  );
}

return (
  <Card>
    <CardHeader>
      <CardTitle>Check-outs Pendentes ({bookings.length})</CardTitle>
    </CardHeader>
    <CardContent>
      {bookings.length === 0 ? (
        <p className="text-muted-foreground text-center py-8">
          Nenhum check-out pendente
        </p>
      ) : (
        <div className="space-y-4">
          {bookings.map((booking) => (
            <div>
              key={booking.bookingId}
              className="border rounded-lg p-4 space-y-3"
            >
              <div>
                <p className="font-semibold">{booking.hotelName}</p>
                <p className="text-sm text-muted-foreground">
                  {booking.roomType === "Single" ? "Individual" : "Duplo"}
                </p>
              </div>
              <div className="bg-muted/50 rounded-md p-3">
                <p className="text-sm font-medium">Hóspede</p>
                <p className="font-semibold">{booking.guestName}</p>
                <p className="text-xs text-muted-foreground">
                  {booking.guestEmail}
                </p>
              </div>
              <div className="text-sm">
                <p>
                  Check-in: {" "}
                  {new Date(booking.checkInDate).toLocaleDateString("pt-PT")}
                </p>
                <p>
                  Check-out: {" "}
                  {new Date(booking.checkOutDate).toLocaleDateString("pt-PT")}
                </p>
              </div>
            </div>
          )}
        </div>
      )}
    </CardContent>
    <Button

```

```

        onClick={() => handleCheckOut(booking.bookingId)}
        disabled={processingId === booking.bookingId}
        className="w-full"
        variant="destructive"
      >
        {processingId === booking.bookingId ? (
          <Loader2 className="h-4 w-4 animate-spin mr-2" />
        ) : (
          <LogOut className="h-4 w-4 mr-2" />
        )}
        Fazer Check-out
      </Button>
    </div>
  )}
</div>
)}
</CardContent>
</Card>
);
}

```

guestList.tsx

```

"use client";

import React, { useState, useEffect } from "react";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Button } from "@components/ui/button";
import { Loader2, LogOut } from "lucide-react";
import {
  getPartnerActiveStays,
  checkOutBooking,
} from "@lib/actions/controlpanel.hotels.actions";
import { toast } from "sonner";

interface Booking {
  bookingId: string;
  checkInDate: string;
  checkOutDate: string;
  roomType: string;
  hotelName: string;
  guestName: string;
  guestEmail: string;
}

export function CheckOutManagement() {
  const [bookings, setBookings] = useState<Booking[]>([]);
  const [loading, setLoading] = useState(true);
  const [processingId, setProcessingId] = useState<string | null>(null);

  const loadBookings = async () => {
    setLoading(true);
    const result = await getPartnerActiveStays();
    if (result.success && result.bookings) {
      setBookings(result.bookings);
    }
  };

```



```

    setLoading(false);
  };

  useEffect(() => {
    loadBookings();
  }, []);

  const handleCheckOut = async (bookingId: string) => {
    setProcessingId(bookingId);
    const result = await checkOutBooking(bookingId);
    if (result.success) {
      toast.success("Check-out realizado com sucesso!");
      loadBookings();
    } else {
      toast.error(result.error || "Erro ao fazer check-out");
    }
    setProcessingId(null);
  };

  if (loading) {
    return (
      <Card>
        <CardHeader>
          <CardTitle>Check-outs Pendentes</CardTitle>
        </CardHeader>
        <CardContent>
          <div className="flex justify-center py-8">
            <Loader2 className="h-8 w-8 animate-spin" />
          </div>
        </CardContent>
      </Card>
    );
  }

  return (
    <Card>
      <CardHeader>
        <CardTitle>Check-outs Pendentes ({bookings.length})</CardTitle>
      </CardHeader>
      <CardContent>
        {bookings.length === 0 ? (
          <p className="text-muted-foreground text-center py-8">
            Nenhum check-out pendente
          </p>
        ) : (
          <div className="space-y-4">
            {bookings.map((booking) => (
              <div
                key={booking.bookingId}
                className="border rounded-lg p-4 space-y-3"
              >
                <div>
                  <p className="font-semibold">{booking.hotelName}</p>
                  <p className="text-sm text-muted-foreground">
                    {booking.roomType === "Single" ? "Individual" : "Duplo"}
                  </p>
                </div>

                <div className="bg-muted/50 rounded-md p-3">
                  <p className="text-sm font-medium">Hóspede</p>
                </div>
              </div>
            ))}
          </div>
        )}
      </CardContent>
    </Card>
  );

```

```

        <p className="font-semibold">{booking.guestName}</p>
        <p className="text-xs text-muted-foreground">
          {booking.guestEmail}
        </p>
      </div>

      <div className="text-sm">
        <p>
          Check-in: {" "}
          {new Date(booking.checkInDate).toLocaleDateString("pt-PT")}
        </p>
        <p>
          Check-out: {" "}
          {new Date(booking.checkOutDate).toLocaleDateString("pt-PT")}
        </p>
      </div>

      <Button
        onClick={() => handleCheckOut(booking.bookingId)}
        disabled={processingId === booking.bookingId}
        className="w-full"
        variant="destructive"
      >
        {processingId === booking.bookingId ? (
          <Loader2 className="h-4 w-4 animate-spin mr-2" />
        ) : (
          <LogOut className="h-4 w-4 mr-2" />
        )}
        Fazer Check-out
      </Button>
    </div>
  )))
</div>
))
</CardContent>
</Card>
);
}

```

partnerBookingList.tsx

```

"use client";

import React, { useState, useEffect } from "react";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Badge } from "@components/ui/badge";
import { Loader2 } from "lucide-react";
import { getAllPartnerGuests } from "@lib/actions/controlpanel.hotels.actions";

interface Guest {
  guestName: string;
  guestEmail: string;
  hotelName: string;
  isStaying: boolean;
}

```

```

export function GuestsList() {
  const [guests, setGuests] = useState<Guest[]>([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const loadGuests = async () => {
      const result = await getAllPartnerGuests();
      if (result.success && result.guests) {
        setGuests(result.guests);
      }
      setLoading(false);
    };
    loadGuests();
  }, []);

  if (loading) {
    return (
      <Card>
        <CardContent className="pt-6">
          <div className="flex justify-center py-8">
            <Loader2 className="h-8 w-8 animate-spin" />
          </div>
        </CardContent>
      </Card>
    );
  }

  return (
    <Card className="max-w-7xl mx-auto">
      <CardHeader>
        <CardTitle>Hóspedes ( {guests.length} )</CardTitle>
      </CardHeader>
      <CardContent>
        {guests.length === 0 ? (
          <p className="text-muted-foreground text-center py-8">
            Nenhum hóspede
          </p>
        ) : (
          <div className="space-y-3">
            {guests.map((guest, index) => (
              <div
                key={index}
                className="border rounded-lg p-4 flex items-center justify-between"
              >
                <div>
                  <p className="font-semibold">{guest.guestName}</p>
                  <p className="text-sm text-muted-foreground">
                    {guest.guestEmail}
                  </p>
                  <p className="text-sm text-muted-foreground mt-1">
                    Hotel: {guest.hotelName}
                  </p>
                </div>
                <Badge variant={guest.isStaying ? "default" : "secondary"}>
                  {guest.isStaying ? "Hospedado" : "Não Hospedado"}
                </Badge>
              </div>
            ))}
          </div>
        )}
      </CardContent>
    </Card>
  );
}

```

```

    </CardContent>
  </Card>
);
}

```

partnerBookingList.tsx

```

"use client";

import React from "react";

interface PartnerBookingsListProps {
  bookings: Booking[];
}

export function PartnerBookingsList({ bookings }: PartnerBookingsListProps) {
  const getStatusStyles = (status: string) => {
    switch (status) {
      case "Confirmed":
        return {
          className:
            "bg-green-100 text-green-800 dark:bg-green-900/30 dark:text-green-300",
          label: "Confirmado",
        };
      case "Pending":
        return {
          className:
            "bg-yellow-100 text-yellow-800 dark:bg-yellow-900/30 dark:text-yellow-300",
          label: "Pendente",
        };
      case "Cancelled":
        return {
          className:
            "bg-red-100 text-red-800 dark:bg-red-900/30 dark:text-red-300",
          label: "Cancelado",
        };
      case "Completed":
        return {
          className:
            "bg-blue-100 text-blue-800 dark:bg-blue-900/30 dark:text-blue-300",
          label: "Concluído",
        };
      default:
        return {
          className:
            "bg-gray-100 text-gray-800 dark:bg-gray-800 dark:text-gray-300",
          label: "Desconhecido",
        };
    }
  };

  if (bookings.length === 0) {
    return (
      <p className="text-center text-muted-foreground">

```

```

    Nenhuma reserva encontrada.
  </p>
  );
}

return (
  <div className="grid gap-4">
    {bookings.map((booking) => (
      <div
        key={booking.bookingId}
        className="rounded-lg border border-border bg-card p-4 shadow-sm transition-shadow hover:shadow-
md"
      >
        <div className="mb-3 flex items-start justify-between">
          <h3 className="text-lg font-semibold text-card-foreground">
            {booking.hotelName}
          </h3>
          <span
            className={`inline-block rounded-md px-2.5 py-1 text-xs font-medium ${
              getStatusStyles(booking.status).className
            }`}
          >
            {getStatusStyles(booking.status).label}
          </span>
        </div>

        <div className="grid gap-3 text-sm">
          <div className="grid grid-cols-1 gap-3 md:grid-cols-2">
            <div className="text-muted-foreground">
              <strong className="text-foreground">Cliente:</strong>{" "}
              {booking.userFullName}
            </div>
            <div className="text-muted-foreground">
              <strong className="text-foreground">Email:</strong>{" "}
              {booking.userEmail}
            </div>
          </div>

          <div className="grid grid-cols-1 gap-3 md:grid-cols-2">
            <div className="text-muted-foreground">
              <strong className="text-foreground">Check-in:</strong>{" "}
              {booking.checkIn}
            </div>
            <div className="text-muted-foreground">
              <strong className="text-foreground">Check-out:</strong>{" "}
              {booking.checkOut}
            </div>
          </div>

          <div className="grid grid-cols-1 gap-3 md:grid-cols-2">
            <div className="text-muted-foreground">
              <strong className="text-foreground">Quarto:</strong>{" "}
              {booking.roomType === "Single" ? "Individual" : "Duplo"}
            </div>
            <div className="text-muted-foreground">
              <strong className="text-foreground">Preço Total:</strong> €
              {booking.totalPrice}
            </div>
          </div>
        </div>
      </div>
    )})
  </div>

```

```

    </div>
  )}
</div>
);
}

```

PartnerHotelsList.tsx

```

"use client";

import * as React from "react";
import {
  Card,
  CardHeader,
  CardTitle,
  CardDescription,
} from "@components/ui/card";
import { Button } from "@components/ui/button";
import { toast } from "sonner";
import { deleteHotel } from "@lib/actions/controlpanel.hotels.actions";

type Hotel = {
  id: string;
  name: string;
  city: string;
  country: string | null;
};

export function UserHotelsList({ hotels }: { hotels: Hotel[] }) {
  const [items, setItems] = React.useState(hotels);
  const [loadingId, setLoadingId] = React.useState<string | null>(null);

  const handleDelete = async (hotelId: string) => {
    if (!confirm("Tens a certeza que queres apagar este hotel?")) return;

    setLoadingId(hotelId);

    const res = await deleteHotel(hotelId);

    if (res.success) {
      toast.success("Hotel removido com sucesso");
      setItems((prev) => prev.filter((h) => h.id !== hotelId));
    } else {
      toast.error(res.error || "Não foi possível remover o hotel");
    }

    setLoadingId(null);
  };

  if (!items.length) {
    return <p className="text-muted-foreground">Ainda não tens hotéis.</p>;
  }

  return (
    <div className="w-full space-y-3 lg:max-w-3xl lg:mr-auto">
      {items.map((hotel) => (
        <Card
          key={hotel.id}

```

```

        className="
          flex flex-col gap-3 p-4
          sm:flex-row sm:items-center sm:justify-between
        "
      >
        <CardHeader className="p-0">
          <CardTitle className="text-base capitalize whitespace-nowrap">
            {hotel.name}
          </CardTitle>
          <CardDescription className="capitalize whitespace-nowrap">
            {hotel.city}, {hotel.country}
          </CardDescription>
        </CardHeader>

        <Button
          variant="destructive"
          size="sm"
          disabled={loadingId === hotel.id}
          onClick={() => handleDelete(hotel.id)}
          className="w-full sm:w-auto"
        >
          {loadingId === hotel.id ? "A remover..." : "Remover"}
        </Button>
      </Card>
    ))}
  </div>
);
}

```

B.1.4 Componentes da Dashboard de Administrador

adminHotelsList.tsx

```

"use client";

import * as React from "react";
import {
  Card,
  CardHeader,
  CardTitle,
  CardDescription,
} from "@components/ui/card";
import { Button } from "@components/ui/button";
import { toast } from "sonner";
import { deleteHotel } from "@lib/actions/controlpanel.hotels.actions";

type Hotel = {
  id: string;
  name: string;
  city: string;
  country: string | null;
};

```

```

export function UserHotelsList({ hotels }: { hotels: Hotel[] }) {
  const [items, setItems] = React.useState(hotels);
  const [loadingId, setLoadingId] = React.useState<string | null>(null);

  const handleDelete = async (hotelId: string) => {
    if (!confirm("Tens a certeza que queres apagar este hotel?")) return;

    setLoadingId(hotelId);

    const res = await deleteHotel(hotelId);

    if (res.success) {
      toast.success("Hotel removido com sucesso");
      setItems((prev) => prev.filter((h) => h.id !== hotelId));
    } else {
      toast.error(res.error || "Não foi possível remover o hotel");
    }

    setLoadingId(null);
  };

  if (!items.length) {
    return <p className="text-muted-foreground">Ainda não tens hotéis.</p>;
  }

  return (
    <div className="w-full space-y-3 lg:max-w-3xl lg:mr-auto">
      {items.map((hotel) => (
        <Card
          key={hotel.id}
          className="
            flex flex-col gap-3 p-4
            sm:flex-row sm:items-center sm:justify-between
          "
        >
          <CardHeader className="p-0">
            <CardTitle className="text-base capitalize whitespace-nowrap">
              {hotel.name}
            </CardTitle>
            <CardDescription className="capitalize whitespace-nowrap">
              {hotel.city}, {hotel.country}
            </CardDescription>
          </CardHeader>

          <Button
            variant="destructive"
            size="sm"
            disabled={loadingId === hotel.id}
            onClick={() => handleDelete(hotel.id)}
            className="w-full sm:w-auto"
          >
            {loadingId === hotel.id ? "A remover..." : "Remover"}
          </Button>
        </Card>
      ))}
    </div>
  );
}

```


dashboardMetrics.tsx

```
"use client";

import * as React from "react";
import {
  Card,
  CardHeader,
  CardTitle,
  CardDescription,
} from "@components/ui/card";
import { Button } from "@components/ui/button";
import { toast } from "sonner";
import { deleteHotel } from "@lib/actions/controlpanel.hotels.actions";

type Hotel = {
  id: string;
  name: string;
  city: string;
  country: string | null;
};

export function UserHotelsList({ hotels }: { hotels: Hotel[] }) {
  const [items, setItems] = React.useState(hotels);
  const [loadingId, setLoadingId] = React.useState<string | null>(null);

  const handleDelete = async (hotelId: string) => {
    if (!confirm("Tens a certeza que queres apagar este hotel?")) return;

    setLoadingId(hotelId);

    const res = await deleteHotel(hotelId);

    if (res.success) {
      toast.success("Hotel removido com sucesso");
      setItems((prev) => prev.filter((h) => h.id !== hotelId));
    } else {
      toast.error(res.error || "Não foi possível remover o hotel");
    }

    setLoadingId(null);
  };

  if (!items.length) {
    return <p className="text-muted-foreground">Ainda não tens hotéis.</p>;
  }

  return (
    <div className="w-full space-y-3 lg:max-w-3xl lg:mr-auto">
      {items.map((hotel) => (
        <Card
          key={hotel.id}
          className="
            flex flex-col gap-3 p-4
            sm:flex-row sm:items-center sm:justify-between
          "
        >

```

```

    <CardHeader className="p-0">
      <CardTitle className="text-base capitalize whitespace-nowrap">
        {hotel.name}
      </CardTitle>
      <CardDescription className="capitalize whitespace-nowrap">
        {hotel.city}, {hotel.country}
      </CardDescription>
    </CardHeader>

    <Button
      variant="destructive"
      size="sm"
      disabled={loadingId === hotel.id}
      onClick={() => handleDelete(hotel.id)}
      className="w-full sm:w-auto"
    >
      {loadingId === hotel.id ? "A remover..." : "Remover"}
    </Button>
  </Card>
)
</div>
);
}

```

B.1.5 Componentes da Navbar e Footer

sidemenu.tsx

```

import Link from "next/link";
import { Session } from "next-auth";
import { ChevronUp, User2 } from "lucide-react";

import {
  Sidebar,
  SidebarContent,
  SidebarFooter,
  SidebarGroup,
  SidebarGroupContent,
  SidebarGroupLabel,
  SidebarMenu,
  SidebarMenuButton,
  SidebarMenuItem,
} from "@components/ui/sidebar";

import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuTrigger,
} from "@components/ui/dropdown-menu";

import { ModeToggle } from "@components/theme/theme-toggle";
import { SignOutButton } from "@components/auth/SignOutButton";

```

```

import { sideMenuConfig } from "./side-menu.config";
import { CartMenuItem } from "@components/cart/cart-menu-item";

export type SideMenuVariant = keyof typeof sideMenuConfig;

type SideMenuProps = {
  session: Session | null;
  variant: SideMenuVariant;
};

export function SideMenu({ session, variant }: SideMenuProps) {
  const config = sideMenuConfig[variant];

  return (
    <Sidebar className={config.colorClass}>
      <SidebarContent>
        <SidebarGroup>
          <div className="flex w-full items-center justify-between px-2">
            <SidebarGroupLabel>{config.label}</SidebarGroupLabel>
            <ModeToggle />
          </div>

          <SidebarGroupContent>
            <SidebarMenu>
              {config.menu.map((item) => (
                <SidebarMenuItem key={item.title}>
                  {item.title === "Carrinho" ? (
                    <CartMenuItem />
                  ) : (
                    <SidebarMenuButton asChild>
                      <Link href={item.url}>
                        <item.icon />
                        <span>{item.title}</span>
                      </Link>
                    </SidebarMenuButton>
                  )}
                </SidebarMenuItem>
              ))}
            </SidebarMenu>
          </SidebarGroupContent>

          {config.help.length > 0 && (
            <
              <SidebarGroupLabel>Ajuda</SidebarGroupLabel>
              <SidebarMenu>
                {config.help.map((item) => (
                  <SidebarMenuItem key={item.title}>
                    <SidebarMenuButton asChild>
                      <Link href={item.url}>
                        <item.icon />
                        <span>{item.title}</span>
                      </Link>
                    </SidebarMenuButton>
                  </SidebarMenuItem>
                ))}
              </SidebarMenu>
            </>
          )}
        </SidebarGroup>
      </SidebarContent>
    </>
  );
}

```

```

<SidebarFooter>
  <SidebarMenu>
    <SidebarMenuItem>
      <DropdownMenu>
        <DropdownMenuTrigger asChild>
          <SidebarMenuButton>
            <User2 />
            {session && (
              <span className="capitalize">{session.user?.name}</span>
            )}
            <ChevronUp className="ml-auto" />
          </SidebarMenuButton>
        </DropdownMenuTrigger>

        <DropdownMenuContent side="top">
          <DropdownMenuItem asChild>
            <Link href="/discover/my-account">Conta</Link>
          </DropdownMenuItem>
          <SignOutButton variant="dropdown" />
        </DropdownMenuContent>
      </DropdownMenu>
    </SidebarMenuItem>
  </SidebarMenu>
</SidebarFooter>
</Sidebar>
);
}

```

side-menu.config.ts

```

import {
  Calendar,
  Home,
  HeartPlus,
  Info,
  HeartHandshake,
  LayoutDashboard,
  Building2,
  FileText,
  ShoppingCart,
} from "lucide-react";

export const sideMenuConfig = {
  discover: {
    label: "StayNet",
    colorClass: "bg-background",
    menu: [
      { title: "Início", url: "/discover", icon: Home },
      { title: "Carrinho", url: "/cart", icon: ShoppingCart },
      { title: "Minhas Reservas", url: "/discover/bookings", icon: Calendar },
      { title: "Favoritos", url: "/discover/favorites", icon: HeartPlus },
      {
        title: "Torne-se Parceiro",
        url: "/discover/partner",
        icon: HeartHandshake,
      },
    ],
  },
};

```

```

    },
  ],
  help: [{ title: "Quem nós somos", url: "/discover/about", icon: Info }],
},

controlpanel: {
  label: "Painel de Parceiros",
  colorClass: "text-blue-400",
  menu: [
    { title: "Visão Geral", url: "/controlpanel", icon: LayoutDashboard },
    {
      title: "Adicionar Hotel",
      url: "/controlpanel/create-hotel",
      icon: Building2,
    },
    { title: "Hoteis", url: "/controlpanel/hotel-list", icon: FileText },
    {
      title: "Reservas",
      url: "/controlpanel/hotel-bookings",
      icon: Calendar,
    },
    { title: "Hóspedes", url: "/controlpanel/guest-list", icon: FileText },
    {
      title: "Check-in/out",
      url: "/controlpanel/check-in-out",
      icon: FileText,
    },
  ],
  help: [],
},
dashboard: {
  label: "Painel de Administrador",
  colorClass: "text-blue-400",
  menu: [
    { title: "Visão Geral", url: "/dashboard", icon: LayoutDashboard },
    {
      title: "Parceiros",
      url: "/dashboard/partners",
      icon: HeartHandshake,
    },
    {
      title: "Hoteis",
      url: "/dashboard/hoteis",
      icon: Building2,
    },
  ],
  help: [],
},
} as const;

```

Footer.tsx

```

import Link from "next/link";
import { Instagram, Linkedin } from "lucide-react";

export default function Footer() {
  return (
    <footer className="mt-16 w-full bg-zinc-900 text-zinc-200">

```

```

<div className="mx-auto max-w-6xl px-6 py-12">
  <div className="grid grid-cols-1 gap-8 md:grid-cols-4">
    <div className="md:col-span-1">
      <div className="text-2xl font-bold tracking-wide">
        STAY<span className="text-zinc-400">net</span>
      </div>
    </div>

    <div className="md:col-span-1">
      <h3 className="text-sm font-semibold text-zinc-100 mb-2">
        Contactos
      </h3>
      <ul className="space-y-1 text-sm text-zinc-300">
        <li>
          <a
            className="hover:text-zinc-100 transition-colors"
            href="mailto:geral@email.com"
          >
            geral@email.com
          </a>
        </li>
        <li>
          <a
            className="hover:text-zinc-100 transition-colors"
            href="tel:+351967777777"
          >
            +351 967 777 777
          </a>
        </li>
      </ul>
    </div>

    <div className="md:col-span-1">
      <h3 className="text-sm font-semibold text-zinc-100 mb-2">
        Links Úteis
      </h3>
      <ul className="space-y-1 text-sm text-zinc-300">
        <li>
          <Link
            className="hover:text-zinc-100 transition-colors"
            href="/sign-in"
          >
            Iniciar Sessão
          </Link>
        </li>
        <li>
          <Link
            className="hover:text-zinc-100 transition-colors"
            href="/sign-up"
          >
            Criar Conta
          </Link>
        </li>
        <li>
          <Link
            className="hover:text-zinc-100 transition-colors"
            href="/discover/partner"
          >

```

```

        Torne-se um parceiro
      </Link>
    </li>
  </ul>
</div>

<div className="md:col-span-1 flex md:justify-end">
  <div>
    <h3 className="text-sm font-semibold text-zinc-100 mb-2 md:text-right">
      Social
    </h3>
    <div className="flex gap-3 md:justify-end">
      <a
        className="rounded-md p-2 text-zinc-300 hover:bg-zinc-800 hover:text-zinc-100 transition-all"
        href="https://instagram.com"
        target="_blank"
        rel="noreferrer"
        aria-label="Instagram"
      >
        <Instagram className="h-5 w-5" />
      </a>
      <a
        className="rounded-md p-2 text-zinc-300 hover:bg-zinc-800 hover:text-zinc-100 transition-all"
        href="https://linkedin.com"
        target="_blank"
        rel="noreferrer"
        aria-label="LinkedIn"
      >
        <Linkedin className="h-5 w-5" />
      </a>
    </div>
  </div>
</div>

<div className="mt-10 border-t border-zinc-800 pt-4 text-center text-xs text-zinc-400">
  © {new Date().getFullYear()} StayNet. Todos os direitos reservados.
</div>
</div>
</footer>
);
}

```

B.1.6 Componentes da visualização de Mapas

Map.tsx

```

"use client";

import { useEffect, useState } from "react";
import { MapContainer, TileLayer, Marker, Popup } from "react-leaflet";
import { LatLngTuple } from "leaflet";
import { geocodeLocation } from "@lib/actions/map.action";

```

```

import "leaflet/dist/leaflet.css";
import "leaflet-defaulticon-compatibility/dist/leaflet-defaulticon-compatibility.css";
import "leaflet-defaulticon-compatibility";

interface MapProps {
  location: string;
  zoom?: number;
}

const Map = ({ location, zoom = 19 }: MapProps) => {
  const [position, setPosition] = useState<LatLngTuple | null>(null);
  const [error, setError] = useState(false);

  useEffect(() => {
    const loadLocation = async () => {
      try {
        const coords = await geocodeLocation(location);

        if (coords) {
          setPosition([coords.lat, coords.lon]);
          setError(false);
        } else {
          setError(true);
        }
      } catch {
        setError(true);
      }
    };

    loadLocation();
  }, [location]);

  if (error) {
    return (
      <p className="flex justify-center items-center text-sm text-center">
        Não foi possível carregar a localização
      </p>
    );
  }

  if (!position) {
    return (
      <p className="text-sm text-gray-500 text-center">A carregar mapa...</p>
    );
  }

  return (
    <MapContainer
      attributionControl={false}
      center={position}
      zoom={zoom}
      scrollWheelZoom={true}
      style={{ height: "100%", width: "100%" }}
    >
      <TileLayer
        url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
        attribution='&copy; <a href="https://www.openstreetmap.org/copyright">OpenStreetMap</a>'

```



```

contributors'
  />
  <Marker position={position}>
    <Popup>{location}</Popup>
  </Marker>
</MapContainer>
);
};

export default Map;

```

B.2 Server Actions

B.2.1 Server Actions Autenticação

```

"use server";

import { db } from "@database/drizzle";
import { users, verificationTokens } from "@database/schema";
import { eq, and } from "drizzle-orm";
import { hash, compare } from "bcryptjs";
import { signIn, signOut } from "@auth";
import { Resend } from "resend";
import crypto from "crypto";
import VerifyEmailTemplate from "@emails/verify-email-template";

const resend = new Resend(process.env.RESEND_API_KEY);

function generateSecureToken(): string {
  return crypto.randomBytes(32).toString("base64url");
}

async function hashToken(token: string): Promise<string> {
  return hash(token, 10);
}

export const signInWithCredentials = async (
  params: Pick<AuthCredentials, "email" | "password">,
) => {
  const { email, password } = params;

  try {
    const result = await signIn("credentials", {
      email,
      password,
      redirect: false,
    });
  }

  if (result?.error) {
    return { success: false, error: result.error };
  }

  return { success: true };
} catch (error) {

```

```

    console.error("Signin error:", error);
    return { success: false, error: "Erro ao iniciar sessão" };
  }
};

export const signUp = async (params: AuthCredentials) => {
  const { fullName, email, password } = params;

  try {
    if (!email || !password || password.length < 6) {
      return { success: false, error: "Dados inválidos" };
    }

    const existingUser = await db
      .select()
      .from(users)
      .where(eq(users.email, email.toLowerCase()))
      .limit(1);

    if (existingUser.length > 0) {
      return { success: false, error: "Este email já está registado" };
    }

    const existingToken = await db
      .select()
      .from(verificationTokens)
      .where(
        and(
          eq(verificationTokens.email, email.toLowerCase()),
          eq(verificationTokens.tokenType, "email_verification"),
        ),
      )
      .limit(1);

    if (existingToken.length > 0) {
      if (new Date() < new Date(existingToken[0].expiresAt)) {
        return {
          success: false,
          error:
            "Já foi enviado um email de verificação. Verifique a sua caixa de entrada.",
        };
      }
    }

    await db
      .delete(verificationTokens)
      .where(eq(verificationTokens.email, email.toLowerCase()));
  }

  const passwordHash = await hash(password, 12);
  const token = generateSecureToken();
  const tokenHash = await hashToken(token);

  const expiresAt = new Date();
  expiresAt.setHours(expiresAt.getHours() + 24);

  await db.insert(verificationTokens).values({
    email: email.toLowerCase(),
    fullName,
  });

```

```

passwordHash,
tokenHash,
tokenType: "email_verification",
expiresAt,
});

const verificationUrl = `${process.env.NEXT_PUBLIC_APP_URL}/verify-email?token=${token}`;

const { error: emailError } = await resend.emails.send({
  from: "StayNet <onboarding@resend.dev>",
  to: [email.toLowerCase()],
  subject: "Verifique o seu email - StayNet",
  react: VerifyEmailTemplate({
    fullName,
    verificationUrl,
  }),
});

if (emailError) {
  console.error("Erro ao enviar email:", emailError);
  await db
    .delete(verificationTokens)
    .where(eq(verificationTokens.email, email.toLowerCase()));

  return {
    success: false,
    error: "Erro ao enviar email. Tente novamente.",
  };
}

return {
  success: true,
  pendingVerification: true,
};
} catch (error) {
  console.error("Signup error:", error);
  return { success: false, error: "Erro ao criar conta" };
}
};

export async function verifyEmailToken(token: string): Promise<{
  success: boolean;
  error?: string;
}> {
  try {
    if (!token || token.length < 20) {
      return { success: false, error: "Token inválido" };
    }

    const allTokens = await db
      .select()
      .from(verificationTokens)
      .where(eq(verificationTokens.tokenType, "email_verification"));

    let matchedToken = null;
    for (const tokenData of allTokens) {
      const isMatch = await compare(token, tokenData.tokenHash);
      if (isMatch) {
        matchedToken = tokenData;
        break;
      }
    }
  }
}

```

```

    }
  }

  if (!matchedToken) {
    return {
      success: false,
      error: "Token inválido ou expirado",
    };
  }

  if (new Date() > new Date(matchedToken.expiresAt)) {
    await db
      .delete(verificationTokens)
      .where(eq(verificationTokens.email, matchedToken.email));

    return {
      success: false,
      error: "Token expirado. Registe-se novamente.",
    };
  }

  const existingUser = await db
    .select()
    .from(users)
    .where(eq(users.email, matchedToken.email))
    .limit(1);

  if (existingUser.length > 0) {
    await db
      .delete(verificationTokens)
      .where(eq(verificationTokens.email, matchedToken.email));

    return {
      success: false,
      error: "Este email já está registado",
    };
  }

  await db.insert(users).values({
    fullName: matchedToken.fullName!,
    email: matchedToken.email,
    password: matchedToken.passwordHash!,
    role: "User",
    emailVerified: true,
  });

  await db
    .delete(verificationTokens)
    .where(eq(verificationTokens.email, matchedToken.email));

  return {
    success: true,
  };
} catch (error) {
  console.error("Erro ao verificar token:", error);
  return {
    success: false,
    error: "Erro ao verificar email",
  };
}

```

```

    };
  }
}

export async function handleSignOut() {
  await signOut({ redirectTo: "/sign-in" });
}

```

B.2.2 Server Actions Reservas

```

"use server";

import { db } from "@database/drizzle";
import { rooms, bookings, hotels, users } from "@database/schema";
import { eq, and, or, lt, gt, inArray, desc } from "drizzle-orm";
import { auth } from "@auth";

export async function checkRoomAvailability(
  roomId: string,
  checkIn: string,
  checkOut: string,
) {
  const room = await db
    .select()
    .from(rooms)
    .where(eq(rooms.id, roomId))
    .limit(1);

  if (!room[0]) {
    return { available: false, room: null };
  }

  const totalRooms = room[0].totalRooms;

  const existingBookings = await db
    .select()
    .from(bookings)
    .where(
      and(
        eq(bookings.roomId, roomId),
        or(eq(bookings.status, "Confirmed"), eq(bookings.status, "Pending")),
        and(
          lt(bookings.checkInDate, checkOut),
          gt(bookings.checkOutDate, checkIn),
        ),
      ),
    );

  const bookedRooms = existingBookings.length;
  const availableRooms = totalRooms - bookedRooms;

  return {
    available: availableRooms > 0,
    availableRooms,
  };
}

```

```

    totalRooms,
    room: room[0],
  };
}

export async function getUnavailableDates(
  roomId: string,
  startDate: string,
  endDate: string,
) {
  try {
    const room = await db
      .select()
      .from(rooms)
      .where(eq(rooms.id, roomId))
      .limit(1);

    if (!room[0]) {
      return { success: false, error: "Quarto não encontrado" };
    }

    const totalRooms = room[0].totalRooms;

    const existingBookings = await db
      .select()
      .from(bookings)
      .where(
        and(
          eq(bookings.roomId, roomId),
          or(eq(bookings.status, "Confirmed"), eq(bookings.status, "Pending")),
          and(
            lt(bookings.checkInDate, endDate),
            gt(bookings.checkOutDate, startDate),
          ),
        ),
      );

    const dateCounts = new Map<string, number>();

    existingBookings.forEach((booking) => {
      const checkIn = new Date(booking.checkInDate);
      const checkOut = new Date(booking.checkOutDate);

      const current = new Date(checkIn);
      while (current < checkOut) {
        const dateStr = current.toISOString().split("T")[0];
        dateCounts.set(dateStr, (dateCounts.get(dateStr) || 0) + 1);
        current.setDate(current.getDate() + 1);
      }
    });

    const unavailableDates: string[] = [];
    dateCounts.forEach((count, date) => {
      if (count >= totalRooms) {
        unavailableDates.push(date);
      }
    });
  }
}

```

```

    return {
      success: true,
      unavailableDates,
      totalRooms,
    };
  } catch (error) {
    console.error("Erro ao buscar datas indisponíveis:", error);
    return {
      success: false,
      error: "Erro ao buscar datas indisponíveis",
    };
  }
}

export async function calculateBookingPrice(
  roomId: string,
  checkInDate: string,
  checkOutDate: string,
) {
  try {
    const room = await db
      .select()
      .from(rooms)
      .where(eq(rooms.id, roomId))
      .limit(1);

    if (!room[0]) {
      return { success: false, error: "Quarto não encontrado" };
    }

    const checkIn = new Date(checkInDate);
    const checkOut = new Date(checkOutDate);

    if (checkOut <= checkIn) {
      return { success: false, error: "Datas inválidas" };
    }

    const nights = Math.ceil(
      (checkOut.getTime() - checkIn.getTime()) / (1000 * 60 * 60 * 24),
    );
    const pricePerNight = parseFloat(room[0].pricePerNight);
    const total = nights * pricePerNight;

    return {
      success: true,
      nights,
      pricePerNight,
      total: total.toFixed(2),
    };
  } catch (error) {
    console.error("Erro ao calcular preço:", error);
    return { success: false, error: "Erro ao calcular preço" };
  }
}

export async function getUserBookings() {
  try {
    const session = await auth();
    if (!session?.user?.id) {
      return {

```

```

    success: false,
    error: "Utilizador não autenticado",
  };
}

const userBookings = await db
  .select({
    booking: bookings,
    room: rooms,
    hotel: {
      id: hotels.id,
      name: hotels.name,
      city: hotels.city,
      country: hotels.country,
    },
  })
  .from(bookings)
  .innerJoin(rooms, eq(bookings.roomId, rooms.id))
  .innerJoin(hotels, eq(rooms.hotelId, hotels.id))
  .where(eq(bookings.userId, session.user.id))
  .orderBy(desc(bookings.createdAt));

return {
  success: true,
  bookings: userBookings,
};
} catch (error) {
  console.error("Erro ao buscar reservas:", error);
  return {
    success: false,
    error: "Erro ao buscar reservas do utilizador",
  };
}
}

export async function getPartnerBookings() {
  try {
    const session = await auth();
    if (!session?.user?.id) {
      return {
        success: false,
        error: "Utilizador não autenticado",
      };
    }
  }

  const userId = session.user.id;

  const partnerHotels = await db
    .select({
      id: hotels.id,
      name: hotels.name,
    })
    .from(hotels)
    .where(eq(hotels.ownerId, userId));

  if (!partnerHotels.length) {
    return { success: true, bookings: [] };
  }
}

```



```

const hotelIds = partnerHotels.map((h) => h.id);

const bookingsList = await db
  .select({
    bookingId: bookings.id,
    checkIn: bookings.checkInDate,
    checkOut: bookings.checkOutDate,
    totalPrice: bookings.totalPrice,
    status: bookings.status,
    roomId: rooms.id,
    roomType: rooms.roomType,
    hotelId: hotels.id,
    hotelName: hotels.name,
    userFullName: users.fullName,
    userEmail: users.email,
  })
  .from(bookings)
  .innerJoin(rooms, eq(bookings.roomId, rooms.id))
  .innerJoin(hotels, eq(rooms.hotelId, hotels.id))
  .innerJoin(users, eq(bookings.userId, users.id))
  .where(inArray(hotels.id, hotelIds))
  .orderBy(desc(bookings.checkInDate));

return { success: true, bookings: bookingsList };
} catch (error) {
  console.error("Erro ao buscar reservas do parceiro:", error);
  return { success: false, error: "Erro ao buscar reservas do parceiro" };
}
}

```

B.2.3 Server Actions Pagamentos

```

"use server";

import { db } from "@database/drizzle";
import { rooms, bookings, hotels } from "@database/schema";
import { eq } from "drizzle-orm";
import { auth } from "@auth";
import Stripe from "stripe";
import { createTemporaryHold, getActiveHolds, releaseHold } from "@lib/redis";
import { checkRoomAvailability } from "../booking.actions";
import { revalidatePath } from "next/cache";

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!, {
  apiVersion: "2025-12-15.clover",
});

export async function createCheckoutSessionMultiple(
  items: CheckoutData[],
): Promise<CheckoutSessionResponse> {
  try {
    const session = await auth();

    if (!session?.user?.id || !session?.user?.email) {
      return {

```

```

    success: false,
    error: "Você precisa estar autenticado para fazer uma reserva",
  };
}

const userId = session.user.id;
const userEmail = session.user.email;

const holdIds: string[] = [];
const lineItems: Stripe.Checkout.SessionCreateParams.LineItem[] = [];
const itemsMetadata: Record<string, any> = {};

for (let i = 0; i < items.length; i++) {
  const data = items[i];

  const availability = await checkRoomAvailability(
    data.roomId,
    data.checkInDate,
    data.checkOutDate,
  );

  if (!availability.available || !availability.room) {
    for (const holdId of holdIds) {
      await releaseHold(holdId);
    }
    return {
      success: false,
      error: `${data.hotelName} não está disponível para as datas selecionadas`,
    };
  }

  const activeHolds = await getActiveHolds(
    data.roomId,
    data.checkInDate,
    data.checkOutDate,
  );

  const totalAvailable = (availability.availableRooms || 0) - activeHolds;

  if (totalAvailable <= 0) {
    for (const holdId of holdIds) {
      await releaseHold(holdId);
    }
    return {
      success: false,
      error: `${data.hotelName} temporariamente reservado. Tente novamente em alguns minutos.`,
    };
  }

  const holdId = await createTemporaryHold(
    data.roomId,
    data.checkInDate,
    data.checkOutDate,
    userId,
  );
  holdIds.push(holdId);

  const hotel = await db

```

```

        .select({ name: hotels.name })
        .from(hotels)
        .innerJoin(rooms, eq(rooms.hotelId, hotels.id))
        .where(eq(rooms.id, data.roomId))
        .limit(1);

const hotelName = hotel[0]?.name || data.hotelName || "Hotel";

lineItems.push({
  price_data: {
    currency: "eur",
    product_data: {
      name: `${hotelName} - ${
        data.roomType === "Single" ? "Quarto Individual" : "Quarto Duplo"
      }`,
      description: `${data.nights} noite${data.nights > 1 ? "s" : ""} (${new Date(
        data.checkInDate,
      ).toLocaleDateString("pt-PT")} - ${new Date(
        data.checkOutDate,
      ).toLocaleDateString("pt-PT")})`,
    },
    unit_amount: Math.round(parseFloat(data.totalPrice) * 100),
  },
  quantity: 1,
});

itemsMetadata[`item_${i}_roomId`] = data.roomId;
itemsMetadata[`item_${i}_checkIn`] = data.checkInDate;
itemsMetadata[`item_${i}_checkOut`] = data.checkOutDate;
itemsMetadata[`item_${i}_specialRequests`] = data.specialRequests || "";
itemsMetadata[`item_${i}_holdId`] = holdId;
}

const checkoutSession = await stripe.checkout.sessions.create({
  payment_method_types: ["card"],
  line_items: lineItems,
  mode: "payment",
  success_url:
    `${process.env.NEXT_PUBLIC_APP_URL}/cart/success?session_id={CHECKOUT_SESSION_ID}`,
  cancel_url: `${process.env.NEXT_PUBLIC_APP_URL}/cart/cancelled`,
  customer_email: userEmail,
  client_reference_id: userId,
  metadata: {
    userId,
    itemCount: items.length.toString(),
    holdIds: holdIds.join(","),
    ...itemsMetadata,
  },
  expires_at: Math.floor(Date.now() / 1000) + 1800,
});

return {
  success: true,
  sessionId: checkoutSession.id,
  url: checkoutSession.url || undefined,
  holdId: holdIds.join(","),
};
} catch (error) {
  console.error("Erro ao criar sessão de checkout:", error);
  return {

```

```

    success: false,
    error:
      error instanceof Error
      ? error.message
      : "Erro ao criar sessão de pagamento",
  };
}
}

export async function confirmBookingPayment(
  sessionId: string,
): Promise<PaymentStatus> {
  try {
    const checkoutSession = await stripe.checkout.sessions.retrieve(sessionId);

    if (checkoutSession.payment_status !== "paid") {
      return { paid: false };
    }

    const metadata = checkoutSession.metadata;
    if (!metadata) {
      throw new Error("sessão não encontrada");
    }

    const itemCount = parseInt(metadata.itemCount || "1");

    const bookingIds: string[] = [];

    for (let i = 0; i < itemCount; i++) {
      const roomId = metadata[`item_${i}_roomId`];
      const checkIn = metadata[`item_${i}_checkIn`];
      const checkOut = metadata[`item_${i}_checkOut`];
      const holdId = metadata[`item_${i}_holdId`];

      if (!roomId || !checkIn || !checkOut) {
        console.error(`Item ${i} incompleto no metadata`);
        continue;
      }

      const existingBooking = await db
        .select()
        .from(bookings)
        .where(eq(bookings.stripeSessionId, sessionId))
        .limit(1);

      if (existingBooking.length === 0) {
        const totalAmount = checkoutSession.amount_total! / 100;
        const itemPrice = (totalAmount / itemCount).toFixed(2);

        const bookingResult = await db
          .insert(bookings)
          .values({
            userId: metadata.userId,
            roomId: roomId,
            checkInDate: checkIn,
            checkOutDate: checkOut,
            totalPrice: itemPrice,
            status: "Confirmed",
          });
      }
    }
  }
}

```

```

        stripeSessionId: sessionId,
        stripePaymentIntentId: checkoutSession.payment_intent as string,
        paidAt: new Date(),
    })
    .returning();

    bookingIds.push(bookingResult[0].id);
  } else {
    bookingIds.push(existingBooking[0].id);
  }

  if (holdId) {
    await releaseHold(holdId);
  }
}

if (metadata.holdIds) {
  const holdIds = metadata.holdIds.split(",");
  for (const holdId of holdIds) {
    await releaseHold(holdId);
  }
}

revalidatePath("/discover/bookings");

return {
  paid: true,
  bookingId: bookingIds[0],
  status: "Confirmed",
};
} catch (error) {
  console.error("Erro ao confirmar pagamento:", error);
  return { paid: false };
}
}

```

B.2.4 Server Actions dos Quartos

```

"use server";

import { db } from "@database/drizzle";
import { rooms } from "@database/schema";
import { revalidatePath } from "next/cache";
import { eq, and } from "drizzle-orm";

function validatePrice(price: string | number): number {
  const numPrice = typeof price === "string" ? parseFloat(price) : price;
  if (!Number.isFinite(numPrice) || numPrice <= 0) {
    throw new Error("Preço inválido");
  }
  return numPrice;
}

```

```

function validateTotalRooms(total: number): number {
  if (!Number.isInteger(total) || total < 0) {
    throw new Error("Número de quartos inválido");
  }
  return total;
}

export async function configureHotelRooms(data: CreateHotelRoomsInput) {
  try {
    const roomsToInsert = [];

    if (data.singleRoom && data.singleRoom.totalRooms > 0) {
      const price = validatePrice(data.singleRoom.pricePerNight);
      const total = validateTotalRooms(data.singleRoom.totalRooms);

      roomsToInsert.push({
        hotelId: data.hotelId,
        roomType: "Single" as const,
        pricePerNight: price.toFixed(2),
        totalRooms: total,
        description: data.singleRoom.description,
      });
    }

    if (data.doubleRoom && data.doubleRoom.totalRooms > 0) {
      const price = validatePrice(data.doubleRoom.pricePerNight);
      const total = validateTotalRooms(data.doubleRoom.totalRooms);

      roomsToInsert.push({
        hotelId: data.hotelId,
        roomType: "Double" as const,
        pricePerNight: price.toFixed(2),
        totalRooms: total,
        description: data.doubleRoom.description,
      });
    }

    if (roomsToInsert.length === 0) {
      return {
        success: false,
        error: "É obrigatório configurar pelo menos um tipo de quarto",
      };
    }

    const existingRooms = await db
      .select()
      .from(rooms)
      .where(eq(rooms.hotelId, data.hotelId));

    if (existingRooms.length > 0) {
      for (const room of roomsToInsert) {
        const existing = existingRooms.find(
          (r) => r.roomType === room.roomType,
        );

        if (existing) {
          await db
            .update(rooms)

```

```

        .set({
          pricePerNight: room.pricePerNight,
          totalRooms: room.totalRooms,
          description: room.description,
          updatedAt: new Date(),
        })
        .where(eq(rooms.id, existing.id));
    } else {
      await db.insert(rooms).values(room);
    }
  }

  const providedTypes = roomsToInsert.map((r) => r.roomType);
  const typesToRemove = existingRooms
    .filter((r) => !providedTypes.includes(r.roomType))
    .map((r) => r.id);

  if (typesToRemove.length > 0) {
    for (const id of typesToRemove) {
      await db.delete(rooms).where(eq(rooms.id, id));
    }
  } else {
    await db.insert(rooms).values(roomsToInsert);
  }

  revalidatePath(`/hotels/${data.hotelId}`);
  return { success: true };
} catch (error) {
  console.error("Erro ao configurar quartos:", error);
  return {
    success: false,
    error:
      error instanceof Error ? error.message : "Erro ao configurar quartos",
  };
}
}

```

B.2.5 Server Actions Mapa

```

"use server";

export async function geocodeLocation(location: string) {
  if (!location) return null;

  const res = await fetch(
    `https://nominatim.openstreetmap.org/search?format=json&q=${encodeURIComponent(
      location,
    )}`,
    {
      headers: {
        "User-Agent": "staynet-app/1.0 (contact@staynet.pt)",
      },
      cache: "no-store",
    },
  );

  const data = await res.json();
}

```

```

if (!data.length) return null;

return {
  lat: parseFloat(data[0].lat),
  lon: parseFloat(data[0].lon),
};
}

```

B.2.6 Server Actions Carrinho

```

export function getCart(): CartItem[] {
  if (typeof window === "undefined") return [];

  const cartData = sessionStorage.getItem("cart");
  if (!cartData) return [];

  try {
    return JSON.parse(cartData);
  } catch {
    return [];
  }
}

export function saveCart(items: CartItem[]): void {
  if (typeof window === "undefined") return;
  sessionStorage.setItem("cart", JSON.stringify(items));

  window.dispatchEvent(new Event("cartUpdated"));
}

export function addToCart(item: Omit<CartItem, "id" | "addedAt">): CartItem {
  const cart = getCart();

  const newItem: CartItem = {
    ...item,
    id: `${item.roomId}-${Date.now()}`,
    addedAt: Date.now(),
  };

  cart.push(newItem);
  saveCart(cart);

  return newItem;
}

export function removeFromCart(itemId: string): void {
  const cart = getCart();
  const filtered = cart.filter((item) => item.id !== itemId);
  saveCart(filtered);
}

export function clearCart(): void {
  if (typeof window === "undefined") return;

```



```
sessionStorage.removeItem("cart");
window.dispatchEvent(new Event("cartUpdated"));
}

export function getCartTotal(): number {
  const cart = getCart();
  return cart.reduce((total, item) => total + parseFloat(item.totalPrice), 0);
}

export function getCartCount(): number {
  return getCart().length;
}
```